WNDRVR
SUPPORT NETWORK

VXWORKS 7 CLI, WEB, MIBWAY PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 CLI, Web, MIBway Programmer's Guide, SR0650

10 June 2020

1. OVERVIEW

1. Introduction

Wind River® CLI, Wind River Web Server, and Wind River MIBway are software run-time products for VxWorks®.

The management configuration editor (MCE) offers easy to use interfaces that you can use to configure the embedded run-time components and create your device's management interfaces.

Together, these run-time products and development tools allow you to build integrated Web-based and command-line device management solutions that both share a common backplane to access device data. Additionally, when you use Wind River MIBway, your Web-based and command-line interfaces can also access and modify simple network management protocol (SNMP) management data.

CLI, Web Server, and MIBway interface with other Wind River device management products, including Wind River SNMP—Wind River's SNMP agent development toolkit—and custom SOAP- or XML-based management applications using Wind River Web Services.

This document describes the use of the MCE, the structure and use of the target run-time code, details about creating command-line and Web-based management interfaces, and integrating those interfaces with an SNMP agent.

2. Technology Overview

CLI, Web Server, and MIBway allow you to create user interfaces for external or remote device management. Device management consists of configuring, controlling, or monitoring a device.

External or Remote Management

An internal management scheme is one where you physically connect to the device (perhaps with a serial line). Once connected—and only while connected—you can control, configure, or monitor the operation of that device. When the serial line is unplugged, you can no longer control the device or monitor its operation. This type of management has some limitations. One limitation in particular is that you must be physically near the device. In some cases, that may be a reasonable limitation. However, in many cases, the person responsible for the device cannot be physically near the device.

An external management scheme is one that accommodates this limitation. As the name implies, the product's end user can manage the device externally through a network connection and some application layer running over the network. The following sections provide overview information about common types of external management interfaces.

Command-Line Interfaces

A command-line interface (CLI) is a management interface using typed commands entered in an ordinary telnet shell or serial application, such as HyperTerminal. In this example of external management, the end user uses a telnet client to connect to a telnet server and command interpreter embedded within the device. As with web-based management, the person managing the device can be on another part of the local network or the Internet.

Despite the fact that virtually every piece of commercial embedded network equipment provides a CLI, the CLI has often been one of the most poorly engineered aspects of a given device architecture. This poor engineering is often due to the perceived lack of importance. Commands are often designed and implemented in a fashion merely to test a newly developed feature. In short, CLIs are rarely developed with sound engineering principles. They often lack an architecture that is both efficient at run-time, yet easy to modify and extend. They are rarely designed to be interoperable with other management schemes.

Web-Based Interfaces

In web-based management, the person responsible for the device does not need to physically connect to the device. Instead, that person uses a web browser to connect to an HTTP server embedded within the device. The person managing the device can be on another part of the local network or the Internet. web-based management has some key advantages over other common approaches to device management. In a web-based scheme, the only real requirement to remotely manage the device is a standards-compliant web browser. Given that browsers are installed on nearly every PC or workstation, virtually any networked PC or workstation can remotely manage your device.

Another key advantage with web-based management is how the user experience compares with other management schemes. In the example where the person responsible for the device needs to physically connect with a serial cable, device configuration probably consists of entering commands using a console. Again, this is completely valid, and for some people quite useful. It generally requires that the operator has knowledge of the commands and parameters, and is comfortable with managing the device using a console. This might be a reasonable assumption for an IT operator configuring an interior router. However, it is probably an unreasonable assumption for an office manager installing a small office network printer.

With a web-based device management scheme, you can create a richer, more intuitive interface than a command-line interface allows. Using the small office printer example, the office manager can then point and click through the configuration of the printer's network setup without worrying about command usage and syntax. Your screens (web pages) could include recommended procedures, tips, or other types of interactive help that would be difficult, if not impossible, with a command-line interface.

The above discussion does not mean to imply that command-line interfaces are not useful (they are tremendously useful to those who are used to them); it just means that you now have the ability to provide a simpler, more intuitive interface for your device for those users who can benefit from it—just as GUIs have for PCs and workstations.

Java Applet-Based Interfaces

In addition to graphical device management interfaces composed of dynamic web pages, you can also create Java applet-based interfaces with Wind River Web Server. Applets offer you the ability to create rich graphical interfaces that display and update device information in real-time without the need to refresh the web page that contains the applet.

SNMP Interfaces

SNMP is a management paradigm for system administrators to keep track of network devices, the performance of the network, and to diagnose and correct network problems. Around 1990, SNMP was designed by the Internet Advisory Board (IAB) to be a standardized protocol for network device management and to provide interoperability, so that one vendor's devices can be connected with another vendor's devices.

An SNMP interface involves a network administrator using a network management station (NMS) to connect to an SNMP agent embedded within the device. It is similar to the web and command-line examples. However, SNMP is defined by a set of Internet Engineering Task Force (IETF) standards. For more information about SNMP, building SNMP agents, and relevant IETF standards, see the VxWorks 7 SNMP Programmer's Guide.

Although SNMP was initially intended only as a placeholder until a more robust protocol could be devised, SNMP has been quickly and widely adopted by the data communications industry—virtually 100% of shipping network equipment is manageable with SNMP.

3. Product Overview

Wind River CLI, Web, MIBway is a family of device management products that work together through a common device data access model—the backplane.

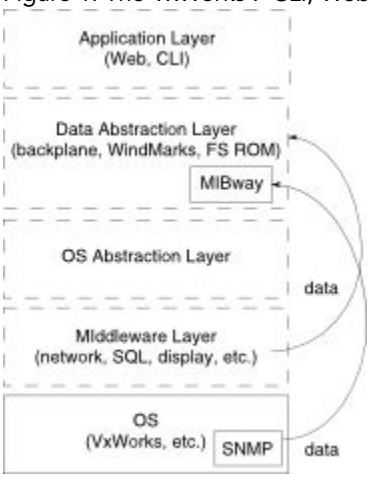
CLI, Web Server, and MIBway are separate products used (independently or together) to create the following types of management interfaces:

- Command-line management interface (CLI)

- CLI integrated with SNMP using MIBway
- CLI integrated with Web and SNMP using MIBway
- Web-based management interface (Web)
- Web integrated with SNMP using MIBway
- Web integrated with XML/SOAP services

VxWorks 7 CLI,Web, MIBway is designed to provide several layers of abstraction, at the top of which sits your web-based or command-line application.

Figure 1. The VxWorks 7 CLI, Web, MIBway Model



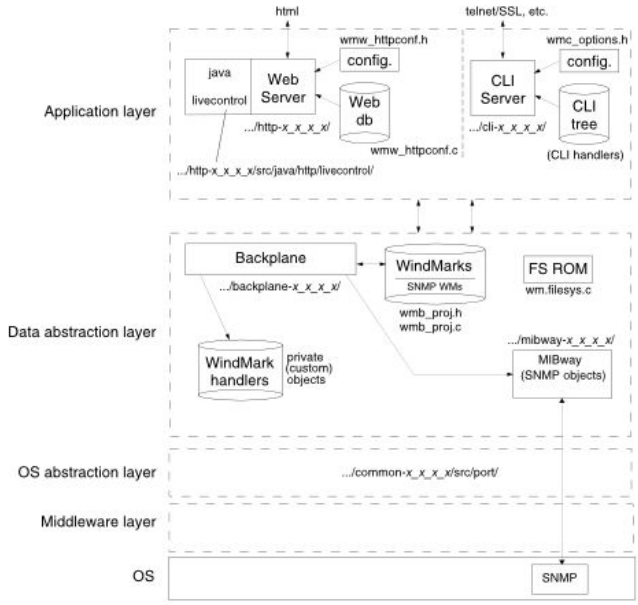
Below that sits a data abstraction layer, whose components (the backplane, WindMarks, and others) allow you to access objects on the managed device; because they hide the details of those objects, these components can be used by either CLI or Web applications. Also on the data-abstraction layer are components for abstracting file systems for accessing remote files through the web. MIBway, in this layer, allows you to integrate your applications with SNMP.

Below that is an operating-system abstraction layer (of which you need not know much, and which is not discussed in detail here), which allows VxWorks 7 CLI,Web, MIBway to interact with several types of OS (VxWorks, Windows, and Linux).

Finally, below these three VxWorks 7 CLI, Web, MIBway layers sit the middleware layer, where actual data comes from (e.g., from the network), and the operating system itself.

The figure below shows the schema in more detail. The various components shown are discussed in greater detail in the following sections.

Figure 2. VxWorks 7 CLI, Web, MIBway Model (Expanded)



Wind River CLI, Web, MIBway Target Code on page 4 Wind River CLI, Web, MIBway includes a body of ANSI C source code that implements the embedded HTTP server, the embedded CLI engine, and the embedded backplane.
Wind River CLI, Web, MIBway Development Tools & Custom Code on page 5 Some parts of a Wind River CLI,Web, MIBway application must be provided by you, the client developer.
Backplane on page 5 The backplane is essentially a database of WindMarks and serves as a data abstraction mechanism for all Wind River CLI,Web, MIBway product components.
Wind River CLI on page 6 Wind River CLI provides you with a set of tools to quickly define a robust and extensible CLI, so that your customers can manage the device from a simple telnet shell, serial console, or SSH.
Wind River Web Server on page 6 Wind River Web Server provides an embedded HTTP server and tools for creating a Web-based device management interface.
Wind River MIBway on page 7 If your device implements an SNMP agent using Wind River SNMP, MIBway allows your Web-based and command-line interfaces to access your SNMP agent as if the browser or telnet session was an SNMP management station.
Configuration Tool on page 7 The management configuration editor (MCE) offers easy to use configuration interfaces.

3.1. Wind River CLI, Web, MIBway Target Code

Wind River CLI, Web, MIBway includes a body of ANSI C source code that implements the embedded HTTP server, the embedded CLI engine, and the embedded backplane.

The code files are organized into the following directories under *installDir* **/vxworks-7/pkgs_v2/app/webcli-***x.x.x.x***/**:

- **backplane-***x.x.x.x*
- **cli-***x.x.x.x*
- **common-***x.x.x.x*
- **http-***x.x.x.x*
- **mibway-***x.x.x.x*

MCE provides graphical facilities you use to create each of these groups of code or files. In addition to the graphical tools provided, the Wind River CLI,Web, MIBway distribution also includes a set of console tools (shell commands) that provide much of the code generation functionality of the MCE. For functional and usage details about these tools, see Console Tools on page 193.

Parent topic: Product Overview on page 2

Related information
Wind River CLI, Web, MIBway Development Tools & Custom Code on page 5
Backplane on page 5
Wind River CLI on page 6
Wind River Web Server on page 6
Wind River MIBway on page 7
Configuration Tool on page 7

3.2. Wind River CLI, Web, MIBway Development Tools & Custom Code

Some parts of a Wind River CLI,Web, MIBway application must be provided by you, the client developer.

The following is a list of what you need to provide:

WindMarks

For each manageable element (variable) that you wish to provide end-user control or configurability, you must create a WindMark. If the WindMark represents a built-in data type, the MCE automatically generates a handler. If the WindMark is a custom type, or a member of a table, you must provide the handler. The MCE provides tools for creating WindMarks and editing WindMark handlers.

Embedded server and backplane configuration settings

The backplane, the embedded HTTP server, and the telnet server each require some basic configuration before you can compile them.

A command tree (CLI only)

You need to create commands and command handlers for each WindMark that needs to be accessible from the command line. Command handlers link the commands entered on the command line to WindMark names and WindMark handlers. MCE provides tools for creating a command tree and a set of command handlers.

Web files (Web Server only)

Web files are the HTML, CSS, JavaScript, and graphic files that create the visual appearance of the Web-based interface. You embed WindMarks in the HTML to link the Web files to WindMark handlers. MCE provides tools that configure target file systems, create ROMBUILD file systems, and configure target URL mappings. The MCE also includes tools to view and edit HTML files.

Parent topic: Product Overview on page 2

Related information
Wind River CLI, Web, MIBway Target Code on page 4
Backplane on page 5
Wind River CLI on page 6
Wind River Web Server on page 6
Wind River MIBway on page 7
Configuration Tool on page 7

3.3. Backplane

The backplane is essentially a database of WindMarks and serves as a data abstraction mechanism for all Wind River CLI,Web, MIBway product components.

WindMarks are name representations of device-resident objects and the handler code to access these objects. The WindMark objects are strongly typed and the MCE allows you to extend the built-in types. The backplane also provides locking, transaction processing, data validation, data conversion, and access type security.

The basic components of Wind River CLI, Web, MIBway are one or more interface servers (the CLI engine and the Web server) and the backplane, which includes a repository of WindMarks (or several repositories of WindMarks).

The small office printer example mentioned in Technology Overview on page 1 illustrates one of the greatest benefits of Web-based management—that it is part of a suite of device management products that allow you to build command-line interfaces and Web-based interfaces that leverage much of the same code. The common point between your CLI application, your Web application, and your Live Control Java applet is the backplane. Since the backplane is where the WindMarks and their handlers are defined and executed, the WindMarks you create for your Web server application can be used by a CLI application. This allows you to provide both a Web management interface and a command-line interface for the same device using the same WindMarks and the same backplane. And you can develop both interfaces in substantially less time than it would take to develop them separately.

Wind River CLI, Web, MIBway allows you to provide a variety of different management interfaces—including access to SNMP data—which maximizes the use of shared components and reduces duplication of work.

For more information about the backplane, see Understanding the Backplane on page 12.

Parent topic: Product Overview on page 2

Related information
Wind River CLI, Web, MIBway Target Code on page 4
Wind River CLI, Web, MIBway Development Tools & Custom Code on page 5
Wind River CLI on page 6
Wind River Web Server on page 6
Wind River MIBway on page 7
Configuration Tool on page 7

3.4. Wind River CLI

Wind River CLI provides you with a set of tools to quickly define a robust and extensible CLI, so that your customers can manage the device from a simple telnet shell, serial console, or SSH.

Wind River CLI consists of a body of ANSI C source code that implements the CLI engine. It also includes a set of development tools that you use to configure the CLI engine code for your hardware and build the custom management command tree for your device. For more information about CLI, see Understanding CLI on page 23.

Parent topic: Product Overview on page 2

Related information
Wind River CLI, Web, MIBway Target Code on page 4
Wind River CLI, Web, MIBway Development Tools & Custom Code on page 5
Backplane on page 5
Wind River Web Server on page 6
Wind River MIBway on page 7
Configuration Tool on page 7

3.5. Wind River Web Server

Wind River Web Server provides an embedded HTTP server and tools for creating a Web-based device management interface.

In Wind River Web Server, manageable objects in the device are represented by WindMarks. A WindMark is a special piece of markup used directly in your HTML that is mapped to a handler function in the backplane. The WindMark's handler function actually reads the data from or writes data to the device-resident variable.

Therefore, when an HTTP request is parsed for WindMarks, and WindMarks are found, the Web server passes the WindMark name and value for a SET operation to the backplane. The backplane calls the registered handler for each WindMark, and whatever functionality you have written into each handler is executed. For many data objects, this functionality involves getting or setting values.

Wind River Web Server decouples your management interface code—the HTML—from the actual embedded code that executes the management operation. Keeping the HTML interface separate from the handler implementation makes both the HTML and the handler code easier to write, understand, and maintain.

This model allows your Web specialist to design the HTML interface without requiring knowledge of the embedded handler code, and your embedded expert can write the custom handler functions without needing to write any HTML. All each needs to know about what the other is doing is the names of WindMarks.

You can deploy Wind River Web Server as the only management interface for a device, or as part of a management application that also includes a command-line interface.

For more information about the Web Server, see Understanding the Web Server on page 25.

Parent topic: Product Overview on page 2

Related information

Wind River CLI, Web, MIBway Target Code on page 4
Wind River CLI, Web, MIBway Development Tools & Custom Code on page 5
Backplane on page 5
Wind River CLI on page 6
Wind River MIBway on page 7
Configuration Tool on page 7

3.6. Wind River MIBway

If your device implements an SNMP agent using Wind River SNMP, MIBway allows your Web-based and command-line interfaces to access your SNMP agent as if the browser or telnet session was an SNMP management station.

From the perspective of the backplane, MIBway is a bridge to the SNMP agent.

Developing a management information base (MIB) for a device can take hundreds or even thousands of engineering hours to create, test, and refine. Rather than discard or duplicate that engineering effort for each management scheme, MIBway inherits all your existing MIBs and creates WindMarks to represent each managed object on the device. You can use these MIBway WindMarks just like any other WindMark in your Web Server and CLI management interfaces.

For more information about MIBway, see Understanding MIBway on page 28.

Parent topic: Product Overview on page 2

Related information

Wind River CLI, Web, MIBway Target Code on page 4
Wind River CLI, Web, MIBway Development Tools & Custom Code on page 5
Backplane on page 5
Wind River CLI on page 6
Wind River Web Server on page 6
Configuration Tool on page 7

3.7. Configuration Tool

The management configuration editor (MCE) offers easy to use configuration interfaces.

You can use these to configure the embedded run-time components and create your device's management interfaces.

Parent topic: Product Overview on page 2

Related information

Wind River CLI, Web, MIBway Target Code on page 4
Wind River CLI, Web, MIBway Development Tools & Custom Code on page 5

Backplane on page 5
Wind River CLI on page 6
Wind River Web Server on page 6
Wind River MIBway on page 7

4. Using This Guide

Information on Wind River CLI, Web, and MIBway is organized by common themes and tasks.

Wind River CLI, Web, MIBway Fundamentals

Provides additional overview and background information on the backplane, Wind River CLI, Wind River Web Server, the HTTP client, and Wind River MIBway.

Configuration and Build

Provides information on creating and configuring MCE-based projects, VSB projects and VIPs for CLI, Web, and MIBway applications.

API Functions

Lists the API libraries available to CLI, Web, and MIBway applications.

Working with the Management Configuration Editor (MCE)

Provides information on how to configure your project with the management configuration editor (MCE).

Creating WindMarks and WindMark Handlers

Provides information on creating and configuring WindMarks and WindMark handlers.

Creating MIBway (SNMP) WindMarks

Provides information on creating SNMP WindMarks.

Creating a CLI Management Command Set

Provides information on creating and configuring a command-line interface. This chapter also provides guidelines for the design and structure of your command-line interface.

Creating a Web-Based Management Interface

Provides information on creating and configuring your Web-based management interface.

Working with Web Content Files

Provides information on designing and working with Web files.

Enabling JavaScript Authentication

Provides information on configuring your project to use JavaScript authentication.

The MCE User Interface Reference

Provides details on the options for each of the dialog boxes that make up the MCE.

Sample Global Commands

Provides details on the sample global commands in commands.rcp, debug.rcp, and files.rcp.

WindMarks in HTML Form Elements

Describes the form elements and how the insert.

Special WindMarks and WindMark Macros

Provides a description and usage information for each of the special WindMarks and WindMark macros.

Console Tools

Provides information on adding your own custom tool commands and running the MIB Compiler.

VxWorks Project Helper Scripts

Provides information on the VxWorks project helper scripts **prjCreate.tcl** and **httpCreate.tcl**.

Glossary

Defines a set of terms that are used throughout this document and the Wind River CLI,Web, MIBway source code.

5. Additional Documentation

You can learn more about the technologies related to the Web, CLI, MIBway product by referencing additional documentation.

Wind River Documentation

Wind River provides a library of publications to support its products. The following Wind River documents present information associated with Wind River CLI,Web, MIBway.

- API Reference: Kernel Networking – provides reference descriptions of all CLI, Web, and MIBway libraries. See the API reference entries.
- VxWorks 7 Creating a CLI Tool and Web Server Tutorial – provides a tutorial for working with the CLI tool and web server.
- VxWorks 7 Creating a CLI Tool Tutorial – provides a tutorial for working with the CLI tool.
- VxWorks 7 Creating a Web Server Tutorial – provides a tutorial for working with the web server.
- VxWorks 7 SNMP Programmer's Guide – describes how to use the Wind River SNMP source code and application programmer's interface (API).The document also provides a technical description of the product, standards compliancy, and details of the server and client model.
- VxWorks 7 SSH Programmer's Guide – describes Wind River SSH, which is Wind River's server implementation of the Secure Shell protocol (SSH) that provides secure remote login, file transfer, and port forwarding over an insecure network.
- VxWorks 7 SSL Programmer's Guide – describes how Wind River implements OpenSSL, the open source SSL development toolkit. It contains instructions for configuring Wind River SSL into a VxWorks image project, plus a brief introduction to the SSL protocols. It does not contain reference documentation on the programming API; this information appears at the OpenSSL Project website http://www.openssl.org.

Online Resources

Online resources are as follows:

- The World Wide Web Consortium, http://www.w3c.org
- W3 Schools, http://www.w3schools.com/
- The Barebones Guide to HTML, http://werbach.com/barebones/
- The Internet Engineering Task Force, http://www.ietf.org
- cURL and libcurl, http://curl.haxx.se/

Books

The following books address subjects in this guide:

- Marshall T. Rose, *The Simple Book: An Introduction to Management of TCP/IP-based Internets*.
- William Stallings, *SNMP, SNMPv2, SNMPv3, and RMON 1 and 2*.
- David Perkins, *Understanding SNMP MIBs*.
- David Zeltzerman, *A Practical Guide to SNMPv3 and Network Management*.
- Paul Simoneau, *SNMP Network Management*.
- Douglas E. Comer, *Internetworking with TCP/IP, Volume I: Principles, Protocols, and Architecture*.

6. RFCs

A significant subset of RFCs are relevant to Wind River CLI,Web, MIBway.

Web-Related RFCs

- RFC 2045: *Multipurpose Internet Mail Extensions (MIME) Part One: Format of Internet Message Bodies*
- RFC 2616: *Hypertext Transfer Protocol -- HTTP/1.1*
- RFC 2617: *Basic and Digest Access Authentication*
- RFC 2818: *HTTP Over TLS*
- RFC 2854: *The 'text/html' Media Type*
- RFC 2965: *HTTP State Management Mechanism*

Telnet-Related RFCs

- RFC 854: *Telnet Protocol Specification*
- RFC 855: *Telnet Option Specifications*
- RFC 856: *Telnet Binary Transmission*
- RFC 857: *Telnet Echo Option*
- RFC 858: *Telnet Suppress Go Ahead Option*
- RFC 859: *Telnet Status Option*
- RFC 860: *Telnet Timing Mark Option*
- RFC 861: *Telnet Extended Options - List Option*
- RFC 1572: *Telnet Environment Option*
- RFC 2941 *Telnet Authentication Option*

SNMP-Related RFCs

- RFC 1155: *Structure and Identification of Management Information for TCP/IP-based Internets*
- RFC 1212: *Concise MIB Definitions*
- RFC 1213: *Management Information Base for Network Management of TCP/IP-based internets: MIB-II*
- RFC 1215: *A Convention for Defining Traps for use with the SNMP*
- RFC 3415: *View-based Access Control Model (VACM) for the Simple Network Management Protocol (SNMP)*

- RFC 3416: *Version 2 of the Protocol Operations for the Simple Network Management Protocol (SNMP)*
- RFC 3417: *Transport Mappings for the Simple Network Management Protocol (SNMP)*
- RFC 3418: *Management Information Base (MIB) for the Simple Network Management Protocol (SNMP)*

2. WIND RIVER CLI, WEB, MIBWAY FUNDAMENTALS

1. Understanding the Backplane

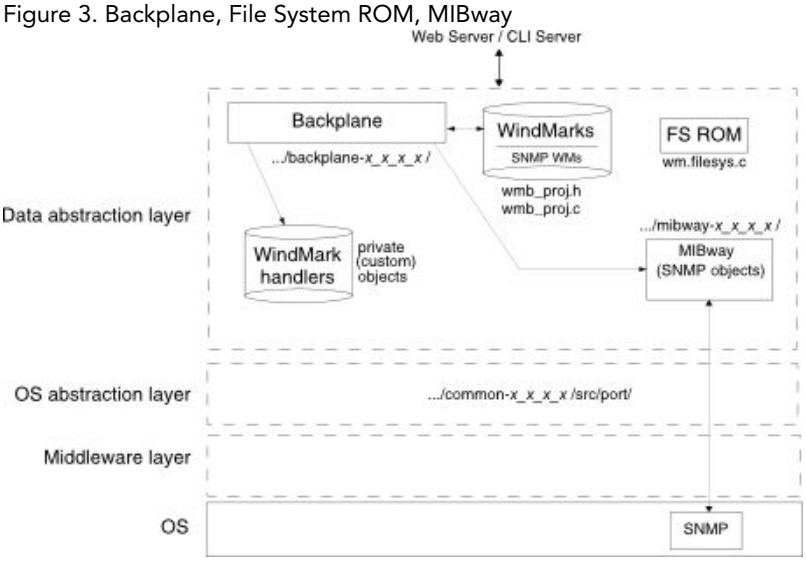
The basic components of VxWorks® CLI, Web, and MIBway are one or more interface servers (the CLI engine and the Web server) and the backplane.

The backplane is essentially a database of WindMarks and serves as a data abstraction mechanism for all Wind River CLI,Web, MIBway product components. The WindMark objects are strongly typed and the MCE allows you to extend the built-in types.

The figure below depicts the VxWorks 7 CLI,Web, MIBway data abstraction layer, which includes the backplane. Refer to Product Overview on page 2 to see how this layer fits into the overall VxWorks 7 CLI,Web, MIBway schema.

This document uses the terms consumers and producers when discussing the components that interact with the backplane. A consumer makes requests to the backplane for WindMarks or data objects. These requests can be to retrieve (GET) or change (SET) the values of the data objects. A producer creates or supplies the actual data each WindMark references and is the repository of WindMarks.

Figure 3. Backplane, File System ROM, MIBway



When an end user initiates a request to GET or SET a WindMark value, the interface server (Web or CLI) passes the WindMark name (and new value in the case of a SET) to the backplane. The backplane then executes the requested WindMark's handler. If the WindMark represents a scalar built-in data type, the automatically generated scalar handler reads (or updates) the value. If the WindMark is a custom data type or a member of a table, the backplane executes whatever functionality you have coded into the WindMark's handler.

For instances where the WindMark represents an SNMP object (MIBway WindMarks), the backplane passes the request to MIBway (a specialized handler for SNMP WindMarks) which formats an SNMP request and passes that request to the SNMP agent on your device.

The backplane offers services beyond GET and SET requests with the data object. The backplane optionally provides the following:

- locking
- transaction processing
- validation
- conversion

- access control

Locking allows multiple clients access to the same object at the same time, but only one client can modify the information in the WindMark. Transaction processing is necessary so multiple GETs or SETs can be done simultaneously on a group of objects, such as a row in a table, and all attempted SETs are backed out of if one of the SETs is not possible. The backplane provides data validation and data conversion of WindMarks in

Rather than passing around a pointer to WindMark objects themselves, the backplane creates a buffer, so that the WindMark itself remains safe in the WindMark repository until the SET is allowed. Likewise with GETs of a WindMark, the value of the WindMark is placed into a buffer. If a type conversion is requested, the type conversion is done on the buffer object, not the WindMark object. The WindMark itself is stored as a native type. A native type is essentially the type that the WindMark is initially set as.

The actual object that is passed is a pointer to a buffer and information about the buffer and its contents, including the maximum size of the buffer, the length of the data stored in the buffer, the native type, and the local type. The native type is how the WindMark was originally stored; the local type is what the consumer is expecting.

Data Access — WindMark Handlers on page 13 In the backplane, objects are organized in a hierarchical manner.
Locking on page 15 The backplane lock manager controls access to WindMark objects by implementing a locking mechanism.
Transaction Processing on page 16 In software development, the word transaction means a number of state changes that either all must occur, or if not all can occur, then none must occur.
Data Validation on page 18 Through the use of the backplane, data validation of objects is conducted as part of the SET request. MCE provides the means for configuration of data validation per object.
Data Types on page 18 The backplane supports a number of built-in data types.
Subranges on page 19 You can extend the data types by applying subranges, which limit the values that the WindMark may store.
Enumerations on page 19 String types and numeric types (short, long, int64, and so forth) can have enumerated values.
SNMP Data Types on page 20 In addition to the built-in data types, the backplane (using MIBway) supports several SNMP data types.
Data Conversion on page 21 When a WindMark is saved or initialized, the data type for the WindMark is set.
Backplane Access Control on page 22
Language Localization on page 22 While the backplane development tools are English based, the backplane itself is designed to support the UTF-8 character set.
WindMark Tables on page 23A WindMark table is a higher level object that contains WindMarks.

1.1. Data Access — WindMark Handlers

In the backplane, objects are organized in a hierarchical manner.

At the top of the hierarchy is the backplane itself. At the next level are the WindMark repositories (producers), which own the data objects. One level below are the WindMark tables, which consist of one or more WindMark objects. The lowest level is the WindMark data object. The build process generates the **wmb_projName.c** file. This file demonstrates the WindMark hierarchy and the organization of the WindMark handlers. In the **wmb_projName.c** file, there are two basic sets of handlers: the table handlers with each table having its own handler, and the scalar handler, which contains the WindMarks that have not been grouped together. (The scalar handler is its own group since it handles all the ungrouped WindMarks together. However, there is no locking at the handler level for the scalar handler, such as there is for the table handlers.)

The generated handlers are broken down into case statements. The first level is to identify the WindMark (Object Identification Case); the next is to identify the action to be taken on the WindMark (Object Action Case), which is then taken by the specific Object Action Handler. Understanding the configuration of the handler is quite important since the generated code is all user modifiable.

This method is used to give maximum flexibility to the customer–developer. In the generated files, **wmb_projName.c** and **wmb_projName.h**, the code that is intended for the customer–developer to manipulate is delimited by begin and end comments. For example:

```
/* %%BeginUserCode ipAddress_Body */
/* %%EndUserCode ipAddress_Body */
```

These begin and end marks are actually around the case for locating the WindMark and the case for the action (that is, GET, SET, GET-NEXT, TEST, COMMIT, UNDO, COMPLETE). GET-NEXT is dependant entirely on how your application manages data; whether it is some lexical method (SNMP OIDs or even alphabetical) or a linked list structure.

Usually, for the most common behavior, GETs and SETs with transaction processing options, there is little need to adjust the handler code segment if the MCE is used properly to configure the WindMarks.

This method of tagging code segments with a begin and end point also keeps these segments from being overwritten when new code is generated. To permanently remove these segments from generated files, you must manually delete the code segments and regenerate the backplane files.

```
case WMID_ipAddress :   /* IP address (DATATYPE_IPV4_K) : read-write */
{
        /* %%BeginUserCode ipAddress_Body */

        /* Note: Locking is indicated as being handled by the user */

        switch (command)
        {
                case CMD_GET_K:
                        status = wmbObjectImport(obj,&ipAddress,4,DATATYPE_IPV4_K);
                        break;
                case CMD_NVMSET_K:
                        status =wmbObjectExport(obj,&ipAddress,4,DATATYPE_IPV4_K,&nSize);
                        break;
                case CMD_GET_NEXT_K:
                        /* Insert Instance Increment Code here */
                        status = wmbObjectImport(obj,&ipAddress,4,DATATYPE_IPV4_K);
                        /* Perform NEXT on instance */
                         break;
                case CMD_TEST_K:
                        /*   (validation string: "NULL") */
                        /* status = wmbObjectSetRangeTest(obj); */

                        /* Insert additional validate code here */

                        if (OK != status)
                        {
                                error = ERROR_GENERAL_OUT_OF_RANGE;
                                break;
                        }

                        /* Attach undo information to transaction */
                        wmbObjectSavedSet(obj,&ipAddress,4,DATATYPE_IPV4_K);
                        break;
                case CMD_COMMIT_K:
                        status =wmbObjectExport(obj,&ipAddress,4,DATATYPE_IPV4_K,&nSize);
                        break;

                case CMD_UNDO_K:
                        /* Retrieve undo information from transaction */
                        wmbObjectSavedGet(obj,&ipAddress,4,DATATYPE_IPV4_K);
                        break;
                case CMD_COMPLETE_K:
                        /* Retrieve undo information from transaction */
                        wmbObjectSavedFree(obj);
                        break;
                default:
```

```
                        error = ERROR_TM_BAD_CMD;
                        break;
        }
        /* %%EndUserCode   ipAddress_Body */
}
```

Within the object action case is an object action handler, which is the function (or code or group of functions) that carries out the required action.

Parent topic: Understanding the Backplane on page 12

Related information

Locking on page 15
Transaction Processing on page 16
Data Validation on page 18
Data Types on page 18
Subranges on page 19
Enumerations on page 19
SNMP Data Types on page 20
Data Conversion on page 21
Backplane Access Control on page 22
Language Localization on page 22
WindMark Tables on page 23

1.2. Locking

The backplane lock manager controls access to WindMark objects by implementing a locking mechanism.

The purpose of this mechanism is to ensure that multiple requests can execute in parallel without violating the isolation principle.

The lock manager does not own the WindMarks, and monitors, rather than controls, the data access. The data is owned by the components that utilize the lock manager to achieve isolated transactions in a multitasking environment.

Objects are organized in a hierarchical manner in the backplane—the backplane itself, the WindMark repositories, tables, and data objects. The lowest level is the WindMark data object, which corresponds to the finest granularity in the lock manager. The lock manager supports locking at each of these levels based on the configuration of the WindMark handler.

The backplane calls the wmbLock( ) function to lock a WindMark. The lock manager discovers the locking level supported by the data object, which is specified as part of each WindMark, through the use of the code generated during the build process.

For example, a single-threaded producer may be locked at the component level and therefore can have at most one transaction active at a time. It is generally expected that the probability of deadlocks increases as the granularity of the locked resources become coarser.

The locks are managed per transaction, and not per task (or thread). The resource locked by a transaction can be accessed by another thread, as long as it is within the same transaction.

There are three types of locks: write locks, read locks, and intent locks. Use the intent lock to lock all the parents of the resource—whether component, table, or WindMark—so that they are not to be removed. The backplane model allows for deregistration of components, tables, and objects; therefore, locking must protect against the parents of the resource being removed. If you want to access a resource for reading or writing, all of its parents must be locked in intent mode. A resource can have multiple intent mode locks at the same time. However, a resource (component, table, WindMark) cannot have a write mode lock and an intent mode lock simultaneously. The tree itself also cannot be modified while a resource within the tree is locked.

When wmbLock( ) is called, the lock manager locks all the parents, starting from the top in intent mode. If any of the parents are already locked in write mode, then the operation fails. If the intent mode lock fails, the lock manager does not block. The intent

mode lock can only fail if the backplane tree structure is being modified. If the tree is being modified, then the resource is locked in write mode, which blocks if there is already an intent mode lock in place.

When all the parents are locked in intent mode, the resource is then locked in read or write mode, based on the parameters provided to the function, usually from a request to the backplane. If the resource is already locked in a mode that is compatible (read mode locks are compatible with each other), the request is granted immediately, and the function returns. Otherwise, the function blocks until the resource is unlocked.

The lock manager uses two semaphores for tracking the locks: Write semaphore and Read semaphore. This is done in order to allow multiple read requests to proceed, and not wait for other read requests.

Parent topic: Understanding the Backplane on page 12

Related information

Data Access — WindMark Handlers on page 13
Transaction Processing on page 16
Data Validation on page 18
Data Types on page 18
Subranges on page 19
Enumerations on page 19
SNMP Data Types on page 20
Data Conversion on page 21
Backplane Access Control on page 22
Language Localization on page 22
WindMark Tables on page 23

1.3. Transaction Processing

In software development, the word transaction means a number of state changes that either all must occur, or if not all can occur, then none must occur.

The backplane supports multiphase transactions for object access. In short, a transaction is a process by which the state of multiple objects are altered in an atomic manner. That is, the state transitions are consistent and simultaneous for all objects. Failure to change one of the objects to the new state causes the entire transaction to abort without changing any of the other objects. There are four basic properties of transactions: atomicity, consistency, isolation, and durability. The backplane supports all of these properties:

- Atomicity requires that all objects within a transaction must change their state as a single atomic operation. No object changes state if the transaction is aborted.
- The consistency property refers to the fact that object state changes are consistent within a transaction.
- Isolation means that a transaction does not depend on and is not a prerequisite of another simultaneous transaction. Transactions must be processed in isolation from one another. In other words, one transaction cannot depend on the interim output of another transaction.
- The durability property requires that the output of the transaction is final and cannot be changed.

In the backplane, the transaction manager component is responsible for managing the transactions. Components that make requests of the backplane—the interface servers—are clients of the transaction manager. The basic phases of a transaction are carried out by specific cases within the Object Action case of each object. The basic phases are test, commit, undo, and complete. Following is a typical case description of the phases, not an exact process that must be followed:

- Test
  - Save present value of WindMarks.
  - If other WindMarks are accessed, they must be explicitly locked (unless best effort transactions are enabled). Test fails if they cannot be locked. For more information about best effort transaction processing, see the Web URL Properties Reference on page 161.
  - Allocate memory.
- Commit

  - Set value.
  - Conduct any necessary consistency checks.
- Undo
  - Rewrite saved value.
- Complete
  - Clean up shared resources.
  - Unlock any locked resources.
  - Clean up any allocated memory.

The commands in the **WMB_RESOURCE_HANDLER_T** function that support and carry out transaction processing are as follows:

CMD_UNDO_K

The handler restores the original state of the objects.

CMD_COMMIT_K

The handler sets the new values of the objects in its storage, saving the existing values for rollback.

CMD_COMPLETE_K

The handler cleans up after the CMD_COMMIT_K or CMD_TEST_K commands.

The transaction manager has the following interfaces:

client interface

The transaction manager client API enables clients, generally the interface server, to initiate and terminate transactions. This gives clients explicit control over transactions. The transaction manager informs the client through a callback mechanism of the result of the transaction commands.

resource handler interface

The transaction manager communicates directly with the resource handlers. Any resource handler that supports transactions is required to implement handlers for certain transaction commands that are initiated by the transaction manager.

The resource handler is the manager of the resources that are altered by the transaction. A producer can have one or more resource handlers that control access to one or more resources. These resources within a producer are accessed by the table handler(s), the scalar handler, and all the individual WindMark handlers. Wind River recommends that resources that have inter-dependencies be managed by the same resource handler, and be locked at the group level to prevent deadlocks.

The backplane manages all components, communications between components, and the events they generate. It ensures authorized access to the resources and controls resource locks.

Handlers are registered by producers during initialization. Like other backplane components, the transaction manager communicates directly with the handlers. It uses the backplane registration database to locate the handler for each object involved in the transaction.

Before proceeding with a transaction, the handler ensures that access is authorized to the resources. The handler also uses the lock manager to ensure that the object is locked (at the object or higher level). Each of the resource handlers do the following:

- Verify authorization of access.
- Lock the resources to prevent inconsistency during the TEST phase.
- Perform necessary dependency and consistency checks.
- Allocate required memory during the TEST phase.
- Create necessary undo objects.

Parent topic: Understanding the Backplane on page 12

Related information

Data Access — WindMark Handlers on page 13
Locking on page 15
Data Validation on page 18
Data Types on page 18
Subranges on page 19
Enumerations on page 19
SNMP Data Types on page 20
Data Conversion on page 21
Backplane Access Control on page 22
Language Localization on page 22
WindMark Tables on page 23

1.4. Data Validation

Through the use of the backplane, data validation of objects is conducted as part of the SET request. MCE provides the means for configuration of data validation per object.

There are a number of built-in types. In addition to the built-in types, there is the option to add subranges to many types (a minimum and maximum value). You can also add a list of specific items or add a number to act as an alias for items on the list (enumeration).

Parent topic: Understanding the Backplane on page 12

Related information

Data Access — WindMark Handlers on page 13
Locking on page 15
Transaction Processing on page 16
Data Types on page 18
Subranges on page 19
Enumerations on page 19
SNMP Data Types on page 20
Data Conversion on page 21
Backplane Access Control on page 22
Language Localization on page 22
WindMark Tables on page 23

1.5. Data Types

The backplane supports a number of built-in data types.

You can create WindMarks and automatically generate WindMark handlers for the following data types:

short	unsigned short
long	unsigned long
int	unsigned int
int64	unsigned int64
char	unsigned char
string	bool

ticks	time
IP address	IPv6 address
MAC Address	OID
void	custom type

Parent topic: Understanding the Backplane on page 12

Related information
Data Access — WindMark Handlers on page 13
Locking on page 15
Transaction Processing on page 16
Data Validation on page 18
Subranges on page 19
Enumerations on page 19
SNMP Data Types on page 20
Data Conversion on page 21
Backplane Access Control on page 22
Language Localization on page 22
WindMark Tables on page 23

1.6. Subranges

You can extend the data types by applying subranges, which limit the values that the WindMark may store.

These values are checked before the WindMark can be changed in the backplane. For example, an integer can have values from -2147483648 to 2147489647, while an unsigned short can only have values between 0 and 65535. With the Sub Range option, you may give an integer a subrange from -2147483648 to 12; for the unsigned short, from 7 to 23. Strings may have subranges based on AlphaNumeric, Alpha, Numeric, Octet, or Display characters.

Parent topic: Understanding the Backplane on page 12

Related information
Data Access — WindMark Handlers on page 13
Locking on page 15
Transaction Processing on page 16
Data Validation on page 18
Data Types on page 18
Enumerations on page 19
SNMP Data Types on page 20
Data Conversion on page 21
Backplane Access Control on page 22
Language Localization on page 22
WindMark Tables on page 23

1.7. Enumerations

String types and numeric types (short, long, int64, and so forth) can have enumerated values.

A list is a set of specific items that the value would have to match. Suppose your list is yes, no, maybe, and asknicely, if any other value were attempted, the validation would not allow the SET of the WindMark. Enumerations add numbers to a list as an alias for the items in the list. If you have yes=1, no=2, maybe=3, and asknicely=4, a value of 4 would validate as equivalent to the list item asknicely.

When an enumerated value is displayed in a text field in HTML, the integer value is displayed rather than the string value (for example, 1 is displayed instead of yes). You can use the IF macro to display the string values; for example:

```
$%IF(begin,ifOperStatus=1)#$
                :<span class="italic">yes</span>
           <!-- You can use any number of IF(else, condition) statements
                as long as the total number is lower than your HTTP
                configuration setting for max number of macros in a
                block.
            -->
           $%IF(else, ifOperStatus=2)#$
                :<span class="italic">no</span>
           $%IF(else, ifOperStatus=3)#$
                :<span class="italic">maybe</span>
           <!-- you can use IF(else) without a condition as a default
                case (for when no conditions are true).
            -->
           $%IF(else)#$
                :<span class="italic">current is ($%ifOperStatus#$)</span>
           $%IF(end)#$
```

In this case, the values of *ifOperStatus* end up being translated to strings for display.

Parent topic: Understanding the Backplane on page 12

Related information

Data Access — WindMark Handlers on page 13
Locking on page 15
Transaction Processing on page 16
Data Validation on page 18
Data Types on page 18
Subranges on page 19
SNMP Data Types on page 20
Data Conversion on page 21
Backplane Access Control on page 22
Language Localization on page 22
WindMark Tables on page 23

1.8. SNMP Data Types

In addition to the built-in data types, the backplane (using MIBway) supports several SNMP data types.

SNMP_Bits	**SNMP_Integer**
SNMP_BridgeId	**SNMP_IpAddress**
SNMP_Counter	**SNMP_MacAddress**
SNMP_Counter32	**SNMP_NetAddress**
SNMP_Counter64	**SNMP_OCTET_STR**
SNMP_DateAndTime	**SNMP_OID**

SNMP_DisplayString	**SNMP_Opaque**
SNMP_Empty	**SNMP_PortList**
SNMP_Gauge	**SNMP_RowStatus**
SNMP_Gauge32	**SNMP_TimeTicks**
SNMP_Int32	**SNMP_UInt32**

These data types are imported when you import a MIB.

Parent topic: Understanding the Backplane on page 12

Related information
Data Access — WindMark Handlers on page 13
Locking on page 15
Transaction Processing on page 16
Data Validation on page 18
Data Types on page 18
Subranges on page 19
Enumerations on page 19
Data Conversion on page 21
Backplane Access Control on page 22
Language Localization on page 22
WindMark Tables on page 23

1.9. Data Conversion

When a WindMark is saved or initialized, the data type for the WindMark is set.

When a WindMark is set, the type is tested to see if it conforms to a number of configurable rules as mentioned in Data Validation on page 18. However, since you can manipulate the object action handler code, there is a great deal of flexibility. WindMarks are stored in what is known as native format. Native format means that it is stored as the type of object it is. If you save an integer WindMark, then alter your object action handler to read it as a string as shown, you are allowed to read the data as a string, but be aware that it is still stored in the backplane as its native type, an integer.

```
case CMD_GET_K:
     status = wmbObjectImport(obj,&VariableAddress,4,DATATYPE_INTEGER_K);
     break;
case CMD_NVMSET_K:
     status = wmbObjectExport(obj,&VariableAddress,4,DATATYPE_STRING_K,&nSize);
     break;
```

Parent topic: Understanding the Backplane on page 12

Related information
Data Access — WindMark Handlers on page 13
Locking on page 15
Transaction Processing on page 16
Data Validation on page 18
Data Types on page 18
Subranges on page 19
Enumerations on page 19
SNMP Data Types on page 20

Backplane Access Control on page 22
Language Localization on page 22
WindMark Tables on page 23

1.10. Backplane Access Control

Each WindMark has the option for restricting access based on an access level assigned to that WindMark. The backplane does not perform any authentication. That task is left to the embedded server component, such as the HTTP server or the telnet server. The embedded server performs the authentication, then passes the authenticated user information to the backplane. The backplane compares the access level of the requester (defined in the WindMark) to the access level required to read or write (GET or SET) the value for the WindMark in question (also defined in the WindMark).

For a more detailed discussion about available access control options, see Access Model Dialog Box Reference on page 132.

The Access Model dialog box allows you to set the access option labels for the access option bit-flags.

Values are automatically assigned by the code generation. However, these values can be overridden by the user either with numerical value or #define values.

Parent topic: Understanding the Backplane on page 12

Related information

Data Access — WindMark Handlers on page 13
Locking on page 15
Transaction Processing on page 16
Data Validation on page 18
Data Types on page 18
Subranges on page 19
Enumerations on page 19
SNMP Data Types on page 20
Data Conversion on page 21
Language Localization on page 22
WindMark Tables on page 23

1.11. Language Localization

While the backplane development tools are English based, the backplane itself is designed to support the UTF-8 character set.

It is not designed to support wide or extended byte character sets.

Parent topic: Understanding the Backplane on page 12

Related information

Data Access — WindMark Handlers on page 13
Locking on page 15
Transaction Processing on page 16
Data Validation on page 18
Data Types on page 18
Subranges on page 19
Enumerations on page 19
SNMP Data Types on page 20
Data Conversion on page 21
Backplane Access Control on page 22
WindMark Tables on page 23

1.12. WindMark Tables

A WindMark table is a higher level object that contains WindMarks.

Such objects include the following:

- C data structures
- groups, which are collections of WindMarks that share a relationship, such as a dependency or a common handler

Tables are represented by a single backplane handler, and all member WindMarks are instantiated in that handler. WindMarks that are not a member of a table are gathered into the scalar handler.

Table descriptions are registered with the backplane, so that users can walk the list of members. The handler configuration section is intended to describe the attributes of the table as a whole, and apply to all of the members. For example, setting the locking to apply at the handler level preempts the object locking choices, as does the handler type and priority.

Note: A WindMark cannot be in more than one table.

C Data Structures

C data structures are treated as tables. When you create a WindMark to represent a structure member, create the WindMark using the member's data type and specify the structure (or its typedef name) as the parent table.

Group of WindMarks

A group of WindMarks is a grouping of WindMarks that share a common handler and are not otherwise part of a table.

For information on how to create tables, see Creating WindMark Tables on page 74.

For details on the options available from the WindMark Table dialog box, see WindMark Table Dialog Reference on page 133.

Parent topic: Understanding the Backplane on page 12

Related information

Data Access — WindMark Handlers on page 13
Locking on page 15
Transaction Processing on page 16
Data Validation on page 18
Data Types on page 18
Subranges on page 19
Enumerations on page 19
SNMP Data Types on page 20
Data Conversion on page 21
Backplane Access Control on page 22
Language Localization on page 22

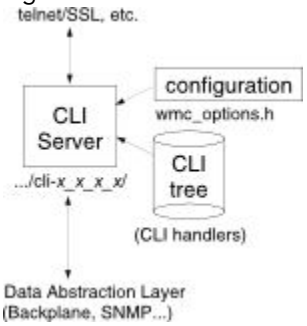
2. Understanding CLI

Wind River CLI provides you with a set of tools to quickly define a robust and extensible CLI, so that your customers can manage the device from a simple telnet shell, serial console, or SSH.

Wind River CLI consists of a body of ANSI C source code that implements the CLI engine. It also includes a set of development tools that you use to configure the CLI engine code for your hardware and build the custom management command tree for your device.

The diagram below depicts CLI. Refer to Product Overview on page 2 to see how CLI fits into the overall VxWorks 7 CLI, Web, MIBway schema.

Figure 4. CLI



This section provides an overview of the CLI engine and interfaces.

CLI Engine Overview on page 24 The CLI engine serves as a command parsing module that reads in the message received from the client, performs simple parsing to tokenize the message, checks for valid data and/or error conditions, and determines the appropriate command handler to execute.
Client Interfaces on page 25 Wind River CLI implements two client interfaces.
SSH on page 25 For VxWorks target operating systems, Wind River CLI allows you to use Wind River SSH to enable a connection from a telnet client to a telnet server.

2.1. CLI Engine Overview

The CLI engine serves as a command parsing module that reads in the message received from the client, performs simple parsing to tokenize the message, checks for valid data and/or error conditions, and determines the appropriate command handler to execute.

Core keywords, such as no (which negates the sense of some commands), are checked for and handled here.

The CLI engine has the following features:

- A standard name:password authentication mechanism used for handling initial authentication and authorization. Although this process is handled by the CLI engine, you must write the code that performs the authentication and manages authorized users.
- A command interpreter (which acts as a UNIX-like shell) with the following features:
  - Command completion – pressing **TAB** at the command prompt initiates a search of the command tree for the first unambiguous match of a command to the text entered by the user to that point. For example, if the user enters **res<TAB>** and the only command that starts with res is reset, the command tree returns reset, and the CLI engine displays reset on the command line.
  - History – the CLI engine records the last x commands issued by the user. The size of the command history buffer is configurable.
  - Help – typing help or help *commandName* retrieves help information from the command tree based on whatever portion of a command the user has typed to that point.
  - Source files – the CLI engine reads in a command, tokenizes it, assigns each token to a command node or parameter, and validates that the parameter information is correct.
  - Intermediate mode support – intermediate mode temporarily moves the root node closer to a particular leaf node on the tree. The shell remains in intermediate mode until the user explicitly enters an exit command such as **CTRL+Z**.
  - Interfaces with the backplane to GET and SET device management data.

Parent topic: Understanding CLI on page 23

Related information
Client Interfaces on page 25

SSH on page 25

2.2. Client Interfaces

Wind River CLI implements two client interfaces.

Packet Interface

This module provides support for the telnet protocol (RFCs 854-861) and SSH protocol, as well as some of the more modern options implemented in client shells (such as RFC 2941 and RFC 1572). It also supports native shell interfaces such as the one supplied with the VxWorks operating system.

Serial Interface

This module executes any code that is necessary to provide standard RS-232 access to the device.

You can interrupt the CLI command execution by using either CTRL+X or CTRL+C. To configure this functionality, enable the WEBCLI_RCC_INTERRUPT_EXEC VSB option. See Source Build Configuration Options (VSB) on page 38.

Parent topic: Understanding CLI on page 23

Related information
CLI Engine Overview on page 24
SSH on page 25

2.3. SSH

For VxWorks target operating systems, Wind River CLI allows you to use Wind River SSH to enable a connection from a telnet client to a telnet server.

Wind River SSH is a server implementation of the Secure Shell protocol (SSH) that provides secure remote login, file transfer, and port forwarding over an insecure network.

For information on enabling a connection to a telnet client using SSH, see VxWorks 7 Configuration and Build Guide.

Parent topic: Understanding CLI on page 23

Related information
CLI Engine Overview on page 24
Client Interfaces on page 25

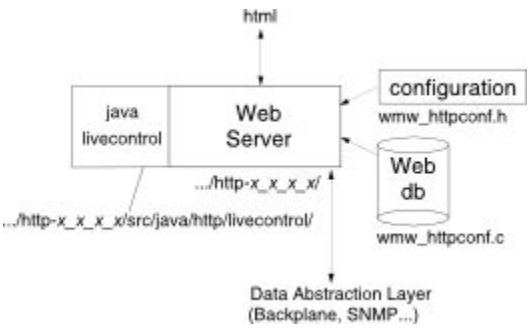
3. Understanding the Web Server

Wind River Web Server consists of an embedded HTTP server and tools for creating a Web-based device management interface.

The server is a scalable embedded HTTP server. All you need to do to use the HTTP server on your target hardware is set a few configuration parameters, write your own custom WindMark error, post reply functions, and build.

The following diagram depicts the Web server. Refer to Product Overview on page 2 to see how the Web server fits into the overall VxWorks 7 CLI, Web, MIBway schema.

Figure 5. Web Server



Embedded HTTP Server on page 26 The server is a scalable embedded HTTP server.

3.1. Embedded HTTP Server

The server is a scalable embedded HTTP server.

It has a modular architecture which allows you to include or exclude modules—called request processing modules, or RPMs—so that you can build in only the features you need, and exclude those you do not need, to keep the compiled application size as small as possible. You can also create your own request processing modules to extend the functionality of the server.

System RPMs

System RPMs are those processing modules that the server calls for every incoming HTTP request. These include request logging, aliasing, access control, and so forth.

User RPMs

User RPMs are processing modules that the server calls only for specific requests. User RPMs are configured to handle requests based on a URL substring. If a user RPM is configured to the substring foo, all HTTP requests made to URLs that contain the initial substring foo are processed by that RPM. For example, the following URL:

http://www.example.net/foo/bar.html

is processed by the RPM that is configured to the substring foo.

User RPM examples include the following:

- file system RPM
- WindMark parsing RPM
- CGI processing RPM
- any custom RPM you implement

Wind River SSL Interface

The Web server provides hooks for the Wind River SSL library.

Parent topic: Understanding the Web Server on page 25

4. Understanding the HTTP Client

The Wind River HTTP client is based on the open-source client-side URL transfer library libcurl, version 7.55.1.

The HTTP client allows you to set up a session with a server using a protocol such as HTTP or HTTPS. Using the libcurl library, you can write applications for transferring files (using the GET and POST methods) with URL syntax.

Note: The HTTP client is available only for VxWorks.

The HTTP client is an independent but related component, and has no dependencies to the MCE.

The HTTP client includes support for the following:

- HTTP 1.1 specification
- cookies
- GET and POST methods
- direct requests as well as proxy server requests
- HTTPS using Wind River SSL

For more information on libcurl, see http://curl.haxx.se/.

Note: libcurl supports many more features than CLI,Web, MIBway. These extra features, such as LDAP and FTP, have not been removed from the source code. However, they have been disabled in the make system by default. These extra features have not been tested and are not supported.

Using the HTTP Client on page 27 After configuring your project, you must also write the source code that accesses the curl functions.

4.1. Using the HTTP Client

After configuring your project, you must also write the source code that accesses the curl functions.

About This Task

For source code example, see **http-post.c**, **cookie_interface.c** and **https.c**. These files are provided in: *installDir* **/vxworks-7/pkgs_v2/app/webcli/libcurl-x.x.x/src/3pp/curl/docs/examples/**. To enable VxWorks secure key database support for the Web client, you must do the following:

Procedure

1. Edit the source code and define the macro **WM_SECURE_KEY_STORAGE**.

   Edit the beginning of the file **openssl.c**, which is in *installDir* **/vxworks-7/pkgs_v2/app/webcli/libcurl-x.x.x/src/3pp/curl/lib/vtls/**.

2. Build the library along with other systems in a VSB project.

   When using the curl library, you must first set **CURLOPT_SSLKEYID** with the value of the key ID, which you generated (for more information, see the VxWorks 7 Cryptography Libraries Programmer's Guide).

   For example:

```
curl = curl_easy_init();
 /* 3rd parameter is the key id you stored the private key before */
curl_easy_setopt(curl, CURLOPT_SSLKEYID, "my_curl_private_key_id");
```

Parent topic: Understanding the HTTP Client on page 26

5. Understanding MIBway

If your device implements an SNMP agent using Wind River SNMP, MIBway allows your Web-based and command-line interfaces to access your SNMP agent as if the browser or telnet session was an SNMP management station.

From the perspective of the backplane, MIBway is a bridge to the SNMP agent.

Unlike the generated or custom WindMark handlers that you create, the backplane accesses MIBway WindMarks (which you generate from a MIB file) by using a built–in handler called the wmmHandler (or the MIBway handler). The MIBway handler automatically generates SNMP request packet data units (PDUs) and transmits those requests to the SNMP agent. The MIBway handler also checks the SNMP error log for existing errors before transactions and then again after execution.

SETs and COMMITs on page 28 Wind River MIBway handles SNMP SET operations as a number of object value pairs in a single SNMP packet data unit (PDU).
Setting Table Object Instance Values on page 29 There are two ways to set a tabular object's instance value.
MIBway Error Handling on page 30 When a NMS issues a PDU request, there is the possibility that the response PDU contains errors.
Displaying SNMP Errors in the Web Server on page 31 It is simple to format and display SNMP errors in an HTML page.
Displaying SNMP Errors in CLI on page 33 In the command handler that accesses a WindMark, you should add a function to check for SNMP errors and display them.
Wind River MIBway and SNMP Security on page 34SNMPv3 has two basic types of security models: the user-based security model (USM) and the view–access control model (VACM).

5.1. SETs and COMMITs

Wind River MIBway handles SNMP SET operations as a number of object value pairs in a single SNMP packet data unit (PDU).

About This Task

The Web Server creates a PDU with multiple object value pairs by creating a transaction list. The Web Server does this automatically whether MIBway is part of your project or not.

This example is only for illustrative purposes. Within Wind River Web Server, you do not need to make any adjustments. For Wind River CLI, you need to create a command handler.

In Wind River Web Server, the module **httpRpmWmbPost.c** allows for building a transaction list using the unique identifier, **tid**.

Steps 1 and 2 build the transaction list and are done iteratively. Giving the wmbObjectCreate( ) the unique transaction identification value (tid) creates the object as part of the transaction list. The value for the object is stored using the wmbObjectValueCreate( ) function. Once all values are entered in the list, the transaction list is processed by wmbTransactionProcess( ), which also is identified by the tid.

SETs are done atomically from Wind River CLI. You could create a transaction list using the same functions as shown above, then process the transaction. However, creating the objects and the call to wmbTransactionProcess( ) must be done from within a single Wind River CLI command handler.

Procedure

1. Create an object to add to the transaction list:

```
wmbObjectCreate(tid, pMagicMarkup, NULL, 0, &obj)
```

2. Store the value of the object.

```
wmbObjectValueCreate
    (
    obj, /* object */
    pVal, /* value */
    STRLEN(pVal) + 1, /* length of value */
    DATATYPE_STRING_K, /* type of value */
    TRUE /* TRUE = copy */
    )
```

3. Iterate steps 1 and 2 above for as many objects as need to be set.
4. Process the transaction:

```
wmbTransactionProcess(tid, CMD_SET_K)
```

Parent topic: Understanding MIBway on page 28

Related information

Setting Table Object Instance Values on page 29
MIBway Error Handling on page 30
Displaying SNMP Errors in the Web Server on page 31
Displaying SNMP Errors in CLI on page 33
Wind River MIBway and SNMP Security on page 34

5.2. Setting Table Object Instance Values

There are two ways to set a tabular object's instance value.

About This Task

The function wmbObjectCreate( ) is one way. However this method has a limitation in that all instance values that begin with a number are treated as if they are SNMP OIDs. Similarly, wmbObjectCreate( ) treats all instance values that begin with non-numeric characters as string values. A problem arises when you have non-SNMP WindMark tables whose instance values are strings that begin with numbers. In such a case, you must use wmbObjectSetupInstance( ) to ensure the instance value is of the proper type.

The following procedure demonstrates setting the instance value using wmbObjectSetupInstance( ):

Procedure

1. Create and add an object to a transaction by calling wmbObjectCreate( ).
2. Set up the instance by calling wmbObjectSetupInstance( ).

   This function takes the following parameters:

   - the transaction ID
   - a pointer to the object record
   - a pointer to the instance
   - the length of the instance (string instances must include the **NULL** terminator)
   - the data type of the instance (either **DATATYPE_STRING_K** or **DATATYPE_OID_K**)

- **TRUE** or **FALSE** to copy the data into the object (you should always enter **TRUE**)

3. Attach the data to the object record by calling wmbObjectValueCreate( ).

   This function takes the following parameters:

   - a pointer to the object record
   - a pointer to the value
   - the length of the value (string instances must include the **NULL** terminator)
   - the data type of the value; for example, **DATATYPE_STRING_K**, **DATATYPE_LONG_K**, and so forth
   - **TRUE** or **FALSE** to copy the value into the object (you should always enter **TRUE**)

```
if ((OK != wmbObjectCreate(tid, pMyWindMarkName, NULL, 0, &obj1)) ||
    (OK != wmbObjectSetupInstance(tid, obj1, pMyInstanceString,
        strlen(pMyInstanceString)+1, DATATYPE_STRING_K, TRUE))
    || (OK != wmbObjectValueCreate(obj1, pMyValueString,
        sizeof(pMyValueString)+1, DATATYPE_STRING_K, TRUE)) )
{
    printf ("Bulk-Set: Could not add WM '%s' to request.", pMyWindMarkName);
    return ERROR;
}
```

Parent topic: Understanding MIBway on page 28

Related information

SETs and COMMITs on page 28
MIBway Error Handling on page 30
Displaying SNMP Errors in the Web Server on page 31
Displaying SNMP Errors in CLI on page 33
Wind River MIBway and SNMP Security on page 34

5.3. MIBway Error Handling

When a NMS issues a PDU request, there is the possibility that the response PDU contains errors.

This possibility also exists for the requests that are made by Wind River MIBway. A ring buffer logs the errors. The size of the ring buffer is determined in *installDir* **/vxworks-7/pkgs_v2/app/webcli/mibway-x.x.x.x/h/wmmErrorLog.h** as follows:

```
#define NMSGS (50) /* Max number of messages in ring buf */
```

Error logging is done independently for each session. When a session is created, the backplane creates an error log for it and records SNMP error messages from the SNMP agent as they occur. For Wind River Web Server, you also need to create a WindMark to display the error logging message. For Wind River CLI, you can call an error handling function from the command handler. SNMP error handling is done with a log buffer and six error handling functions, as shown in the following table.

Table 1. Error Handling Routines

Routine	Action
wmmErrorLogStart( )	Start logging SNMP packets from a given session.
wmmErrorLogEnd( )	Stop error logging for this sessionID.

Routine	Action
wmmErrorNumMsgsGet( )	Return the number of unread SNMP response messages.
wmmErrorLostMsgsGet( )	Return the number of lost error messages (overwritten in the ring buffer).
wmmErrorMsgPurge( )	Delete all messages from the log buffer.
wmmErrorMsgGet( )	Return SNMP response data from current transaction.

Parent topic: Understanding MIBway on page 28

Related information

SETs and COMMITs on page 28
Setting Table Object Instance Values on page 29
Displaying SNMP Errors in the Web Server on page 31
Displaying SNMP Errors in CLI on page 33
Wind River MIBway and SNMP Security on page 34

5.4. Displaying SNMP Errors in the Web Server

It is simple to format and display SNMP errors in an HTML page.

About This Task

If your page contains many SNMP WindMarks, you may run out of space in the substitution buffer. Consider using the **REPEAT** macro instead. The serving of each page is considered a session.

Procedure

1. Create a WindMark named **snmpErrorDisplayHtml** that is a string type.
2. When you generate the handler code from the WindMark, alter the object action handler as is shown in the following sample code swatch:

```
switch (command)
        {
                case CMD_GET_K:
                        /* NOTE: this would be better if accessed through the REPEAT macro in t
he
                         * browser, instead of sending such a large buffer at once - jc
                         */
                        if (0 == (nMsgs = wmmErrorNumMsgsGet(sessionID)))
                                {
                                strcpy (tempBuf, "no SNMP errors");
                                status = wmbObjectImport(obj,tempBuf,sizeof(tempBuf),DATATYPE_S
TRING_K);
                                break; /* done */
                                }

                        /* there are errors... allocate an error descriptor */
                        pSnmpErr = (WMM_SNMP_ERROR_T*) wmMalloc (sizeof(WMM_SNMP_ERROR_T));
                        if (NULL == pSnmpErr)
```

```
                                {
                                wmLogPrintf ("tutSNMPerrScalarHandler: failed to allocate mem")
;
                                break;
                                }

                        /* allocate a buffer to dump the HTML text to be displayed */
                        pSnmpHtml = (sbyte *) wmMalloc (nMsgs * kSizeOfErrorMessage);
                        if (NULL == pSnmpHtml)
                                {
                                wmLogPrintf ("tutSNMPerrScalarHandler: failed to allocate mem")
;
                                wmFree (pSnmpErr);
                                break;
                                }
                        /* loop once for each error from the error store */
                        while (wmmErrorNumMsgsGet(sessionID))
                                {
                                /* trim and terminate string */
                                pSnmpHtml[kSizeOfErrorMessage * msgCount++] = '\0';
                                sprintf (tempBuf, "ses:%d ", sessionID);
                                strcat (pSnmpHtml, tempBuf);
                                /* Get 1 message from the error store */
                                if (0 == wmmErrorMsgGet ((INT8*) pSnmpErr,
                                                sizeof(WMM_SNMP_ERROR_T),1,sessionID))
                                        {
                                        strcat (pSnmpHtml, "failed to read SNMP errors");
                                        break;
                                        }
                                /* Translate error descriptor into readable HTML */
                                sprintf (tempBuf, "idx:%d sts:%d",pSnmpErr->errorIndex,
                                                pSnmpErr->errorStatus);strcat (pSnmpHtml, tempB
uf);

                                /* Display the OID associated with the error index */
                                strcat (pSnmpHtml, " OID: ");
                                for (j = 0; j < pSnmpErr->vbList[0].vbOid.numComponents; j++)
                                        {
                                        if (j != (pSnmpErr->vbList[0].vbOid.numComponents - 1))
                                                {
                                                sprintf (tempBuf, "%d.", (int)
                                                                pSnmpErr->vbList[0].vbOid.compo
nentList[j]);
                                                }
                                        else
                                                {
                                                sprintf (tempBuf, "%d", (int)
                                                pSnmpErr->vbList[0].vbOid.componentList[j]);
                                                }
                                        strcat (pSnmpHtml, tempBuf);
                                        }
                                strcat (pSnmpHtml, " ");
                                strcat (pSnmpHtml, pSnmpErr->errorDesc);
                                strcat (pSnmpHtml, "<BR>");
                                }
                        status = wmbObjectImport(obj, pSnmpHtml, strlen(pSnmpHtml) +1,
                                        DATATYPE_STRING_K);
```

```
                        wmFree (pSnmpHtml);
                        wmFree (pSnmpErr);
                        break;

                default:
                        wmbObjectStatusSet(obj,ERROR_TM_BAD_CMD);
                        return ERROR;
                        break;
        }
```

3. Add the WindMark whenever you want to display SNMP errors to your Web page.

Parent topic: Understanding MIBway on page 28

Related information

SETs and COMMITs on page 28
Setting Table Object Instance Values on page 29
MIBway Error Handling on page 30
Displaying SNMP Errors in CLI on page 33
Wind River MIBway and SNMP Security on page 34

5.5. Displaying SNMP Errors in CLI

In the command handler that accesses a WindMark, you should add a function to check for SNMP errors and display them.

About This Task

The following sample code swatch shows one way to read for errors and display them.

Procedure

Add a call to the function cliSnmpErrors(pCliEnv).

Following is the sample function:

```
/* cliSnmpErrors.c - a sample function to display SNMP errors in CLI */
#include "wm.h"
#include "wmb.h"
#include "wmmLib.h"   /* WindManage for Mibs header   */
#include "wmmErrorLog.h"
void cliSnmpErrors
        (
        WMB_SESSION_T   *   pSession
        )
        {
        sbyte4              nMsgs;
        sbyte4              j;
        INT32               sessionID;
        WMM_SNMP_ERROR_T *  pSnmpErr;

        sessionID = wmbSessionIdGet(pSession);
        if (0 == (nMsgs = wmmErrorNumMsgsGet(sessionID)))
                return;
```

```
        /* there are errors... allocate an error descriptor */
        if (! wmNew(pSnmpErr))
                {
                wmLogPrintf ("cliSnmpErrors: failed to allocate mem");
                return;
                }

        /* loop once for each error from the error store */
        while (0 < wmmErrorNumMsgsGet(sessionID))
                {
                /* Get 1 message from the error store */
                if (0 == wmmErrorMsgGet ((INT8 *) pSnmpErr,
                                sizeof(WMM_SNMP_ERROR_T), 1, sessionID))
                        {
                        wmLogPrintf ("cliSnmpErrors: failed to read SNMP errors");
                        break;
                        }

                /* Translate error descriptor into readable text */
                wmbSessionPrintf (pSession, "idx:%d sts:%d OID: ",pSnmpErr->errorIndex,
                                                pSnmpErr->errorStatus);

                /* Display the OID associated with the error index */
                for (j = 0; j < pSnmpErr->vbList[0].vbOid.numComponents; j++)
                        {
                        wmbSessionPrintf (pSession, "%d",(int)
                                                        pSnmpErr->vbList[0].vbOid.componentList[j]
);
                        if (j != (pSnmpErr->vbList[0].vbOid.numComponents - 1))
                                wmbSessionPrintf (pSession, ".");
                        }
                wmbSessionPrintf (pSession, " %s \r\n",pSnmpErr->errorDesc);
                }
        }
```

The cliSnmpErrors( ) function is intended as a sample only. This function only pulls an error from the error log ring buffer. It does not ensure that the SNMP error that is returned matches the error created by the command handler that calls cliSnmpErrors( ).

Parent topic: Understanding MIBway on page 28

Related information

SETs and COMMITs on page 28
Setting Table Object Instance Values on page 29
MIBway Error Handling on page 30
Displaying SNMP Errors in the Web Server on page 31
Wind River MIBway and SNMP Security on page 34

5.6. Wind River MIBway and SNMP Security

SNMPv3 has two basic types of security models: the user-based security model (USM) and the view–access control model (VACM).

The USM security framework controls access to the device while VACM determines which objects may be accessed by different users.

Wind River MIBway is on the device with the SNMP agent, thus USM is not an issue for Wind River MIBway. Wind River MIBway uses a bit–masked access control scheme that does not map to the VACM. The bit–masked access control scheme is described in Access Model Dialog Box Reference on page 132.

Parent topic: Understanding MIBway on page 28

Related information

SETs and COMMITs on page 28
Setting Table Object Instance Values on page 29
MIBway Error Handling on page 30
Displaying SNMP Errors in the Web Server on page 31
Displaying SNMP Errors in CLI on page 33

3. CONFIGURATION AND BUILD

1. Configuration and Build Workflow

The starting point for all configuration activities for CLI, Web, and MIBway projects is the Web CLI Configuration root object located in the management configuration editor (MCE).

> Note: These features only work in kernel mode for VxWorks. They are not supported for RTP projects.

The overall project workflow is as follows:

1. Configure and build a project using the MCE. A typical workflow is as follows:
   - Create WindMarks to represent manageable objects (see Creating WindMarks and WindMark Handlers on page 70 and *Creating MIBway (SNMP) WindMarks*).
   - Create a Command Set (see Creating a CLI Management Command Set on page 83).
   - Create web files and URLs (see Creating a Web-Based Management Interface on page 106).

   These steps do not have to be performed in any specific order. Once you have completed these tasks (and often at various iterations during development), you then build your management application. For information on enabling and accessing the MCE from Workbench 4, see Enabling the MCE from Workbench 4 on page 37 and Accessing the MCE from Workbench 4 on page 37. For information on how to configure a project with MCE, see Working with the Management Configuration Editor (MCE) on page 57.
2. Configure and build a VxWorks Source Build (VSB) project for the CLI, Web, and MIBway project defined in step 1. From the VSB, you can change a number of CLI, Web server, and Common configuration parameters. See Source Build Configuration Options (VSB) on page 38.

   For specific instructions on how to configure and build a VSB, see the VxWorks 7 Configuration and Build Guide.
3. Configure and build a VxWorks Image Project (VIP) based on the CLI, Web, and MIBway VSB defined in step 2. See Kernel Configuration Components (VIP) on page 51.

   For specific instructions on how to configure and build a VIP, see the VxWorks 7 Configuration and Build Guide.
4. Start the simulator or target and load the image.
5. Access the web pages of your web server. For example web pages and how to access them, see the VxWorks 7 Creating a Web Server Tutorial.
6. Access the CLI. Example CLI usage forms part of the CLI tool demonstration, see VxWorks 7 Creating a CLI Tool Tutorial.

Configuration and Build Workflow for the Demonstration Projects

To configure and build a CLI tool, Web server, or CLI tool and Web server demonstration project, see the following tutorials:

- VxWorks 7 Creating a CLI Tool Tutorial

  This tutorial requires steps 2, 3, 4 and 6.
- VxWorks 7 Creating a Web Server Tutorial

  This tutorial requires steps 2 to 5.
- VxWorks 7 Creating a CLI Tool and Web Server Tutorial

  This tutorial requires steps 2 to 6.

> Note: Project configuration with the MCE does not form part of the demonstration projects.

Custom Projects

To create a custom CLI tool and Web server project based on the CLI tool and Web server demonstration project, see Customizing Your Environment on page 67.

2. Enabling the MCE from Workbench 4

You can enable access to the management configuration editor (MCE) from Workbench 4.

Procedure

1. Copy the **com.windriver.webcli.ui**_*x.x.x.x*_**.jar** file.

   Copy the file from:

   installDir **/vxworks-7/pkgs_v2/app/webcli-**_x.x.x.x_**/tools-**_x.x.x.x_**/bin/**

   to

   installDir **/workbench-4/eclipse/x86-win32/plugins/**

2. Copy the **gzip.dll** file.

   Copy the file from:

   installDir **/vxworks-7/pkgs_v2/app/webcli-**_x.x.x.x_**/tools-**_x.x.x.x_**/bin/x86-win32/WMIT/**

   to

   installDir **/vxworks-7/pkgs_v2/app/webcli-**_x.x.x.x_**/tools-**_x.x.x.x_**/bin/x86-win32/**

3. Accessing the MCE from Workbench 4

Open the appropriate **config.webcli** file from the **File > Open File...** menu.

About This Task

To create custom projects based on a demonstration project, see Customizing Your Environment on page 67.

For information on using the MCE, see Working with the Management Configuration Editor (MCE) on page 57.

To access the MCE from Workbench 4, proceed as follows:

Procedure

1. From Workbench 4, select **File > Open File...** and navigate to a **config.webcli** file.

   For example:

installDir **/vxworks-7/pkgs_v2/app/webcli/clidemo-***x.x.x.x* **/src/config.webcli**

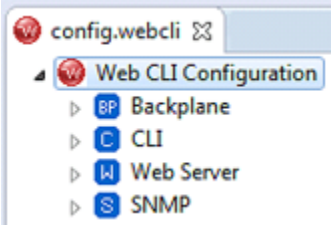
installDir **/vxworks-7/pkgs_v2/app/webcli/webdemo-***x.x.x.x* **/src/config.webcli**

installDir **/vxworks-7/pkgs_v2/app/webcli/webclidemo-***x.x.x.x* **/src/config.webcli**

2. Select the **config.webcli** file and click **Open**.

   Observe that a config.webcli tab is present.

   For example:



4. Source Build Configuration Options (VSB)

The main configuration items for CLI, Web, and MIBway are as follows:

- WEBCLI
  - WEBCLI Menu
- WEBCLI_BACKPLANE
- WEBCLI_CLI
  - WEBCLI_CLI Menu
- WEBCLI_CLIDEMO
- WEBCLI_COMMON
  - WEBCLI_COMMON Menu
- WEBCLILIBCURL
- WEBCLI_HTTP
  - WEBCLI_HTTP Menu
- WEBCLI_MIBWAY
- WEBCLI_WEBCLIDEMO
- WEBCLI_WEBDEMO

The following table lists the CLI, Web, and MIBway VSB options and parameters.

WEBCLI

Enable or disable Web CLI base.

By default the WEBCLI option and all associated WEBCLI_ options are disabled.

Default: Disabled

Type: BOOL

WEBCLI Menu - System Specifications

WEBCLI_GENERAL_SYS_THREAD_OPT

Task spawn options.

Configures the mud task spawn options of taskSpawn( ).

The options include VX_FP_TASK, VX_SPE_TASK, VX_DSP_TASK and other BSP options. The required value is BSP-dependent.

Default: 0

Type: int

For example, since WEBCLI_GENERAL_SYS_THREAD_OPT is of type int, in order to set this option to VX_FP_TASK of value **0x1000000**, you actually need to set it to the integer value of **16777216**.

WEBCLI_GENERAL_SYS_THREAD_MULTI

This option determines whether the server is running in single threaded or multi-threaded mode.

Default: Enabled

Type: BOOL

WEBCLI_GENERAL_SYS_USE_ANSI

Use the American National Standards Institute (ANSI) C library.

Default: Enabled

Type: BOOL

WEBCLI_WM_INET6

Use the Internet Protocol version.

Default: Disabled

Type: BOOL

WEBCLI_GENERAL_SYS_MEM_HOST

Use the host memory management system.

Mutually exclusive with WEBCLI_GENERAL_SYS_MEM_ALLOC.

Default: Enabled

Type: BOOL

WEBCLI_GENERAL_SYS_MEM_ALLOC

Use Wind River middleware memory. The allocation is in bytes.

Mutually exclusive with WEBCLI_GENERAL_SYS_MEM_HOST.

Default: 256000

Type: int

WEBCLI Menu - Debugging Specifications

WEBCLI_GENERAL_DEBUG_WM

Define the macro **__WM_DEBUG__** to enable general CLI, Web, and MIBway debug messages.

Default: Disabled

Type: BOOL

WEBCLI_GENERAL_DEBUG_RRC

Define the macro **__RCC_DEBUG__** to enable CLI debug messages.

Default: Disabled

Type: BOOL

WEBCLI_GENERAL_DEBUG_HTTP

Define the macro **HTTP_DEBUG** to enable web server debug messages, in particular initialization messages.

Default: Disabled

Type: BOOL

WEBCLI_GENERAL_DEBUG_HTTP_SHOW

Define the macro **HTTP_SHOW** to enable the web server show routines such as httpListShow( ), httpAliasesShow( ) and httpFileSystemsShow( ).

For a complete listing of the web server show routines, see *installDir* **/vxworks-7/pkgs_v2/app/webcli/http/h/httpShow.h**.

Default: Enabled

Type: BOOL

WEBCLI_GENERAL_DEBUG_INFO

Define the macro **DEBUG_INFO** to enable additional debug messages. For instance, most of the RPMs print out a message when they are called if **DEBUG_INFO** is defined.

Default: Disabled

Type: BOOL

WEBCLI_GENERAL_DEBUG_WM_ERROR

Define the macro **__WM_ERROR_SHOW__** to enable the display of error messages.

Default:

Type:

WEBCLI Menu - File and Directory Specification

WEBCLI_GENERAL_FILE_MGR

Enable the file manager.

Default: Enabled

Type: BOOL

WEBCLI_GENERAL_SESSION_DIR

Enable the session directory.

Default: Enabled

Type: BOOL

WEBCLI_GENERAL_DIR_PATH

Enable the default directory path.

Default: **/romfs/**

WEBCLI_GENERAL_DIR_LEN

Set the maximum directory length in characters.

Default: 256

Type: int

WEBCLI_BACKPLANE

Enable or disable the database of WindMarks for the CLI, Web, and MIBway product components.

Default: Enabled

Type: BOOL

WEBCLI_CLI

Enable or disable the Wind River command line interpreter (CLI).

Wind River CLI provides you with a set of tools to quickly define a robust and extensible CLI, so that your customers can manage the device from a simple telnet shell, serial console, or SSH. Wind River CLI consists of a body of ANSI C source code that implements the CLI engine. It also includes a set of development tools that you use to configure the CLI engine code for your hardware and build the custom management command tree for your device.

Default: Enabled

Type: BOOL

WEBCLI_CLI Menu - Basic CLI Properties

WEBCLI_RCC_MAX_LOGIN_LEN

Define the maximum number of characters for a user name.

Default: 16

Type: int

WEBCLI_RCC_MAX_PASSWORD_LEN

Define the maximum number of characters for a password.

Default: 16

Type: int

WEBCLI_RCC_MAX_PROMPT_LEN

Define the maximum number of characters for a prompt.

Default: 64

Type: int

WEBCLI_RCC_ALIAS_BUFFER_SIZE

Define the CLI alias buffer size in bytes.

Default: 1024

Type: int

WEBCLI_RCC_ALIAS_COUNT

Define the number of CLI aliases.

Default: 3

Type: int

WEBCLI_RCC_MAX_CMD_LEN

Define the maximum number of characters for a CLI command.

Default: 254

Type: int

WEBCLI_CLI Menu - Advanced CLI Properties - Login Selection

Set the connection type for the CLI.

The options are mutually exclusive.

Select either Telnet, SSH, or Telnet and SSH. Requires IPNET_SSH for WEBCLI_SSH_LOGIN and WEBCLI_TELNET_SSH_LOGIN.

Type: BOOL

WEBCLI_TELNET_LOGIN

Default: Enabled

WEBCLI_SSH_LOGIN

Default: Disabled

Type:

WEBCLI_TELNET_SSH_LOGIN

Default: Disabled

WEBCLI_CLI Menu - Advanced CLI Properties - Parameter Order

Set the CLI parameter order. These options are mutually exclusive.

Type: BOOL

WEBCLI_RCC_PARAMETER_ORDER_NONE

To allow parameter entry as name and value in any order, select WEBCLI_RCC_PARAMETER_ORDER_NONE.

Default: Disabled

WEBCLI_RCC_PARAMETER_ORDER_UNNAMED

To allow parameter entry as value only and in the proper order, select WEBCLI_RCC_PARAMETER_ORDER_UNNAMED.

Default: Enabled

WEBCLI_RCC_PARAMETER_ORDER_FULL

To force parameter entry as name and value in the proper order, select WEBCLI_RCC_PARAMETER_ORDER_FULL.

Parameter ordering must be set to WEBCLI_RCC_PARAMETER_ORDER_FULL to provide scripting support and **stdio.h** support.

Default: Disabled

WEBCLI_CLI Menu - Advanced CLI Properties - Options

WEBCLI_RCC_SKIP_LOGIN

Enable or disable the omission of the login prompt.

The default setting (Disabled) enables the login prompt.

Default: Disabled

Type: BOOL

WEBCLI_RCC_SERIAL_CONSOLE

Enable or disable serial console support for the CLI component.

Default: Enabled

Type: BOOL

WEBCLI_RCC_STDIO_REDIRECTION

Enable or disable redirection of CLI stdout to a telnet socket.

Default: Disabled

Type: BOOL

WEBCLI_RCC_FILTER_IO

Enable or disable the input/output filter for redirect mode.

Default: Disabled

Type: BOOL

WEBCLI_RCC_INTERRUPT_EXEC

Enable or disable the ability to interrupt command execution.

Default: Enabled

WNDRVR

Type: BOOL

WEBCLI_CLI_SCRIPTING

Enable or disable script support for the CLI component.

Default: Disabled

Type: BOOL

WEBCLI_CLIDEMO

Enable or disable the command line interpreter (CLI) demonstration code.

This option is mutually exclusive with WEBCLI_WEBDEMO and WEBCLI_WEBCLIDEMO.

Default: Disabled

Type: BOOL

WEBCLI_COMMON

Enable or disable the common utilities for backplane, CLI, HTTP, and MIBway.

Default: Enabled

Type: BOOL

WEBCLI_COMMON Menu

WEBCLI_ENABLE_WMB_USERS

Enable or disable Wind River management backplane users (WMB_USERS) in the common component.

Default: Enabled

Type: BOOL

WEBCLI_INCLUDE_OPENSSL_SUPPORT

Enable or disable Open SSL support for the HTTPS feature of the common component.

Requires OPENSSL.

Default: Disabled

Type: BOOL

WEBCLI_HTTP_SSL_PORT

Define the SSL port number.

Requires WEBCLI_INCLUDE_OPENSSL_SUPPORT.

Default: 443

Type: int

WEBCLI_HTTP_SSL_CA_FILE

Define the file name of the SSL certificate authority file.

Requires WEBCLI_INCLUDE_OPENSSL_SUPPORT.

Default: NULL

Type: String

WEBCLI_HTTP_SSL_CA_PATH

Define the file path to the SSL certificate authority file.

Requires WEBCLI_INCLUDE_OPENSSL_SUPPORT.

Default: NULL

Type: String

WEBCLI_HTTP_SSL_CERTIFICATE_FILE

Define the file name and path of the SSL certificate file.

Requires WEBCLI_INCLUDE_OPENSSL_SUPPORT.

Default: **/romfs/htdocs/se rver.pem**

Type: String

WEBCLI_HTTP_SSL_CIPHER_LIST

Define the SSL cipher string.

Requires WEBCLI_INCLUDE_OPENSSL_SUPPORT.

Default: DEFAULT

Type: String

WEBCLI_HTTP_SSL_PRIVATE_KEY_FILE

Define the file name and path of the SSL private key.

Requires WEBCLI_INCLUDE_OPENSSL_SUPPORT.

Default: **/romfs/htdocs/server.pem**

Type: String

WEBCLI_HTTP_SSL_V2

Enable or disable support for Secure Sockets Layer (SSL) version 2.

Requires WEBCLI_INCLUDE_OPENSSL_SUPPORT.

Default: Disabled

Type: BOOL

WEBCLI_HTTP_SSL_V3

Enable or disable support for Secure Sockets Layer (SSL) version 3.

Requires WEBCLI_INCLUDE_OPENSSL_SUPPORT.

Default: Disabled

Type: BOOL

WEBCLI_HTTP_TLS_V1

Enable or disable support for Transport Layer Security (TLS) version 1.

Requires WEBCLI_INCLUDE_OPENSSL_SUPPORT.

Default: Disabled

Type: BOOL

WEBCLI_MD5_ENABLED_COMMON

Enable or disable support for Message Digest 5 (MD5) in the common component.

Default: Enabled

Type: BOOL

WEBCLI_INCLUDE_HTTP_CGI_COMMON

Enable or disable support for HTTP Common Gateway Interface (CGI) RPMs in the common component.

Default: Enabled

Type: BOOL

WEBCLI_LIBCURL

Enable or disable support for the client-side URL transfer library with URL syntax.

The Wind River HTTP client is based on the open-source client-side URL transfer library libcurl. The HTTP client allows you to set up a session with a server using a protocol such as HTTP or HTTPS. Using the libcurl library, you can write applications for transferring files (using the GET and POST methods) with URL syntax.

Default: Enabled

Type: BOOL

WEBCLI_HTTP

Enable or disable the HTTP protocol request process modules (RPMs) for the web server. Wind River Web Server consists of a scalable embedded HTTP server and tools for creating a web-based device management interface.

Default: Enabled

Type: BOOL

WEBCLI_HTTP Menu - Advanced Web Server Properties

WEBCLI_JAVASCRIPT_DIGEST_ENABLED

Enable or disable javascript authentication.

Default: Enabled

Type: BOOL

WEBCLI_kHwIdleTimeoutSecs

Define the secure session timeout (seconds).

Default: 120

Type: int

WEBCLI_kHwMaxSimultaneousClients

Define the maximum number of simultaneous clients.

Default: 10

Type: int

WEBCLI_kJavaScriptLogonPage.

Define the logon web page file name for authentication

Default: **logon.html**

Type: string

WEBCLI_INCLUDE_HTTP_LIVE_CONTROL

Enable or disable support for live control java processing.

Default: Enabled

Type: BOOL

WEBCLI_HTTP_LC_POLL_MED

Define the medium priority polling period (milliseconds).

Default: 500

Type: int

WEBCLI_HTTP_LC_POLL_HGH

Define the high priority polling period (milliseconds).

Default: 300

Type: int

WEBCLI_INCLUDE_SMTP_ENABLE

Enable or disable support for Simple Mail Transfer Protocol (SMTP) processing.

Default: Enabled

Type: BOOL

WEBCLI_HTTP_ETAG_ENABLE

Enable or disable support for the HTTP entity tag (ETAG).

Default: Enabled

Type: BOOL

WEBCLI_STANDALONE_WEBSERVER

Enable or disable the instantiation of a standalone HTTP Server. Requires WEBCLI_HTTP and WEBCLI_COMMON.

Mutually exclusive with WEBCLI_CLI, WEBCLI_BACKPLANE, WEBCLI_MIBWAY, WEBCLI_CURL, WEBCLI_INCLUDE_SMTP_ENABLE, WEBCLI_INCLUDE_HTTP_LIVE_CONTROL, WEBCLI_INCLUDE_HTTP_UPLOAD, WEBCLI_INCLUDE_HTTP_POST, WEBCLI_ENABLE_WMB_USERS, and WEBCLI_JAVASCRIPT_DIGEST_ENABLED.

Default: Disabled

Type: BOOL

WEBCLI_HTTP Menu - RPM configuration Web Server

WEBCLI_INCLUDE_HTTP_CGI .

Enable or disable Common Gateway Interface (CGI) for RPMs

Default: Enabled

Type: BOOL

WEBCLI_INCLUDE_HTTP_UPLOAD

Enable or disable support for file uploads for RPMs.

Default: Enabled

Type: BOOL

WEBCLI_INCLUDE_HTTP_LOG

Enable or disable log file support for RPMs.

Default: Enabled

Type: BOOL

WEBCLI_HTTP_LOG_FILE

Define the location of the log file for RPMs.

Default: **/logfile.txt**

Type: string

WEBCLI_HTTP_LOG_FILE_SIZE_MAX_BYTES

Define the maximum size of the log file in bytes.

Default: 100000

Type: int

WEBCLI_INCLUDE_HTTP_MIME_HDR_PROC

Enable or disable support for Multipurpose Internet Mail Extensions (MIME) header processing.

Default: Enabled

Type: BOOL

WEBCLI_INCLUDE_HTTP_MIME_GUESSTYPE

Enable or disable support for MIME type guessing.

Default: Enabled

Type: BOOL

WEBCLI_INCLUDE_HTTP_FS

Enable or disable file system support for RPMs.

Default: Enabled

Type: BOOL

WEBCLI_INCLUDE_HTTP_ALIASING

Enable or disable support for RPM aliasing.

Default: Enabled

Type: BOOL

WEBCLI_INCLUDE_HTTP_ACCESS_CTRL

Enable or disable access control support for RPMs.

Default: Enabled

Type: BOOL

WEBCLI_INCLUDE_HTTP_IP_CHECKING

Enable or disable support for IP checking.

Default: Enabled

Type: BOOL

WEBCLI_INCLUDE_HTTP_PWD_CHECKING

Enable or disable support for password checking.

Requires INCLUDE_HTTP_MIME_HDR_PROC.

Default: Enabled

Type: BOOL

WEBCLI_INCLUDE_HTTP_SHOWAUTH

Enable or disable support for authentication information.

Default: Enabled

Type: BOOL

WEBCLI_INCLUDE_HTTP_WMB_INTERFACE

Enable or disable WebSphere Message Broker (WMB) support for RPMs.

Default: Enabled

Type: BOOL

WEBCLI_INCLUDE_HTTP_POST

Enable or disable form post RPM.

Default: Enabled

Type: BOOL

WEBCLI_HTTP Menu - File System Image Generation

WEBCLI_ROMBUILD_IMAGE

Enable or disable the build flag for the non-volatile memory (NVM) file system.

Default: Enabled

Type: BOOL

WEBCLI_HTTP_SECTOR_SIZE

Define the sector size of the read-only memory (ROM) build file system non-volatile random-access memory (NVRAM).

Default: **0x00010000**

Type: Hexadecimal

WEBCLI_HTTP_NVRAM_SIZE

Define the memory size of the ROM build file system NVRAM.

Default: 0x00080000

Type: Hexadecimal

WEBCLI_HTTP_NVRAM_START

Define the start address of the ROM build file system NVRAM.

Default: 0x00000000

Type: Hexadecimal

WEBCLI_MIBWAY

Enable or disable support for a bridge to the SNMP agent from the CLI backplane.

MIBway allows web-based and command-line interfaces to access your SNMP agent as if the browser or telnet session was an SNMP management station.

If you are planning to enable SNMP in your project, you must build the SNMP user libraries as part of your VSB project. For more information on SNMP VSB components, see the VxWorks 7 SNMP Programmer's Guide.

Default: Disabled

Type: BOOL

WEBCLI_WEBCLIDEMO

Enable or disable support for the web server and simple CLI demonstration code.

This option is mutually exclusive with WEBCLI_CLIDEMO and WEBCLI_WEBDEMO.

Default: Disabled

Type: BOOL

WEBCLI_WEBDEMO

Enable or disable support for the web server demonstration code.

This option is mutually exclusive with WEBCLI_CLIDEMO and WEBCLI_WEBCLIDEMO.

Default: Disabled

Type: BOOL

5. Additional VSB Options

Other options for related CLI, Web, and MIBway functionality are available in addition to the standard VxWorks source build (VSB) configuration options.

Table 1. Additional VSB Options for CLI, Web, and MIBway

Name	Description
OPENSSL	Enable the Secure Socket Layer (SSL). This option must be enabled if WEBCLI_CURL or WEBCLI_INCLUDE_OPENSSL_SUPPORT is enabled. For more information, see the VxWorks 7 SSL Programmer's Guide.
PNET_SSH	Enable secure shell (SSH). This option is required for WEBCLI_SSH_LOGIN and WEBCLI_TELNET_SSH_LOGIN. For more information, see the VxWorks 7 SSL Programmer's Guide.
SNMP	Enable Simple Network Management Protocol (SNMP). For more information, see VxWorks 7 SSL Programmer's Guide.

6. Kernel Configuration Components (VIP)

You can configure your kernel with CLI, Web, and MIBway support.

The following table lists the available CLI, Web, and MIBway VxWorks image project (VIP) components. The corresponding VxWorks source build (VSB) option must be enabled to access each component, see Source Build Configuration Options (VSB) on page 38.

Table 1. CLI, Web, and MIBway VIP Components

Name	Description
INCLUDE_WEBCLI_BACKPLANE	Provide support for a database of WindMarks for the Wind River CLI, Web, and MIBway product components.
INCLUDE_WEBCLI_CLI	Provide support for the Wind River command line interpreter (CLI).
INCLUDE_WEBCLI_CLIDEMO	Provide support for the command line interpreter (CLI) demonstration code.
INCLUDE_WEBCLI_COMMON	Provide support for common utilities related to the backplane, CLI, HTTP, and MIBway.
INCLUDE_WEBCLI_CURL	Provide support for the client-side URL transfer library with URL syntax.
INCLUDE_WEBCLI_HTTP	Include the HTTP protocol request process module.
INCLUDE_WEBCLI_MIBWAY	Provide support for web-based and CLI access to an SNMP agent.
INCLUDE_WEBCLI_WEBCLIDEMO	Provide support for the web server and simple CLI demonstration code.
INCLUDE_WEBCLI_WEBDEMO	Provide support for the web server demonstration code.

7. Additional VIP Components

Additional VxWorks image project (VIP) components are available for CLI, Web, and MIBway-related functionality.

Table 1. Additional VSB Options for CLI, Web, and MIBway

Name	Description
INCLUDE_IPSSL	Include the Secure Socket Layer (SSL). This component must be included if you use the HTTP client. For more information, see the VxWorks 7 SSL Programmer's Guide.
INCLUDE_IPSSH	Include secure shell. For more information, see the VxWorks 7 SSL Programmer's Guide.
INCLUDE_TLS	Include thread variables support. This component provides thread-local storage, a compiler facility that allows for allocation of a variable

Name	Description
	such that there are unique instances of the variable for each thread (or task, in VxWorks terms). This component must be included if you use symmetric multiprocessing (SMP).
INCLUDE_IPWRAP_GETHOSTBYNAME	This component provides access to the backwards compatibility function, **gethostbyname( )**. This component must be included if you use the HTTP client.
INCLUDE_POSIX_TIMERS	This component provides access to the Portable Operating System Interface (POSIX) timers. This component must be included if you need the HTTP client.
INCLUDE_ZLIB	This component provides support for the common zlib compression libraries. This component must be included if you use the HTTP client.

4. API FUNCTIONS

1. CLI, Web, and MIBway API Libraries

Short reference description.

The table below lists the API libraries available to CLI, Web, and MIBway applications. The libraries are available from the API reference entries.

The source code for the API functions are in the following directories:

- *installDir* /**vxworks-7/pkgs_v2/app/webcli/backplane-***x.x.x.x* **/src/**
- *installDir* **/vxworks-7/pkgs_v2/app/webcli/cli-***x.x.x.x* **/src/**
- *installDir* **/vxworks-7/pkgs_v2/app/webcli/common-***x.x.x.x* **/src/**
- *installDir* **/vxworks-7/pkgs_v2/app/webcli/http-***x.x.x.x*/ **src/**
- *installDir* **/vxworks-7/pkgs_v2/app/webcli/mibway-***x.x.x.x* **/src/**

Table 1. CLI, Web, and MIBway API Libraries

Library Name and Filename	Description
rcc	The main CLI library for Wind River CLI Management.
httpFileProcess	This library provides the basic API functions used by SMTP to retrieve the email alert file and parse it for WindMarks before sending the parsed data to the SMTP server.
httpAuthUtils	Access functions library for authentication.
httpBase64Util	Utility library for handling base 64 encoded data.
httpData	Access functions for operating system abstraction layer (OSAL) date functions.
httpErrorHandling	This module provides information and error message logging for the HTTP server.
httpFileMgr	This module provides a unified file access API that can read ANSI files as well as non-volatile files generated with Rombuild. It can be easily expanded to support custom file systems.
httpFileRegistry	This module implements registry functions for files that need special handling. This will typically be all files using the file system type **CUSTOM_FS**, and any files in other file systems that require non-default access functions.
httpGenericListUtil	This module provides generic list management utilities.
httpIsMatching	String pattern-matching functions for the HTTP server. For example, to match URLs to patterns.

Library Name and Filename	Description
httpLib	The main module of the HTTP server. Its task is to listen on the sockets stored in an array.
httpLineUnescape	Convert escape characters in buffer to ASCII.
httpIo	Abstraction library for HTTP socket tasks.
httpMem	This module provides pool memory management functions.
httpMemGzio	Access functions for compressed files of the ROM build file manager.
httpMimeAcceptEncoding	This module contains HTTP_REQ access functions useful for MIME Accept-Encoding: determination.
httpMimeCookie	This module contains HTTP_REQ access functions which are useful for server cookie handling.
httpMimeParse	This module contains the MIME header processing facilities. If this feature is enabled in the httpconf.h configuration file, custom and system MIME header processing functions may be registered at this module and can parse and store any MIME header information.
httpOutput	This module provides output generating functions.
httpParseUtils	This module provides utilities for parsing argument strings.
httpRpmDispatch	The RPM dispatcher calls the RPMs in the configured order with the **HTTP_REQ_ID** handle as an argument. Each RPM must return a processing status to the dispatcher indicating how further processing of this request should be done.
httpStrCaseCmp	This module provides utility functions for the HTTP server.
httpStringFuncs	This module provides string utility functions for the HTTP server.
httpWmbExpandInstance	This module implements the expansion of Wind Manage wind instances.
httpWmbHandlers	Wind River backplane handlers.
lstLib	This module supports the creation and maintenance of a doubly linked list.
wmAccessLib	This module implements access control support for Wind Manage.
wmBufferLib	This module contains support routines for Wind Manage products.
wmCompareLib	Comparison test functions for RCB data.

Library Name and Filename	Description
wmConvertLib	This module implements various data conversion functions.
wmUserLib	This module implements multi-user access functions.
wmbObjectLib	Backplane object management library.

5. WORKING WITH THE MANAGEMENT CONFIGURATION EDITOR (MCE)

1. Introduction

The starting point for all configuration activities is the Web CLI Configuration root object located in the management configuration editor (MCE).

To enable and access the MCE from Workbench 4, see Enabling the MCE from Workbench 4 on page 37 and Accessing the MCE from Workbench 4 on page 37.

For an overview of the project creation process, see Configuration and Build Workflow on page 36.

2. Adding and Removing Components

The Web CLI menu allows you to view, add and remove components.

About This Task

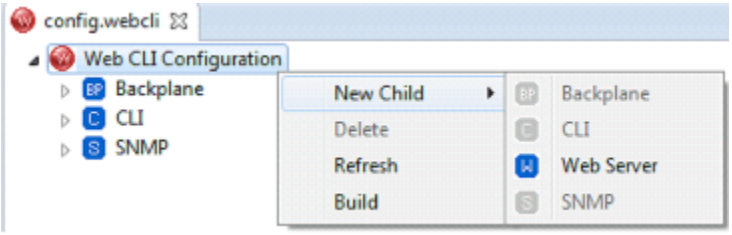
Procedure

1. To view the existing components in your project, click the **Web CLI Configuration** drop-down arrow.

   The current objects are displayed.

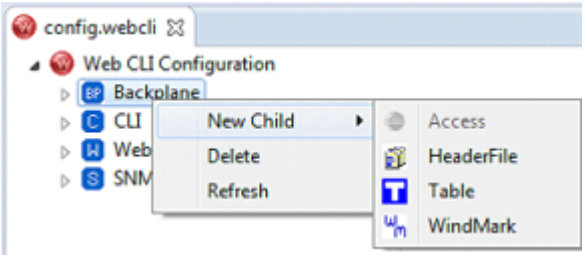
2. To view objects that can be created, right-click **Web CLI Configuration** and select **New Child**.

   In the following example, the Backplane, CLI, and SNMP objects are present and the Web Server object has not been created:



   The MCE is based on an object model hierarchy. Component objects and related elements are added, deleted, rearranged, and copied within an object tree using right-click context menus and drag-and-drop operations. Properties are set or changed by selecting an object and then using the property page associated with the object to change settings or values.

   Objects have properties associated with them, which you may modify. Objects appear in the tree as parent objects and child objects, for example:



3. Access a config.webcli tab and right-click the root object to display and choose the child and sibling objects that are available.

   Selecting a child object from the menu creates the object, and a property page (if appropriate) is displayed.

   The following tree lists objects that may be created. The objects are ordered by their parent/child relationships. The number in parentheses indicates the maximum number of that object type that may be created. Note that you do not need to create all available objects to have a valid configuration.

```
+ - - Web CLI Configuration
         + - - Backplane (1)
          |       + - - Access (1)
          |        |         | - - Accessgroup (unlimited)
          |        | - - Header File (unlimited)
          |       + - - Table (unlimited)
          |        |          | - - WindMark (unlimited)
          |        |                  | - - Enumeration (unlimited)
          |       + - - WindMark (unlimited)
          |                  | - - Enumeration (unlimited)
         + - - CLI (1)
          |       + - - Command (unlimited)
          |                  | - - Command (unlimited)
          |                  | - - Parameter (unlimited)
          |                 + - - Handler (unlimited)
          |                           | - - Handler Parameter (unlimited)
         + - - Web Server (1)
          |       + - - URL (unlimited)
          |        |         | - - ACL Group/Location Pairs (unlimited)
          |       + - - Web Files (1)
          |        |         | - - File (unlimited)
          |       + - - Access Control List (1)
          |        |        + - - User Groups (1)
          |        |         |      + - - Group (unlimited)
          |        |         |                | - - User (unlimited)
          |        |        + - - IP Locations (1)
          |        |                + - - Location (unlimited)
          |        |                          | - - IP Address (unlimited)
          |       + - - Custom RPMs(1)
          |        |         | - - Custom RPM (unlimited)
          |       + - - Custom MIMEs(1)
          |        |         | - - Custom MIME (unlimited)
         + - - SNMP (1)
                  + - - MIB Files (1)
                            | - - File (unlimited)
```

4. Click on objects in the left pane of the editor to navigate between objects.
5. Continue creating new objects by using the right-click context menu on existing objects until you have created a configuration you are happy with.
6. To delete an object, right-click the object and select **Delete**, or select the object and press **Delete**.

To delete more than one object, highlight the desired objects, right click them, and select **Delete**, or select the objects and press **Delete**.

7. Validate your project periodically by saving it.

   During project validation, incorrect, invalid, and inconsistent values are flagged with a red box surrounding the field. When this occurs, the project cannot be built until the problem is rectified. Some fields that have errors provide tooltip hints describing the problem. If an error tooltip exists, you can access it by placing the mouse pointer over the field for a few moments.

   Note: You must configure all feature-specific build options using a VSB project and the MCE. Wind River CLI,Web, MIBway does not make use of any build macros passed to make on the command line.

3. Configuring the Project-Level Settings

About This Task

To configure your project-level settings, do the following:

Procedure

1. In the left pane of the MCE, click on **Web CLI Configuration**.
2. Enter the project name in the **Name for Project** field.
3. Enter code in the Extensions to **wm_options.h** panel.

   Any code you enter in the Extensions to **wm_options.h** panel is added verbatim to the **wm_options.h** file, which is included by all other Wind River CLI,Web, MIBway source files.

   The CLI engine and Web Server component settings each have an extensions section of their own. The custom code you enter in the CLI extensions gets added to the generated file **wmc_options.h**, and the custom code you enter in Web server extensions gets added to the generated file **wmw_httpconf.h**.

   For more information on CLI or Web Server extensions, see Adding an SNMP Object to Your Project on page 64.

Converting an MCE Project to a WMIT Project on page 59 For backwards compatibility, you can convert an MCE project to a Wind River Management Integration Tool (WMIT) project.
Converting a WMIT Project to an MCE Project on page 60

3.1. Converting an MCE Project to a WMIT Project

For backwards compatibility, you can convert an MCE project to a Wind River Management Integration Tool (WMIT) project.

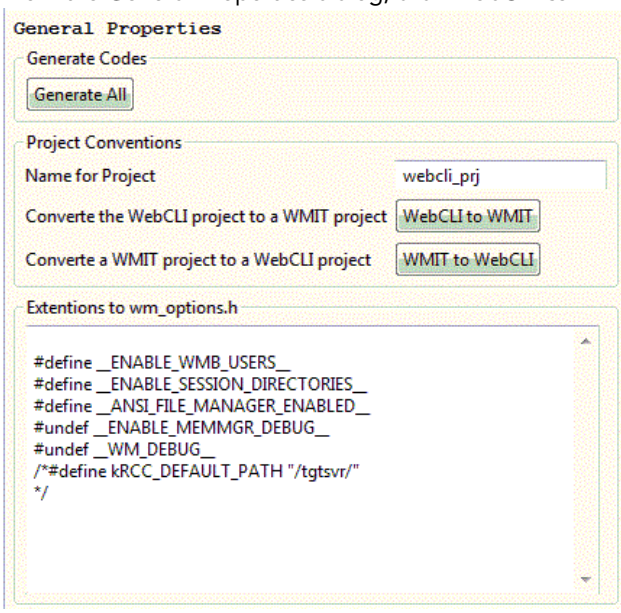
About This Task

Note: VxWorks 7 does not support project editing with WMIT.

To convert your MCE project to a WMIT project, do the following:

Procedure

1. Save your project.
2. Select **WEB CLI Configuration** in the left pane of the MCE.
3. From the General Properties dialog, click **WebCLI to WMIT**.



Results

The output files, including the WMIT project file (projName.rcp), are located in the src directory of your project. For example:

installDir **/vxworks-7/pkgs_v2/app/webcli/clidemo/src/**

installDir **/vxworks-7/pkgs_v2/app/webcli/webdemo/src/**

installDir **/vxworks-7/pkgs_v2/app/webcli/webclidemo/src/**

Parent topic: Configuring the Project-Level Settings on page 59

Related information
Converting a WMIT Project to an MCE Project on page 60

3.2. Converting a WMIT Project to an MCE Project

About This Task

To convert a WMIT project back to MCE, proceed as follows:

Procedure

1. Select **WEB CLI Configuration** in the left pane of the MCE.
2. click **WMIT to WebCLI** from the General Properties dialog.

Parent topic: Configuring the Project-Level Settings on page 59

Related information
Converting an MCE Project to a WMIT Project on page 59

4. Configuring the Backplane

Whether you are creating a CLI or a Web Server project (or both), you must include a backplane.

About This Task

To create a backplane object with MCE and configure the backplane for use with your project, do the following:

Procedure

1. Right-click **Web CLI Configuration** and select **New Child > Backplane**.

   A new child Backplane object appears beneath **Web CLI Configuration**. The Backplane Properties dialog box appears in the right pane of the editor.

2. Set the properties associated with the backplane options.

   The properties are as follows:

   - Name for this backplane
   - Deregistration Handler
   - Deregistration Cookie
   - Initial Max Resources
   - Initial Max Components

   For details on these options, see The Backplane Properties Dialog Box on page 131.

5. Configuring the CLI

Before you can create a command-line interface for your project, you must first add and configure a CLI object.

About This Task

To create a CLI object with MCE and configure CLI for use with your project, do the following:

Procedure

1. Right-click **Web CLI Configuration** and select **New Child > CLI**.

WNDRVR

A new child CLI object appears beneath Web CLI Configuration. The properties associated with CLI appear in the right pane of the editor.

2. Select the General tab to set the general CLI properties.

   The General tab includes the following:

   - telnet options
   - login options
   - command options
   - shell options

   For details on these options, see General CLI Properties Dialog Reference on page 139.

3. Select the Advanced tab to configure the behavior of the CLI engine's command interpreter.

   The Advanced tab includes the following:

   - command options
   - help options
   - shell options
   - display options
   - miscellaneous options

   For details on these options, see Advanced CLI Properties Dialog Reference on page 142.

4. Select the CLI Import tab to import and add CLI commands from other MCE configurations. To import commands, do the following:
   1. Click Import CLI Commands.
   2. Navigate to the **config.webcli** file for the project you would like to import commands from.
5. Select the CLI Global Setting tab to set the CLI global settings.

   The CLI Global Settings dialog box allows you to enter code that will be inserted at the top of the command files. For example, if you want to add a #define to **wmc_projectName.h**, type the define in the **wmc_proj.h Preamble** text box, and the #define is inserted at the top of **wmc_projectName.h** every time it is generated by the MCE.

6. Select the Extensions tab to add your own code to the generated configuration files.

   The custom code you enter in CLI extensions gets added to the generated file **wmc_options.h**.

6. Configuring the Web Server

Before you can create a Web server interface for your project, you must first add and configure a Web Server object.

About This Task

To create a Web Server object with MCE and configure the Web Server for use with your project, do the following:

WNDRVR

Procedure

1. Right-click **Web CLI Configuration** and select **New > Child Web Server**.

   A new child Web Server object appears beneath Web CLI Configuration. The properties associated with the Web Server appear in the right pane of the editor.

2. Select the Basic tab to configure the basic Web Server properties.

   The Basic tab contains the following:

   - server identification options
   - port options
   - HTTP error messages options
   - basic settings options
   - WindMark options
   - system RPM options

   For details on these options, see Basic Web Server Properties Reference on page 152.

3. Select the Registration tab to change reserved registrations for run time.

   When you generate your embedded HTTP server configuration files, many compile macro values are determined automatically by the MCE. For example, if you configured five URL aliases, the generated file **wmw_httpconf.h** would contain a line similar to the following:

   ```
   #define HTTP_ALIAS_CONF_MAX 5 /* max # of alias definitions */
   ```

   The **Registration** tab allows you to increase the values of these macros to a fixed number beyond that determined by the MCE. If you configure five URL aliases using the MCE, and then enter the value 10 for the Number of Alias Definitions, the result in the generated file would be the configured number plus 10; or in this case, 15.

   ```
   #define HTTP_ALIAS_CONF_MAX 15 /* max # of alias definitions */
   ```

   For details on the options available on the Registration tab, see Web Server Registration Properties Reference on page 155.

4. Select the Resources tab to set socket, task, and memory options.

   The Resources tab contains the following:

   - socket options
   - task options
   - memory options
   - file system options
   - presets for socket, task, and memory properties

   For details on these options, see Web Server Resource Properties Reference on page 158.

   You must choose either ANSI or NVM as the file system. This selection does not indicate the mount point or anything particular about the actual file system you will use. It simply indicates which access functions the server (the file system RPM) should use to access nonregistered files on the first attempt. The phrase nonregistered files is important here because registered files are

WNDRVR

accessed by their registered access method regardless of what is selected as the default file system. For more information about file systems, the file system search order, and the file registry, see Working with Web Content Files on page 122.

The physical location/mount point of the default file system is configured in the file system RPM. The configuration files for the file system RPM are part of the generated code. See Creating a Web-Based Management Interface on page 106.

5. Select the Extensions tab to add your own code to the generated configuration files.

   The custom code you enter in Web server extensions gets added to the generated file **wmw_httpconf.h**.

7. Adding an SNMP Object to Your Project

If your project will utilize SNMP and SNMP WindMarks, you must add an SNMP object to your project.

About This Task

To add an SNMP object to your project with MCE.

Procedure

Right-click **Web CLI Configuration** and select **New Child > SNMP**. A new child SNMP object appears beneath **Web CLI Configuration**.

Once you have created an SNMP object, you can automatically generate WindMarks that correspond to MIB objects when you import a MIB file into your project. For more information, see Creating MIBway (SNMP) WindMarks on page 78.

If you enable SNMP in your project, you must build the SNMP user libraries as part of your VxWorks source build (VSB) project. For more information on building your VSB, see Configuration and Build on page 36. For more information on SNMP VSB components, see the VxWorks 7 SNMP Programmer's Guide.

8. Customizing the Sample Implementation

In addition to a configured project file, there are certain features of the embedded code behavior that you must provide before you can build.

Wind River Web Server requires you to provide your own version of the httpWindMarkGetFailed( ), httpWindMarkSetFailed( ), and httpPostReply( ) custom server response functions.

Sample implementations of these functions are available in sampleHttpUserFuncs.c located at:

installDir **/vxworks-7/pkgs_v2/app/webcli/webdemo-x.x.x.x/src/**

and

installDir **/vxworks-7/pkgs_v2/app/webcli/webclidemo-x.x.x.x/src/**

httpWindMarkGetFailed( )

> This function implements the HTTP server behavior in cases where the backplane reports WindMark read errors. In some applications, you may prefer to just ignore the WindMark silently. In those cases, the function should just return a pointer to an empty string. In other cases, you may prefer an error message printed in (or in place of) the HTML page.

The sample implementation provided in **sampleHttpUserFuncs.c** prints an error message in the HTML page along with the errno code.

httpWindMarkSetFailed( )

By default, WindMark SET attempts that fail remain in the request environment, so that you can retrieve them later, when httpPostReply( ) is called.

This function is called right after each failed SET attempt so you can store more information about the error in the environment, such as the errno or a custom error message. The information stored at this point can be retrieved by httpPostReply( ) (also a user-defined function) by calling httpWmbErrorIterator( ).

The sample implementation provided in **sampleHttpUserFuncs.c** prints an error page in place of the referring HTML page (along with the errno code) for each error instance.

httpPostReply( )

The POST reply function specifies the behavior and appearance of the response HTML after a HTTP POST request completes. This function is required to complete HTTP POST operations.

The sample implementation provided in **sampleHttpUserFuncs.c** provides different behavior based on the type and HTTP error status of the active request.

9. Building a MCE Project for CLI, Web, and MIBway

After you have created and configured your project, you must build it.

About This Task

Do the following to build the project:

Procedure

Right-click your project from **Web CLI Configuration** and select **Build** or click **Web CLI Configuration** and click **Generate All** in the General Properties dialog box.

Building the project results in the generation of the following files:

- **wm_options.h**: This file contains the global configuration options. There is only one **wm_options.h** file for each workspace.
- **wmb_projName.c/h**: These files contain backplane configuration settings, component registration/deregistration handlers, WindMark handlers, and backplane access control settings.

  These files also contain the WindMark identification and object action case statements, which you may modify directly. If you choose to initialize variables within the generated code, there are sections set aside with begin and end special comment tags.

  In the generated files, **wmb_projName.c/h**, the code that is intended for you to manipulate is delimited by begin and end comments:

  ```
  /* %%BeginUserCode */
  /* %%EndUserCode */
  ```

  This method of tagging code segments with a begin and end point also keeps these segments from being overwritten when new code is generated. To remove these segments, delete the code segments and regenerate the files.

  These two files are an exception to the general rule that you should avoid manually editing generated files. In fact, the **wmb_projName.c/h** files contain specified areas for custom declarations, and so forth. Be sure to manually edit only in the appropriate user blocks of these files.

- **wmw_httpconf.c/h**: These files contain configuration information for the embedded HTTP server.
- **wmc_projName.c**: This file contains the full command note tree, plus associated parameter and handler definitions.
- **wmc_projName.h**: This file contains prototypes and parameter flags.
- **wmc_options.h**: This file contains configuration information for your command-line interface.

  rcc_handler.c: This file contains the listing of handler functions that are invoked by the command.
- **rcc_handler.h**: This file contains all the function prototypes specified CLI commands in the MCE.

Note: The underscores in file names indicate that a particular source file is generated by the MCE. You should avoid manually editing any generated file (except **wmb_projectName.c** and **wmb_projectName.h**, which has provisions to save your edits). For custom code that needs to be included in other generated files, use extensions (see Configuring the Project-Level Settings on page 59) for adding code verbatim to generated files.

6. CUSTOMIZING YOUR ENVIRONMENT

1. Introduction

You can customize any of the demonstration environments to suit your requirements.

This section describes how to adapt the CLI Tool and Web Server demonstration to suit your environment (for more information, see the VxWorks 7 Creating a CLI Tool and Web Server Tutorial).

Web Server Customization

To modify the server, you should customize the following files:

- **sampleHttpUserFuncs.c**
- **sampleJSUser.c**
- **sampleJSUser.h**
- **sampleUserCgi.c**
- **sampleUserCgi.h**
- **sampleUserRpm.c**
- **sampleUserRpm.h.**

The standalone embedded HTTP server is a non-typical use case. If you enable the WEBCLI_STANDALONE_WEBSERVER option in the VSB, you must also enable the WEBCLI_HTTP and WEBCLI_COMMON VSB options.

You must also disable WEBCLI_CLI, WEBCLI_BACKPLANE, WEBCLI_MIBWAY, WEBCLI_LIBCURL, WEBCLI_INCLUDE_SMTP_ENABLE, WEBCLI_INCLUDE_HTTP_LIVE_CONTROL, WEBCLI_INCLUDE_HTTP_UPLOAD, WEBCLI_INCLUDE_HTTP_POST, WEBCLI_ENABLE_WMB_USERS, and WEBCLI_JAVASCRIPT_DIGEST_ENABLED.

To add HTTPS functionality, you must enable OPENSSL in the VSB and add INCLUDE_IPSSL to the VIP.

To add Web server functionality without CLI functionality, enable WEBCLI_WEBDEMO in the VSB and add INCLUDE_WEBCLI_WEBDEMO to the VIP. For more information, see the VxWorks 7 Creating a Web Server Tutorial.

CLI Tool Customization

To modify the CLI tool, you should customize the following files:

- **usrCustomFunc.c**
- **usrCustomFunc.h**

For SSH functionality, enable IPNET_SSH in the VSB and add INCLUDE_SSH to the VIP.

To add CLI functionality without Web server functionality, enable WEBCLI_CLIDEMO in the VSB and add INCLUDE_WEBCLI_CLIDEMO to the VIP. For more information, see the .VxWorks 7 Creating a CLI Tool Tutorial

2. Customizing the CLI Tool and Web Server Demonstration

You can customize the CLI tool and Web server demonstration.

About This Task

To customize the demonstration presented in the VxWorks 7 Creating a CLI Tool and Web Server Tutorial, proceed as follows:

Procedure

1. Make a backup of the *installDir* **/vxworks-7/pkgs_v2/app/webcli-***x.x.x.x* **/webclidemo-***x.x.x.x* folder.

   At any time you can reinstate the original folder to recreate the demonstration.

   > Note: If you rename the **webclidemo-***x.x.x.x* folder, some file names and symbol names must be changed in the following files:
   >
   > *installDir* **/vxworks-7/pkgs_v2/app/webcli-***x.x.x.x* **/webclidemo-***x.x.x.x* **/src/Makefile**
   >
   > *installDir* **/vxworks-7/pkgs_v2/app/webcli-***x.x.x.x* **/webclidemo-***x.x.x.x* **/cdf/40webclidemo.cdf** and
   >
   > *installDir* **/vxworks-7/pkgs_v2/app/webcli-***x.x.x.x* **/common-***x.x.x.x* **/cdf/40webcli.cdf**
   >
   > The **CliInit.c**, **HttpInit.c** and **CommonInit.c** files must be copied to the user layer.

2. Modify the *installDir* **/vxworks-7/pkgs_v2/app/webcli-***x.x.x.x* **/webclidemo-***x.x.x.x* **/cdf/40webclidemo.cdf** file to suit your environment.
3. Modify the *installDir* **/vxworks-7/pkgs_v2/app/webcli-***x.x.x.x* **/webclidemo-***x.x.x.x* **/layer.vsbl** file to suit your environment.
4. Add user code and modify the *installDir* **/vxworks-7/pkgs_v2/app/webcli-***x.x.x.x* **/webclidemo-***x.x.x.x* **/src/Makefile** accordingly.
5. Add or modify Web server resource files in the **htdocs** folder.
6. Modify **config.webcli** to suit your environment. To change the configuration, you can use the MCE or edit the file manually.
7. Run the **preMake.bat** (Windows), or **preMake.sh** (Linux) script to generate the following files in the **webclidemo-***x.x.x.x* **/src** directory:

   > Note: The **preMake.bat** (Windows) and **preMake.sh** (Linux) are located in *installDir* **/vxworks-7/pkgs_v2/app/webcli-***x.x.x.x* **/tools-***x.x.x.x* **/bin/x86-win32** or **x86-linux**. The source files will be replaced with the new generated files. Hence, the original files should be backed-up.

   Web server files:

   - **wmb_webclidemo.h**
   - **wmw_httpconf.h**
   - **wmb_webclidemo.c**
   - **wmw_httpconf.c**
   - **wm_filesys.c**

   CLI tool files:

   - **wmb_webclidemo.h**
   - **wmc_webclidemo.h**
   - **rcc_handler.h**
   - **wmc_options.h**
   - **wmb_webclidemo.c**
   - **rcc_handler.c**
   - **wmc_webclidemo.c**

WNDRVR

8. Configure and build a VSB for your target based on the options specified in *VxWorks 7 Creating a CLI Tool and Web Server Tutorial:* Creating a VSB for the CLI Tool and Web Server.
9. Configure and build a VIP for your target based on the components specified in *VxWorks 7 Creating a CLI Tool and Web Server Tutorial:* Creating a VIP for the CLI Tool and Web Server.
10. Configure a ROMFS project and link it to the VIP based on the instructions in *VxWorks 7 Creating a CLI Tool and Web Server Tutorial:* Creating a ROMFS Project.
11. Load and run the image on the target.
12. Access the web pages from a web browser.

    For more information, see the VxWorks 7 Creating a Web Server Tutorial.

13. Access the CLI shell from a telnet session. For more information, see the VxWorks 7 Creating a CLI Tool Tutorial.

7. CREATING WINDMARKS AND WINDMARK HANDLERS

1. Introduction

WindMarks are name representations of device-resident objects and the handler code to access these objects. WindMarks can be created using the MCE.

Whether you plan to implement a command-line management interface, a Web-based management interface, or both, certain pieces of your management application are common to all of CLI, Web, and MIBway. Among these common pieces are the WindMarks and WindMark handlers. Your WindMarks and handlers together with the backplane target code integrate the CLI, Web, and SNMP (through MIBway) management interfaces.

Creating WindMarks and handlers is a substantial part of developing a Wind River CLI,Web, MIBway device management application. WindMarks represent manageable objects that device users can configure, control, and monitor using the CLI command set or through Web pages in a browser.

2. Creating a WindMark

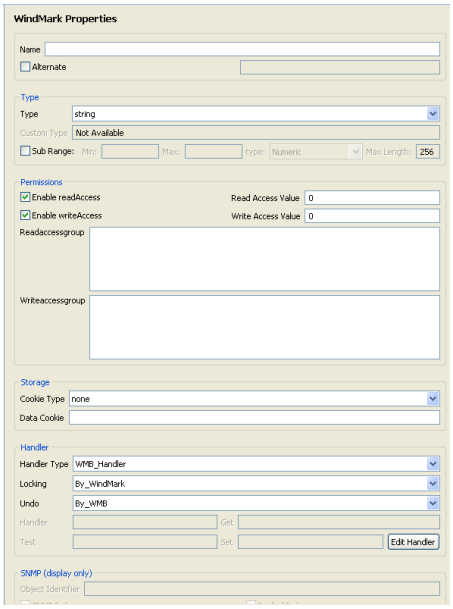
About This Task

Do the following to create a new WindMark:

Procedure

1. Right-click **Backplane** in the left pane of the MCE and select **New Child > WindMark**.

   The WindMark Properties dialog box appears in the right pane of the editor.



2. Type the name value of the WindMark.

This is a required value. The name can consist of the following characters: a-z, A-Z, 0-9, dash, underscore, and colon. Also, the first character of a WindMark name cannot be a number.

3. Type an alternate name value if desired.

   Using an alternate name causes the backplane to look up and use the alternate name in place of the regular name. This is useful for deprecated WindMarks—for example, RFC 1213 objects that have been superseded by RFC 2233 objects—where the physical element is now embodied in another WindMark.

4. Select the WindMark's data type from the **Type** drop-down menu.

   You can further restrict the selected WindMark data type by adding subranges. The available options change for each WindMark type.

   For example, if you select type string, then select Sub Range, you can select subranges such as Alpha, or Display 32-127 (which excludes control characters). You can also specify the maximum length of the string.

   Most selected subranges also allow you to edit minimum and maximum values as appropriate to that type.

5. Set the read and write permissions for the WindMark.

   For a more detailed discussion about available access control options, see Access Model Dialog Box Reference on page 132.

6. Set the storage options for the WindMark.

   For details on these options, see the discussion of storage options in WindMark Properties Dialog Reference on page 135.

7. Set the handler options.

   For details on these options, see the discussion of handler options WindMark Properties Dialog Reference on page 135.

   For information on editing a WindMark handler, see Editing a WindMark Handler on page 75.

8. Save your project.

   The SNMP section of the WindMark Properties dialog box is display only; the settings cannot be changed. For details on these options, see SNMP options WindMark Properties Dialog Reference on page 135.

   For information on creating SNMP WindMarks, see *Creating MIBway (SNMP) WindMarks*.

Next Steps

To edit a saved WindMark, select it in the left pane of the MCE. The WindMark Properties dialog box appears in the right pane of the editor.

Adding Enumerations to a WindMark on page 72 String types and numeric types (short, long, int64, and so forth) can have enumerated values.
Getting Windmarks as Enumerated Strings on page 72 The **CMD_GET_K** and **CMD_GET_NEXT_K** processes of the windmark handler get and return a numeric string by default.
Setting Windmarks as Enumerated Strings on page 73 The **CMD_COMMIT_K** process of the windmark handler sets and returns a numeric string value by default.
Adding Access Control to a WindMark on page 73 The Permissions section of the WindMark Properties dialog box allows you to add access control to a WindMark.
Creating WindMark Tables on page 74A WindMark table is a higher level object that contains WindMarks.

2.1. Adding Enumerations to a WindMark

String types and numeric types (short, long, int64, and so forth) can have enumerated values.

About This Task

For example:

- 1=yes
- 2=no
- 3=maybe
- 4=asknicely

For more information on enumerations, see Enumerations on page 19.

For each enumeration you would like to add to a WindMark, perform the following steps:

Procedure

1. Right-click the desired WindMark and select **New Child > Enumeration**.
2. In the Enumeration Properties pane, type a **Name** and a **Value** for your enumeration.
3. Save the file.

Parent topic: Creating a WindMark on page 70

Related information
Getting Windmarks as Enumerated Strings on page 72
Setting Windmarks as Enumerated Strings on page 73
Adding Access Control to a WindMark on page 73
Creating WindMark Tables on page 74

2.2. Getting Windmarks as Enumerated Strings

The **CMD_GET_K** and **CMD_GET_NEXT_K** processes of the windmark handler get and return a numeric string by default.

About This Task

However, to obtain the enumeration string rather than the numeric string (for example, enable instead of 1), do the following:

Procedure

Invoke wmbObjectValueImportAdapt( ) after calling wmbObjectLoad( ) or wmbObjectImport( ).

For example:

```
...
case CMD_GET_K:
    status = wmbObjectLoad(obj);
    if (status == OK)
        status = wmbObjectValueImportAdapt(obj,DATATYPE_ENUM_K);
```

```
    break;
...
```

Parent topic: Creating a WindMark on page 70

Related information
Adding Enumerations to a WindMark on page 72
Setting Windmarks as Enumerated Strings on page 73
Adding Access Control to a WindMark on page 73
Creating WindMark Tables on page 74

2.3. Setting Windmarks as Enumerated Strings

The **CMD_COMMIT_K** process of the windmark handler sets and returns a numeric string value by default.

About This Task

However, if the set value is an enumeration string rather than a numeric string value (for example, enable instead of 1), do the following:

Procedure

Invoke wmbObjectValueExportAdapt( ) to convert the enumeration string to a numeric string before calling wmbObjectStore( ) or wmbObjectExport( ).

For example:

```
...
case CMD_COMMIT_K:
    status = wmbObjectValueExportAdapt(obj,DATATYPE_ENUM_K);
    if (status == OK)
        status = wmbObjectStore(obj);
    break;
...
```

Parent topic: Creating a WindMark on page 70

Related information
Adding Enumerations to a WindMark on page 72
Getting Windmarks as Enumerated Strings on page 72
Adding Access Control to a WindMark on page 73
Creating WindMark Tables on page 74

2.4. Adding Access Control to a WindMark

The Permissions section of the WindMark Properties dialog box allows you to add access control to a WindMark.

About This Task

You can add read and write access to a WindMark and set the values for each. You can also add read and write access groups.

To create an access group, do the following:

Procedure

1. In the left pane of the MCE, right-click **Backplane** and select **New Child > Access**.

   A new child **Access** object appears beneath **Backplane**.

2. Select the desired options for the access model. You have a choice of level, mask, or a combination of both.

   For more information, see Access Model Dialog Box Reference on page 132.

3. Right-click **Access** and select **New Child > Accessgroup**.
4. Type a name for your access group.

Results

You can now select the read or write access group from the WindMark Properties dialog box.

Parent topic: Creating a WindMark on page 70

Related information

Adding Enumerations to a WindMark on page 72
Getting Windmarks as Enumerated Strings on page 72
Setting Windmarks as Enumerated Strings on page 73
Creating WindMark Tables on page 74

2.5. Creating WindMark Tables

A WindMark table is a higher level object that contains WindMarks.

About This Task

Such objects include the following:

- C data structures
- groups, which are collections of WindMarks that share a relationship, such as a dependency or a common handler.
  For more information about WindMark tables, see WindMark Tables on page 23.

Note: A WindMark cannot be in more than one table.

For more details on the options available, see WindMark Table Dialog Reference on page 133.

To create a table, do the following:

Procedure

1. In the left pane of the MCE, right-click **Backplane** and select **New Child > Table**.
2. In the right pane of the MCE type a name for the table and select the desired options.

Parent topic: Creating a WindMark on page 70

Related information

Adding Enumerations to a WindMark on page 72

Getting Windmarks as Enumerated Strings on page 72
Setting Windmarks as Enumerated Strings on page 73
Adding Access Control to a WindMark on page 73

3. Editing a WindMark Handler

You can edit WindMark handlers from the MCS at any time.

About This Task

To edit a WindMark handler, do the following:

Procedure

1. Select the desired WindMark from the left pane of the MCE.
2. In the Handler section, you can set the following:
   - handler type options
   - locking options
   - undo options
3. Click **Edit Handler** to add code that is preserved between code regenerations.

   The generated file **wmb_projName.c** opens in a text editor.

   Note: Before you can edit the code in **wmb_projName.c**, you must build your project files.

4. To build your project files, right-click your project and select **Build Project**.

4. Creating a Custom WindMark Handler

You can create a custom WindMark handler and how to use a custom handler to access static data.

About This Task

To create a custom WindMark handler, do the following:

Procedure

1. From the left pane of the MCE, select the desired WindMark.

   The WindMark Properties dialog box appears.

2. Click **(Re)Configure Handler**.

   The Configure New Handler dialog box appears.

3. From the **Handler Type** drop-down menu, select **External API**.

4. In the **Handler** field, type the name of your custom handler function.
5. Save the file.

5. Accessing Static Data Using a Custom WindMark Handler

The default WMB handler can be used to access and modify variables on the running system by supplying the address of the data variable as a data cookie (in the Storage section of the WindMark Properties dialog box in the MCE) and setting the cookie type to Absolute Address.

The address is supplied using the "address of" operator "&" and the variable name. However, this method does not work if the data is private to a separate module, because, in such a case, the variable is out of scope with respect to the handler.

Inside your custom handler, you must use the WindMark handler functions wmbObjectImport( ) and wmbObjectExport( ) instead of wmbObjectLoad( ) and wmbObjectStore( ), which are provided by default. wmbObjectLoad( ) and wmbObjectStore( ) are wrappers of the import/export functions used only for global data where the memory address is provided. wmbObjectImport( ) and wmbObjectExport( ) provide the ability for arbitrary data to be handled as a **WMB_OBJ_T**, thus providing the interface between your data and the handlers.

A sample handler called **testPrivateHandler** follows:

```
/* test data to be accessed */
static sbyte mPrivateVar[100] = "privateString";


/* Prototypes */

/* %%BeginUserCode Tutorial07_40_41_Prototypes */

/* prototype of windmark handler to access test data */

STATUS testPrivateHandler (WMB_TID_T tid, WMB_OBJ_T obj, COMMAND_T command);

/* %%EndUserCode Tutorial07_40_41_Prototypes */


/* %%BeginUserCode Tutorial07_40_41_UserFunctions */
/* body of windmark handler */

STATUS testPrivateHandler
    (
    WMB_TID_T tid,        /* wmb transaction pointer */
    WMB_OBJ_T obj,        /* wmb object pointer */
    COMMAND_T command     /* transaction type (e.g. GET,SET) */
    )
    {
    STATUS status = OK;
    sbyte4 written;

    /* loop through all objects in the group linked list */
    while (wmbObjectGood(obj))
        {
        switch (command)
            {
            case CMD_GET_K:
                status = wmbObjectImport(obj, mPrivateVar, 100,
```

```
                                        DATATYPE_STRING_K);
                break;

            case CMD_GET_NEXT_K:
                break;

            case CMD_TEST_K:
                /* backup data */
                status = wmbObjectRedo(obj);
                /* add validation test here */
                break;

            case CMD_COMMIT_K:
                status = wmbObjectExport(obj, mPrivateVar, 100,
                                        DATATYPE_STRING_K, &written);
                break;

            case CMD_UNDO_K:
                status = wmbObjectUndo(obj);
                /*
                status = wmbObjectSavedGet(obj, wmbObjectCookieGet(obj),
                                          256, DATATYPE_STRING_K);
                */
                break;

            case CMD_COMPLETE_K:
                wmbObjectSavedFree(obj);
                break;

            default:
#ifdef _DEBUG
                printf("unknown command: %d\n", command);
#endif
                return ERROR;
            }

        obj = wmbObjectNextGet(obj);
        }
    return OK;
    }
/* %%EndUserCode Tutorial07_40_41_UserFunctions */
```

8. CREATING MIBWAY (SNMP) WINDMARKS

1. MIBway SNMP WindMarks Overview

SNMP WindMarks allow your CLI- and Web-based management interfaces to access SNMP objects.

You can automatically generate WindMarks that correspond to MIB objects when you import a MIB file into your project. The MCE and the WMIT use the MIB compiler to accomplish this. When importing a MIB file to create MIBway WindMarks, you do not have the wide range of input and output options available to you when using the MIB compiler directly. This particular method of access to the MIB compiler is specifically for creating WindMarks. If you do not intend to create WindMarks from the MIB objects, you can launch the MIB compiler from the WMIT (see Using the MIB Compiler on page 185).

For a list of supported SNMP data types, see SNMP Data Types on page 20.

Note: If you enable SNMP in your project, you must build the SNMP user libraries as part of your VSB project. For more information on building your VSB, see user's guide for your Platform. For more information on SNMP VSB components, see the *VxWorks 7 SNMP Programmer's Guide*.

2. Creating MIBway WindMarks

About This Task

To generate WindMarks from MIB objects, perform the following steps:

Procedure

1. Right-click **Web CLI Configuration** and select **New Child > SNMP**.

   A new child SNMP object appears beneath **Web CLI Configuration**.

2. In the left pane of the MCE, right-click **SNMP** and select **New Child > MIB Files**.
3. In the left pane of the MCE, right-click **MIB Files** and select **New Child > File**.
4. In the MIB File Properties dialog box, click **Select File** and navigate to the MIB file you would like to import objects from.
5. In the left pane of the MCE, select **SNMP**.
6. In the SNMP Properties dialog box, click **Import Objects**.

   The MIB objects from the MIB files are imported as WindMarks and appear beneath **Backplane** in the left pane of the MCE.

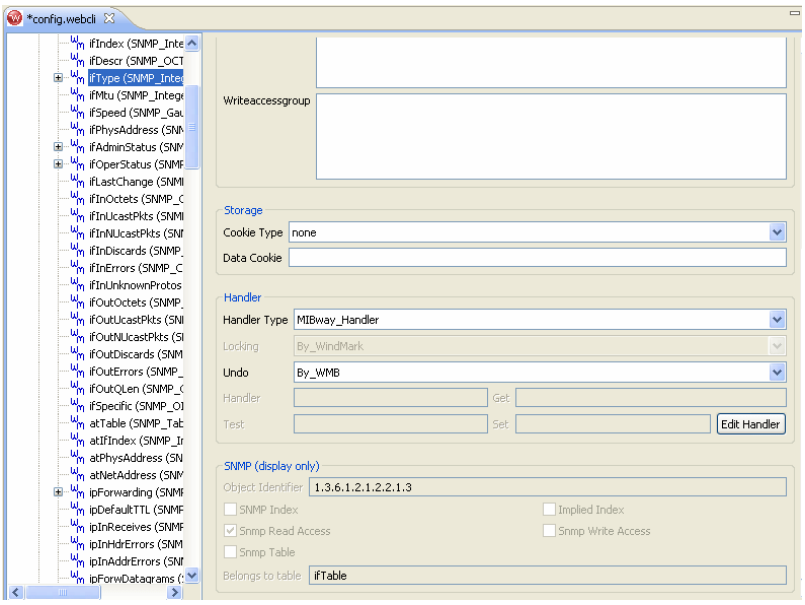
3. Viewing Properties of SNMP WindMarks

About This Task

Procedure

To view the properties of an SNMP WindMark, select the desired WindMark from the left pane of the MCE.

The SNMP properties appear in the SNMP section of the WindMark Properties dialog box.



The **SNMP** section of the WindMark Properties dialog box is display-only; the settings cannot be changed.

4. Viewing MIB Objects in the WMIT

The WMIT has different ways for viewing MIB objects and MIB files once they have been imported into your project.

You can view ASN.1 files, MIB trees, MIB leafs, and SNMP tables.

Viewing ASN.1 Files on page 79 When you use the MIB File List dialog box to compile an ASN.1 file, you are able to view that ASN.1 file using WMIT.
Viewing MIB Trees on page 80
Viewing MIB Leaf Properties on page 80
Viewing SNMP Tables on page 81

4.1. Viewing ASN.1 Files

When you use the MIB File List dialog box to compile an ASN.1 file, you are able to view that ASN.1 file using WMIT.

Procedure

To view the ASN.1 file for a compiled MIB, double-click the ASN.1 file name in the **SNMP** tab of the project window.

The viewer is read only. You cannot use it to edit ASN.1 files.

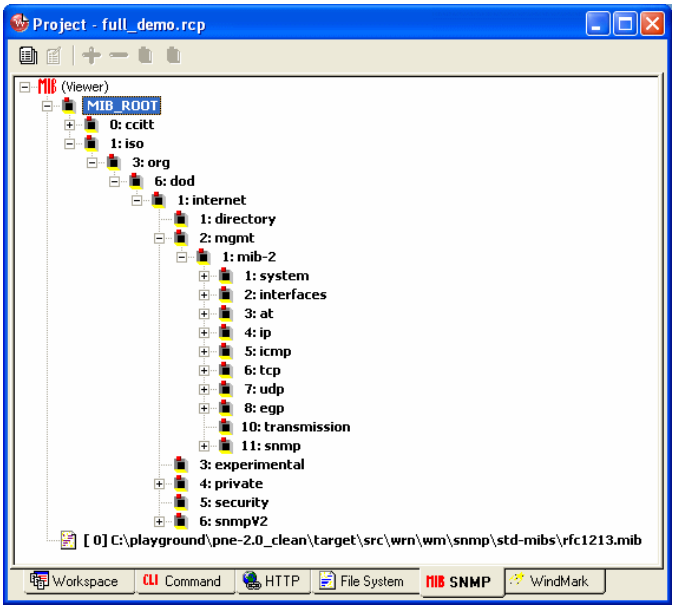
Parent topic: Viewing MIB Objects in the WMIT on page 79

4.2. Viewing MIB Trees

Procedure

To view items under a management node, double-click the node to open the branch.

One exception is double-clicking **MIB_ROOT**, which opens all the way to the **Internet** branch, the most commonly used branch. Double-clicking a leaf node allows you to view the information about a MIB leaf.



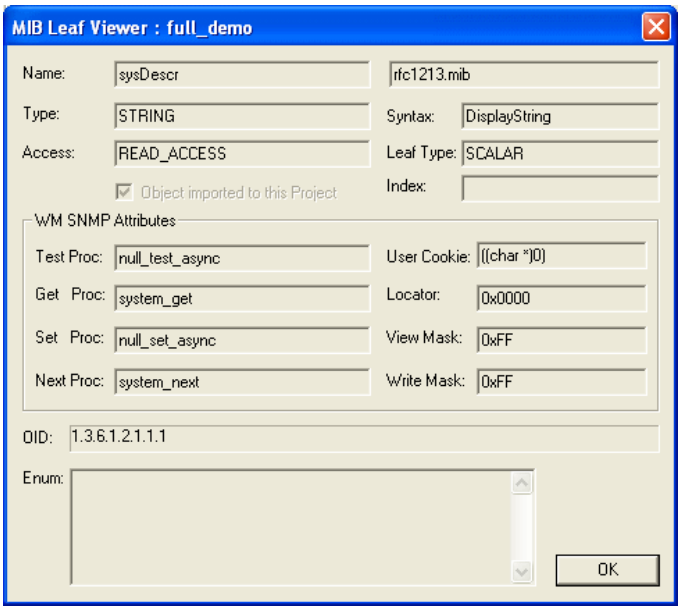
Parent topic: Viewing MIB Objects in the WMIT on page 79

4.3. Viewing MIB Leaf Properties

Procedure

To view MIB objects in the WMIT, double-click on the leaf node.

The MIB Leaf Viewer appears, containing information about the MIB leaf object.



Parent topic: Viewing MIB Objects in the WMIT on page 79

4.4. Viewing SNMP Tables

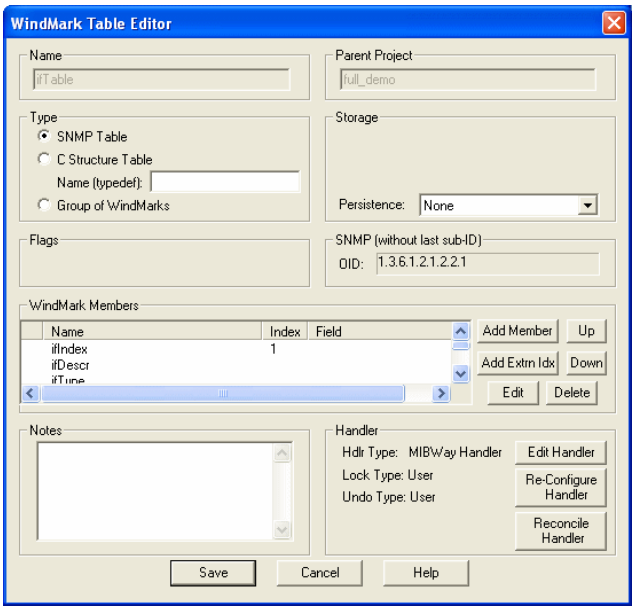
About This Task

You can view SNMP tables in the WindMark Table Editor. They are identified as SNMP tables as opposed to a C Structure or a group of WindMarks.

Procedure

1. Select **Project > Edit WindGroup**.
2. Select the desired group.
3. Click **Edit**.

   The WindMark Table Editor appears.



Parent topic: Viewing MIB Objects in the WMIT on page 79

9. CREATING A CLI MANAGEMENT COMMAND SET

1. Introduction

You use the MCE to create and edit nodes in the command tree.

The commands are displayed in a command tree in the left pane of the MCE. The command tree is a visual representation of the command hierarchy and relationships. Each command is shown as a node on the tree. The tree displays command parameters, handler functions, and WindMarks associated with each command node. The visual command tree lets you see the tree your users will have to navigate, and the tree structure prevents inconsistencies that might otherwise occur.

The MCE visually decouples the command names from handlers so you can design the interface separately from the implementation of the handlers for each command. This allows for quick prototyping where you can create a tree of command nodes, then you can edit each node, in turn, to add parameters and handler functions (for specifying unique functions).

2. Before you Begin

Careful consideration must be given to the design and structure of your CLI tree.

There is more than one right way to design your command-line interface. You should be familiar with the following design and implementation issues before creating your CLI tree:

- using multiple commands vs. multiple handlers
- reserved keywords
- creating a node vs. a parameter
- using multiple parameters of type string
- assigning multiple handlers to a node
- access control
- command no forms
- intermediate mode commands
- automatic naming

Using Multiple Commands vs. Multiple Handlers on page 84 Given a set of commands, your goal should be to execute the most appropriate handler.
Reserved Keywords on page 84 There are reserved keywords and characters that you should not use when creating your command.
Creating a Node vs. Parameter on page 85
Using Multiple Parameters of Type String on page 86 Having a node that takes three parameters, none of which have keywords, and all of which are marked optional can cause problems in your commands.
Assigning Multiple Handlers to a Node on page 87 You can attach multiple handlers to a given command node.
Implementing Access Control on page 88
Understanding Command No Forms on page 88A no form of a command is usually used to negate a specific command.
Understanding Intermediate Mode Commands on page 89 Intermediate mode support allows the user to enter partial commands.
Automatic Naming on page 90 During code generation, automatic internal ANSI standard C compatible names are generated for commands, handlers, parameters, and enumerated types.
Using Meta Commands on page 91 Meta commands are commands that act on other commands.
Interrupting a Running Command on page 91 You can interrupt the CLI command execution

WNDRVR

2.1. Using Multiple Commands vs. Multiple Handlers

Given a set of commands, your goal should be to execute the most appropriate handler.

For a node with more than one handler, the CLI engine executes the command that has all of its required parameters met, plus the greatest number of optional parameters met. If you give a command without all of its required parameters, the CLI engine generates an error.

Consider a node with two optional parameters and four handlers—that means that there are 16 possible permutations.

If a user enters a command comprising multiple nodes on a single line, if the nodes have handlers, the node handlers get executed (in node order). The first/best handler is executed for each node. CLI does not execute multiple handlers for one command.

Parent topic: Before you Begin on page 83

Related information

Reserved Keywords on page 84
Creating a Node vs. Parameter on page 85
Using Multiple Parameters of Type String on page 86
Assigning Multiple Handlers to a Node on page 87
Implementing Access Control on page 88
Understanding Command No Forms on page 88
Understanding Intermediate Mode Commands on page 89
Automatic Naming on page 90
Using Meta Commands on page 91
Interrupting a Running Command on page 91

2.2. Reserved Keywords

There are reserved keywords and characters that you should not use when creating your command.

Reserved Character/Keyword	Comment
!	Executes the recent command in the history buffer by index.
?	Shows help for command or parameter.
no	Used to execute No Form type handler for command.
$	Used as a prefix of a session variable.
#	If the scripting-support flag is enabled, # is regarded as the comment keyword, otherwise # is regarded as a normal character.
%	The item following is treated as a WindMark.

There are many constants that you can modify during development (see Sample Global Commands on page 170). Thus, you can use these keywords if you change the system keywords to something else.

If possible, also avoid using direct ANSI calls (escape characters using hex code) unless you have tested the behavior and are certain that nothing will break. Wind River recommends using alphanumeric characters only.

Parent topic: Before you Begin on page 83

Related information

Using Multiple Commands vs. Multiple Handlers on page 84
Creating a Node vs. Parameter on page 85
Using Multiple Parameters of Type String on page 86
Assigning Multiple Handlers to a Node on page 87
Implementing Access Control on page 88
Understanding Command No Forms on page 88
Understanding Intermediate Mode Commands on page 89
Automatic Naming on page 90
Using Meta Commands on page 91
Interrupting a Running Command on page 91

2.3. Creating a Node vs. Parameter

You need to make a decision as to whether a given node will actually be a parameter, or vice versa. This really boils down to user perception. Consider the following:

```
cli%>show ip address
```

From the above, it is not clear to the end user whether address is a node or a parameter. You could design the above command in two ways: with address as a node, or address as a parameter.

If you treat address as a node, *address* is a node with zero parameters that has a handler function, or multiple functions, associated with it.

If you treat address as a parameter, *ip* is the node and *address* is one of the passed-in parameter values. The CLI parsing engine determines that address is a value that is either a WindMark entry in the backplane, or a customer-specific entry in some proprietary code space.

Pros and Cons

The benefit of treating address as a parameter is that the ip node could take multiple and optional parameters, of which address was only one member. The full syntax for the command could thus be:

```
cli%>show ip address netmask gateway routes
```

However, the disadvantage of treating address as a parameter is that it cannot be treated like a node. One reason why you need to choose between nodes and parameters is whether or not you want a loose command or a strict command. Take the following command (at face value):

```
cli%>ifconfig eth0 address 127.0.0.1 netmask 255.255.255.0 broadcast 255.255.255.255
```

This command can be implemented in two ways. The first way, the loose way, allows you to type in a partial command list; for example:

```
cli%>ifconfig eth0 address 127.0.0.1
value changed
cli%(ifconfig)>
```

In the above example, ifconfig is also an intermediate mode so that you could continue entering ifconfig-specific values. This approach has some benefits, one of which is that you can rearrange the order of parameters; for example:

```
cli%>ifconfig eth0 netmask 255.255.255.0 address 127.0.0.1
```

The second way, the strict way, is to use multiple nodes to validate the code, and thus call the right handler. Do not use intermediate nodes for this type of solution, as it would confuse your users. What is really happening is as follows:

- You define a node called **eth0**, under which you define another called address, and then netmask, and finally broadcast, thus creating a set of nodes four deep.
- Each node has one parameter that does not take a keyword (implied parameter) and has the correct parameter type (IP address, integer, and others).
- Only the deepest node has a handler associated with it, in this case broadcast. This handler takes each parent parameter from the nodes above broadcast— that is, (using dot notation), the broadcast node takes eth0.char, address.ip, netmask.ip, and finally broadcast.ip.

Thus, the handler is only called if you complete the command all the way down to the last value. Otherwise you get an unrecognized command error.

Parent topic: Before you Begin on page 83

Related information

Using Multiple Commands vs. Multiple Handlers on page 84
Reserved Keywords on page 84
Using Multiple Parameters of Type String on page 86
Assigning Multiple Handlers to a Node on page 87
Implementing Access Control on page 88
Understanding Command No Forms on page 88
Understanding Intermediate Mode Commands on page 89
Automatic Naming on page 90
Using Meta Commands on page 91
Interrupting a Running Command on page 91

2.4. Using Multiple Parameters of Type String

Having a node that takes three parameters, none of which have keywords, and all of which are marked optional can cause problems in your commands.

The reason this causes a problem is that the string type parameter is not type-checked. It is not validated and is the only type for which you can enter any value. The problem arises when the user attempts to rearrange the order of parameters entered. The parsing engine has no way of knowing which values are for which parameters. To avoid this, develop such command nodes using either a) stated keywords for each parameter, or b) a different type for each parameter.

The reason that b) works above, is that the CLI parsing engine is capable of differentiating between different parameter types and can actually use that information to determine the proper ordering of parameters. So a command such as the following:

```
cli%>command type-ip type-char type-absolute
```

works just as well as the same command typed as follows:

```
cli%>command type-absolute type-ip type-char
```

Parent topic: Before you Begin on page 83

Related information

Using Multiple Commands vs. Multiple Handlers on page 84
Reserved Keywords on page 84
Creating a Node vs. Parameter on page 85
Assigning Multiple Handlers to a Node on page 87
Implementing Access Control on page 88
Understanding Command No Forms on page 88
Understanding Intermediate Mode Commands on page 89

Automatic Naming on page 90
Using Meta Commands on page 91
Interrupting a Running Command on page 91

2.5. Assigning Multiple Handlers to a Node

You can attach multiple handlers to a given command node.

You must be aware of the following issues:

- The CLI parsing engine expects all parameters to have values, unless they are of type absolute (see the entry for *absolute* in the *Glossary*.)

  If the end user omits a value for any other (non-absolute) parameter, and if that parameter is marked required, they get an "expected mandatory value" error.

  If the end user omits a value for any other (non-absolute) parameter, and if the parameter is marked optional, the handler will probably not get called at all.

  If you want boolean style commands, they should be parameters of type absolute, with associated parameters and multiple handlers. absolute is a built-in parameter type that has no value associated with it—either it is present or it is not. In the example below, up is a parameter of type absolute that tells what to do to the parameter keyword address:

  ```
  set ip address x.x.x.x up
  ```

  where

  - set = node
  - ip = node
  - address = param #1 keyword
  - x.x.x.x = param #1 value
  - up = param #2 (absolute value)

  Thus, you can have a command node that takes multiple (and possibly optional) parameters that each trigger a different handler.
- You must add each parameter to the optional list in the Handler Function editor in the MCE one by one. That is, add one parameter to the optional list, and then close the window, create your next handler for this node, add the second parameter to the optional list, then close the window, add your third handler, and so on. The finished output will appear as follows:

  ```
  cli%>command param1
  first handler has been called, output
  cli%>command param2
  second handler has been called, output
  cli%>command param3
  third handler has been called, output
  cli%>
  ```

  If you list all the parameters at the same time, as shown in the following example:

  ```
  cli%>command param1 param2 param3
  ```

  having marked all parameters as optional, you trigger only the first handler. This is because when presented with multiple handlers for a given node, the parsing engine only attempts to find the first handler that matches the criteria, at which point the parsing stops. This situation is also true for command handlers that reference parent parameters from a point higher up on the tree. The parsing engine discards the second and third optional parameters and does not call the second and third handlers.

Parent topic: Before you Begin on page 83

Related information
Using Multiple Commands vs. Multiple Handlers on page 84

WNDRVR

Reserved Keywords on page 84
Creating a Node vs. Parameter on page 85
Using Multiple Parameters of Type String on page 86
Implementing Access Control on page 88
Understanding Command No Forms on page 88
Understanding Intermediate Mode Commands on page 89
Automatic Naming on page 90
Using Meta Commands on page 91
Interrupting a Running Command on page 91

2.6. Implementing Access Control

Access control is the process of allowing or denying a user permission to access a command or WindMark by comparing the user's permission level to the level required to access the command or WindMark. This is different from authentication, which is the process of verifying the authenticity of a user at login.

WindMarks, commands, and parameters can each have their own access control restrictions. This allows the possibility of a command with different parameters and different behavior for different users. Access is allowed or denied based on access level. In order to have access, the user must have an access level greater or equal to that of the resource being requested.

Command-level access control

Using command-level access control, you can restrict access to individual commands. An example is a command that shows all users currently logged onto the device. This might be a useful command for a system administrator, but probably not desirable for all users.

Parameter-level access control

Using parameter-level access control, you can create commands that have different options for different users. If a user does not have the appropriate access level to access a particular parameter, that parameter is not available to that user.

WindMark-level access control

Using WindMark-level access control, you can restrict get or set access to WindMark objects or groups. You might want to allow certain users read/write privileges while allowing other users read-only privileges.

Parent topic: Before you Begin on page 83

Related information

Using Multiple Commands vs. Multiple Handlers on page 84
Reserved Keywords on page 84
Creating a Node vs. Parameter on page 85
Using Multiple Parameters of Type String on page 86
Assigning Multiple Handlers to a Node on page 87
Understanding Command No Forms on page 88
Understanding Intermediate Mode Commands on page 89
Automatic Naming on page 90
Using Meta Commands on page 91
Interrupting a Running Command on page 91

2.7. Understanding Command No Forms

A no form of a command is usually used to negate a specific command.

It is up to the programmer of the device to maintain the negation; Wind River CLI sets up the no form as another handler.

Using the no form of a command overrides all other commands or handlers associated with that node, regardless of the parser and regardless of whether or not the parameters were matched.

A node with one handler supports no forms automatically. If a node has two or more handlers, you must specify which (if any) is the no form.

Parent topic: Before you Begin on page 83

Related information

Using Multiple Commands vs. Multiple Handlers on page 84
Reserved Keywords on page 84
Creating a Node vs. Parameter on page 85
Using Multiple Parameters of Type String on page 86
Assigning Multiple Handlers to a Node on page 87
Implementing Access Control on page 88
Understanding Intermediate Mode Commands on page 89
Automatic Naming on page 90
Using Meta Commands on page 91
Interrupting a Running Command on page 91

2.8. Understanding Intermediate Mode Commands

Intermediate mode support allows the user to enter partial commands.

For example, when you enter

```
config router ip bgp 10
```

this places the CLI in an intermediate state whereupon the prompt changes to **router–config:#** and successive commands are effectively preceded with

```
config router ip bgp 10
```

This state lasts until the user issues an exit command.

Note:

You must now explicitly type commit before exiting an intermediate mode in order for changes to take effect if the handler is setting SNMP values with MIBway.

For example, the following session would be correct usage:

```
# set ip
(set)#address 1.2.3.4
(set)#netmask 255.0.0.0
(set)#commit
(set)#exit
```

If you exit intermediate mode without issuing the commit command, the SNMP SET operations do not occur.

If you do not have a handler associated with an intermediate node, then there is no error checking. Nodes and parameters do not in themselves have an association— only the existence of a handler (even a dummy handler) creates this association.

Parent topic: Before you Begin on page 83

Related information

Using Multiple Commands vs. Multiple Handlers on page 84
Reserved Keywords on page 84
Creating a Node vs. Parameter on page 85
Using Multiple Parameters of Type String on page 86
Assigning Multiple Handlers to a Node on page 87
Implementing Access Control on page 88
Understanding Command No Forms on page 88

WNDRVR

Automatic Naming on page 90
Using Meta Commands on page 91
Interrupting a Running Command on page 91

2.9. Automatic Naming

During code generation, automatic internal ANSI standard C compatible names are generated for commands, handlers, parameters, and enumerated types.

Commands, handlers, and parameters are given names that include parent nodes separated by underscores. The only non-ANSI standard C compatible character allowed is the dash (or minus sign), which is converted to an underscore.

Enumerated and list members are allowed any standard typewriter character except spaces (and the selected delimiter character in the case of a list). The following substitution table is used in building the ANSI standard C compatible name:

Table 1. Automatic Naming Character Set

Character	Replacement	Character	Replacement
+	Qpl	&	Qam
-	Qml	(	Qlp
/	Qfs	)	Qrp
=	Qeq	{	Qlc
|	Qpp	}	Qrc
<	Qlt	[	Qlb
>	Qgt	]	Qrb
?	Qqm	;	Qsc
!	Qex	:	Qcl
@	Qat	~	Qtd
#	Qpd	\"	Qdq
$	Qdl	'	Qsq
%	Qpc	\\	Qbs
^	Qct	-	Qmi

For example, if you name a node *port#*, it appears in the generated C code as *portQpd*.

For more information on Wind River SSH, see the VxWorks 7 SSH Programmer's Guide.

Parent topic: Before you Begin on page 83

Related information
Using Multiple Commands vs. Multiple Handlers on page 84
Reserved Keywords on page 84
Creating a Node vs. Parameter on page 85
Using Multiple Parameters of Type String on page 86
Assigning Multiple Handlers to a Node on page 87
Implementing Access Control on page 88
Understanding Command No Forms on page 88
Understanding Intermediate Mode Commands on page 89
Using Meta Commands on page 91
Interrupting a Running Command on page 91

2.10. Using Meta Commands

Meta commands are commands that act on other commands.

What differentiates a meta command from a normal command with optional parameters is that the commands following the meta command are parsed as normal commands, but the action is passed to the meta command rather than the CLI execution unit. In this manner, the meta command can take advantage of all of the features of the CLI parser (like command syntax validation, command completion, and aliasing), and yet control the resulting action.

The sample global command help is an example of a meta command. It is located in the file:

installDir **/vxworks-7/pkgs_v2/app/webcli-***x.x.x.x***/cli-***x.x.x.x***/src/examples/commands.rcp**

For example, given the following command:

```
help show table xyz
```

The command show table xyz is first parsed to make sure it is a valid command. However, instead of executing, the command is then passed to the meta command, and help is given on the command.

Meta commands work on global commands the same way they work on normal commands, thus providing a higher sense of a global operator. For more information on sample global commands, see Sample Global Commands on page 170.

Parent topic: Before you Begin on page 83

Related information
Using Multiple Commands vs. Multiple Handlers on page 84
Reserved Keywords on page 84
Creating a Node vs. Parameter on page 85
Using Multiple Parameters of Type String on page 86
Assigning Multiple Handlers to a Node on page 87
Implementing Access Control on page 88
Understanding Command No Forms on page 88
Understanding Intermediate Mode Commands on page 89
Automatic Naming on page 90
Interrupting a Running Command on page 91

2.11. Interrupting a Running Command

You can interrupt the CLI command execution

Before You Begin

You must first enable the WEBCLI_RCC_INTERRUPT_EXEC VSB option as described in Source Build Configuration Options (VSB) on page 38.

Procedure

Enter either **CTRL+X** or **CTRL+C**.

Parent topic: Before you Begin on page 83

Related information

Using Multiple Commands vs. Multiple Handlers on page 84
Reserved Keywords on page 84
Creating a Node vs. Parameter on page 85
Using Multiple Parameters of Type String on page 86
Assigning Multiple Handlers to a Node on page 87
Implementing Access Control on page 88
Understanding Command No Forms on page 88
Understanding Intermediate Mode Commands on page 89
Automatic Naming on page 90
Using Meta Commands on page 91

3. Creating and Removing Command Nodes

Learn how to create a command node and edit its basic properties.

About This Task

Before you start specifying commands, give some thought to the design and structure of your command tree. For more information on design considerations, see CLI Design Considerations on page 83.

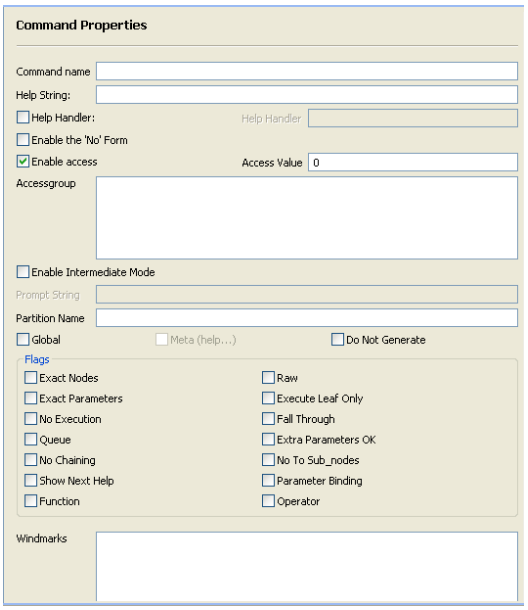
Subcommands can be created using the same procedure. When creating a subcommand, be sure to right-click the appropriate parent command instead of CLI in step 1.

Procedure

1. In the left pane of the MCE, right-click **CLI** and select **New Child > Command**.

   The Command Properties dialog box appears in the right pane of the editor.

WNDRVR



For details on the options in the Command Properties dialog box, see Command Properties Reference on page 146.

2. In the **Command Name** field, type a name for your command.

> Note: When naming your commands, keep in mind that command names cannot include spaces, and there are reserved keywords and characters that you should not use. For more information, see Reserved Keywords on page 84.

3. Set the basic properties, flags, and WindMarks associated with the command.

   For details on each of the options available, see Command Properties Reference on page 146.

4. Save **config.webcli**.
5. Optionally, to remove a command, right-click the desired command and select **Delete**.

4. Creating and Editing Parameters

Command object parameter names must be unique on the branch that they are on.

About This Task

For example, the following construction is not valid:

```
+ - - commandA
          | - - parameter1
         + - - commandB
                  | - - parameter1
```

However, identical parameter names may be used across different branches. The following construction is valid:

```
+ - - commandA
 |          | - - parameter1
 |        + - - commandB
 |                  | - - parameter2
+ - - commandC
           | - - parameter1
         + - - commandD
                  | - - parameter2
```

The invalid state of a duplicate parameter on the same command branch does not result in a project error. However, this state leads to a condition where the incorrect parameter may be used by the build process.

To create a new parameter, use the following procedure:

Procedure

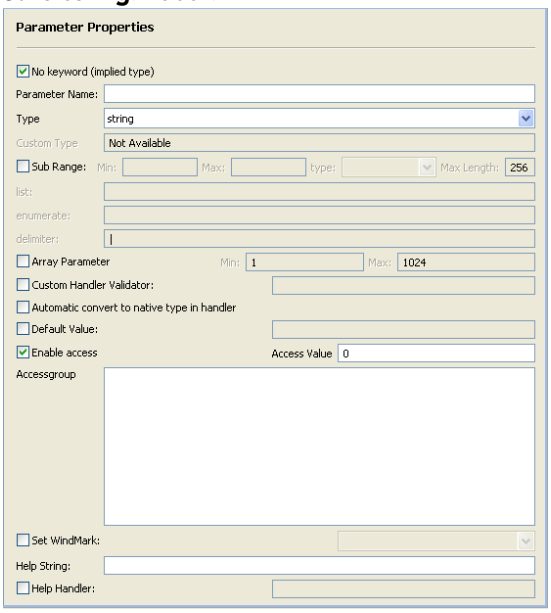
1. In the left pane of the MCE, right-click the desired command and select **New Child Parameter**.

   The Parameter Properties dialog box appears in the right pane of the editor.

2. In the **Parameter** field, type a name for your parameter.
3. Set the properties for the parameter.

   For details on the options in the Parameter Properties dialog box, see Parameter Properties Dialog Reference on page 149.

4. Save **config.webcli**.



5. Optionally, to edit an existing parameter, select it from the left pane of the MCE.

   The Parameter Properties dialog box appears in the right pane of the editor.

5. Creating and Editing Command Handlers

About This Task

Use the MCE to create command handlers and configure the features that can be associated with them, including the list of available parameters. You can specify which parameters are required, which are optional, whether the function is designed to handle no forms of the command, and so forth.

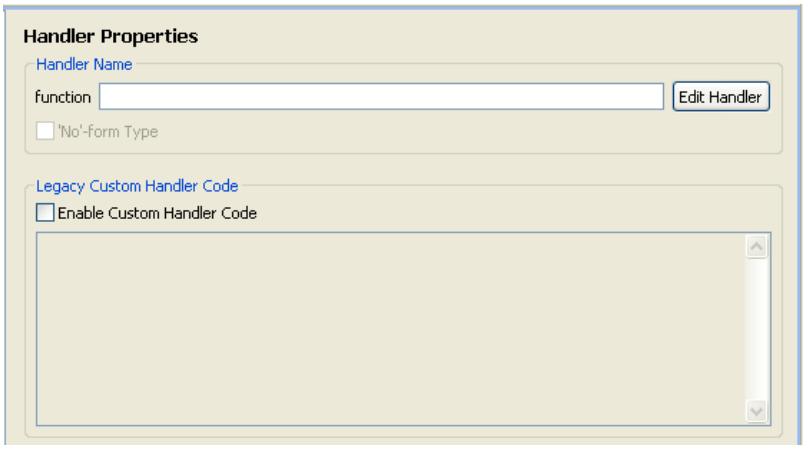
You can attach multiple handlers to a given command. For more information, see Assigning Multiple Handlers to a Node on page 87.

To create a command handler, use the following procedure:

Procedure

1. In the left pane of the MCE, right-click the desired command and select **New Child > Handler**.

   The Handler Properties dialog box appears in the right pane of the editor.



2. Set the properties for the handler.

   For details on the options in the Handler Properties dialog box, see Handler Properties Dialog Reference on page 151.

3. Save **config.webcli**.

Results

After you build your project files, you can use the MCE to edit the generated skeleton handler functions for your command nodes.

6. Creating Handler Parameters

About This Task

To create a parameter for your handler, do the following:

Procedure

1. Right-click the handler you just created, and select **New Child > Handler Parameter**.

WNDRVR

The Handler Parameter Properties dialog box appears in the right pane of the editor.

2. Select the desired parameter from the list of available parameters in the **Handler Parameter** drop-down list.
3. Select **Required** to add the parameter as a required handler parameter.
4. Save **config.webcli**.

7. Editing Handlers

Before you can edit your command handlers, you must build your project files.

About This Task

To build your project files, right-click your MCE project and select Build Project.

To edit a handler, do the following:

Procedure

1. In the left pane of the MCE, select the desired handler.

   The Handler Properties dialog box appears in the right pane of the editor.

2. Click **Edit Handler**.

   The generated file **wmc_projName.c** opens in a text editor.

3. Add your code between the beginning and end of the following code:

```
/* %%BeginUserCode myNewHandler_HandlerBody */
     /* TO DO: Add your handler code here */
/* %%EndUserCode myNewHandler_HandlerBody */
```

8. Creating a Help Handler Routine

Instead of a help string, you can use a help handler to provide help if you want to display different help strings under different conditions.

About This Task

If you use a help handler, you must provide your own help handler function.

```
extern void HelpHandler(cli_env *pCliEnv, cmdNode *pNode, paramDefn *pParam);
```

In this example, the help handler looks for the chassis number of the device. If there is no chassis number, it displays:

```
No Supervision allowed
```


WNDRVR

If the chassis number is less than 30, it displays the help string:

```
Supervision not available on this chassis
```

Finally, if the chassis number is greater than 29, it displays:

```
Supervise traffic on this chassis
```

Procedure

To create this help handler, implement the following code:

```
extern void HELP_SetEnableSupervise
        (
         struct cli_env *pCliEnv,
         struct cmdNode *pNode,
         struct paramDefn *pParam
        )
        {
        sbyte4 level = 0;

        if (OK != RCC_DB_ParamValue(pCliEnv, mSet_Chassis, NULL, NULL, &level))
                {
                RCC_EXT_WriteStr(pCliEnv, "No Supervision allowed");
                return;
                }

        if (level < 30)
                RCC_EXT_WriteStr(pCliEnv, "Supervision not available on
                         this chassis");
        else
                RCC_EXT_WriteStr(pCliEnv, "Supervise traffic on this chassis");
        }
```

9. Customizing the CLI Engine

In addition to the basic CLI engine configuration, you can modify certain aspects of the CLI engine by editing macro definitions in **rcc_custom.h** or by using custom defines on the Extensions tab of the MCE.

Basic CLI engine configuration is covered in Configuring the CLI on page 61. The Extensions dialog box is used for application level customization.

Macro Definitions on page 97 You can edit the macros in *installDir* **/vxworks-7/pkgs_v2/app/webcli/cli-***x.x.x.x* **/h/rcc_custom.h** to specify various system options, help messages, and error messages.
Custom Defines Reference on page 98 You can add custom definitions to the **rcc_custom.h** file.
Customizations from the Command Prompt on page 100

9.1. Macro Definitions

You can edit the macros in *installDir* **/vxworks-7/pkgs_v2/app/webcli/cli-***x.x.x.x* **/h/rcc_custom.h** to specify various system options, help messages, and error messages.

The following table shows some examples of the system options defined in **rcc_custom.h**:

Table 1. Example System Option Macros

Macro	Default Value	Notes
kRCC_DEFAULT_LOGIN	"admin"	Default login for development.
kRCC_DEFAULT_PASSWORD	"admin"	Default password for development.
kRCC_DEFAULT_PROMPT_TAIL	"#"	Always appended to prompt.
kRCC_INTERMEDIATE_PROMPT	">"	After line continuation char.

The following table shows some examples of the help messages defined in **rcc_custom.h**:

Table 2. Example Help Messages

Macro	Message Text
kRCC_MSG_COMMANDS	"\r\nAvailable internal commands:\r\n"
kRCC_MSG_DEPENDENCIES	"Dependencies:"
kRCC_MSG_NO_HELP	"No help available for: "
kRCC_MSG_PARAMS_AVAIL	"Available parameters:"

The following table shows some examples of the error messages defined in **rcc_custom.h**:

Table 3. Example Error Messages

Macro	Message Text
kRCC_MSG_LOGIN_FAILED	"Login failed for: "
kMSG_ERROR_RCC_INVALID_PARAM	"Invalid parameter."
kMSG_ERROR_RCC_BAD_COMMAND	"Bad command"
kMSG_ERROR_RCC_EXTRA_PARAMS	"Too many parameters"

Parent topic: Customizing the CLI Engine on page 97

Related information

Custom Defines Reference on page 98
Customizations from the Command Prompt on page 100

9.2. Custom Defines Reference

You can add custom definitions to the **rcc_custom.h** file.

Table 1. Custom Defines

#define	Description
__APPLY_NO_TO_ALL__	If the user types *no*, all nodes in command have no form.
__DISABLE_PAGED_OUTPUT__	Disables "more" feature (no paging of CLI output).
__EOL_USES_CR__	I/O uses c/r only instead of default c/r l/f combo.
__EOL_USES_LF__	I/O uses l/f only instead of default c/r l/f combo.
__NO_MEANS_NO__	*no* applied to nodes without a no handler generate an error.
__RCC_CASE_INSENSITIVE__	Ignore case of commands and parameter keywords.
__RCC_DASHED_ERROR_LINE__	Prefix error indicator (^) with dashes instead of spaces.
__RCC_ENABLE_GETHOSTNAME__	Print hostname of device in prompt.
__RCC_HIDE_BLANK_HELP__	If node or parameter has no help text or custom help function, skip it and do not generate help text for it.
__RCC_NO_HELP_INDENT__	Disable indenting help text that wraps.
__RCC_NO_KEYWORD_ARGUMENTS__	Prevent commands from accepting parameter data that matches command/parameter keywords.
__RCC_SERIAL_CONSOLE__	Enable the serial console.
__RCC_SNMP_AUTOCOMMIT__	SNMP SET operations in intermediate mode do not require explicit "commit" command.
__USE_OTHER_TELNETD__	Use third-party telnet daemon.
__USE_PASSWORD_ONLY__	Login requires just password.
AUX_TELNETD_FN	Third-party telnetd function to execute.
CONSOLE_SESSION_FN	Name of console session function.
IS_INTERACTIVE_FN	Support function for running in noninteractive mode.
RCC_LOG_INPUT_FN	Developer supplied function to log user input.
RCC_LOG_IO_FN	Developer supplied function to log user input/output.
RCC_LOG_LOGIN_FN	Developer supplied function to log login attempts.
RCC_LOG_OUTPUT_FN	Developer supplied function to log user output.

#define	Description
RCC_LOGOUT_FN	Developer supplied function to execute on user logout.
TELNET_RECV_FN	Read handler.
TELNET_SEND_FN	Write handler.

Parent topic: Customizing the CLI Engine on page 97

Related information

Macro Definitions on page 97
Customizations from the Command Prompt on page 100

9.3. Customizations from the Command Prompt

Customizable Command Prompt

You can customize the text that displays as the command prompt. In the Command Node Editor, enter the desired prompt text and, if desired, any existing WindMark in the Prompt String field (for example, Interface $%InterfaceNumber#$>). The prompt looks up the value of the WindMark and replaces that instance in the prompt text. If the WindMark is not found, nothing is added to the prompt. The use of a WindMark here enables real-time changes at the prompt.

Command History

Wind River CLI supports use of the up and down arrow keys at the command prompt to view command history.

By default, typing !! at the command prompt executes the most recent command in the history buffer. This character is defined by kRCC_CMD_BANG in **rcc_custom.h**.

Keyboard Shortcuts

Shortcuts, such as the use of keyboard arrows, can be edited in **rcc_system.h**. Keyboard behavior is dependant on the client being used.

Note: On Linux, prior to opening a telnet session, the backspace key works correctly. After opening a CLI telnet session, the backspace key does not work (**CRTL+H** continues to work). This works correctly on Windows.

IOS-like Line Editing

You can use standard IOS control keys and escape key combinations to move and modify the current command line. If you have added the help and edit commands to your command tree, this information is displayed by typing help edit at the command prompt. Editing keys can be defined in **rcc_ext.h**.

Internal Global Commands

To see the always available commands (for example, clear, exit, logout) you must configure a node to call the handler function RCC_EXEC_Help_Commands( ). An example of this is the help commands in the global sample commands file *installDir* **/vxworks-7/pkgs_v2/app/webcli/cli-***x.x.x.x* **/src/examples/commands.rcp**.

If you import these commands into your project, typing help commands at the command prompt displays the available commands.

Command Completion for Parameter Keywords

Tab completion of command names includes command name and parameter keyword completion.

Case Sensitivity

You can set case sensitivity for commands and parameters using the compile-time flag **__RCC_CASE_INSENSITIVE__**.

Parent topic: Customizing the CLI Engine on page 97

Related information

Macro Definitions on page 97
Custom Defines Reference on page 98

10. Enabling Access to the Kernel Shell

Wind River CLI provides the code to support the command to transfer from the CLI environment to the kernel target shell environment.

About This Task

For example usage, see VxWorks 7 Creating a CLI Tool Tutorial and VxWorks 7 Creating a CLI Tool and Web Server Tutorial. The commands for this feature are located in the following files:

installDir **/vxworks-7/pkgs_v2/app/webcli-***x.x.x.x* **/cli/src/examples/cli2krnl.c**

installDir **/vxworks-7/pkgs_v2/app/webcli-***x.x.x.x* **/cli/src/examples/krnl2cli.c**

installDir **/vxworks-7/pkgs_v2/app/webcli-***x.x.x.x* **/webclidemo/src/wmc_options.h**

For information on working with the kernel shell as part of the CLI tool tutorial, see the VxWorks 7 Creating a CLI Tool Tutorial.

To configure your project to use this feature, do the following:

Procedure

1. Import the following files into your project.
   - **cli2krnl.c**
   - **krnl2cli.c**
   - **wmc_options.h**
2. Create a new command for your project to transfer to the kernel shell.

   For example, you could create a command named vxshell to transfer to the kernel shell.
3. Create a handler named **wmvxshell** for the command to transfer to the kernel shell.

   Note: The handler name is fixed unless you edit the **cli2krnl.c** file.
4. Build the project.

The build will fail due to a conflict between the wmvxshell( ) null stub function generated in the **rcc_handler.c** file and the function in the **cli2krnl.c** file.

5. In the **rcc_handler.c** file, delete the wmvxshell( ) null stub function so that the **wmvxshell** handler uses the function in the **cli2krnl.c** file.
6. Build your project again.

Next Steps

When you run the DKM on the target, use the vxshell command in the CLI shell to enter the kernel shell and use the exit command in the kernel shell to return to the CLI shell.

11. Enabling Scripting Support

Wind River CLI supports scripting using the commands you implement in your project.

About This Task

This lets you define your own scripting grammar that is not tied to the Wind River CLI target code. Parameter ordering must be set to Full to enable scripting support.

WindMarks are accessed directly by preceding the name with % as #defined by kRCC_RM_PREFIX in **rcc_custom.h**. For example, the following command prints the value of the WindMark sysName:

```
echo %sysName
```

Session variables are prefixed with $ as #defined by kRCC_VAR_PREFIX in **rcc_custom.h**. They are local to the user's session (user A's $myvar is distinct from user B's $myvar). Both can see the same %mywindmark.

WindMarks are accessed directly by preceding the name with % as #defined by kRCC_RM_PREFIX in **rcc_custom.h**. For example, the following command prints the value of the WindMark sysName:

```
echo %sysName
```

To enable scripting from a VSB, see Source Build Configuration Options (VSB) on page 38. To review pre-defined commands, see scripting.rcp Scripting Commands Reference on page 103.

To enable scripting support from the MCE, do the following:

Procedure

1. Select **CLI** from the left pane of the MCE.
2. Select the **General** tab.
3. In the Command section, set **Parameter Ordering** to Full.

   For more information on this option, see General CLI Properties Dialog Reference on page 139.

4. Select the **Advanced** tab.
5. In the Shell section, select **Scripting Support**.

For more information on this option, see General CLI Properties Dialog Reference on page 139.

6. Save **config.webcli**.

12. scripting.rcp Scripting Commands Reference

A number of commands are found in the **scripting.rcp** file.

This file is located at:

installDir **/vxworks-7/pkgs_v2/app/webcli/cli-***x.x.x.x* **/src/examples/**

Table 1. Commands in scripting.rcp

Command	Description
decrement	Decrease value of; takes an integer as a required parameter.
divide by	Divides parameter original by parameter denominator.
eq	Equals - is same value.
ge	Greater than or equal to.
gt	Greater than.
if	If x is true then do y.
if else	If x is true then do y, else do z.
if else end	Used to end else.
increment	Increase value of; takes an integer as a required parameter.
is	Assign value.
le	Less than or equal to.
ls	List files.
lt	Less than.
minus	Subtracts parameter added from parameter initial.
plus	Adds parameter added and parameter initial.
times	Multiplies parameter initialValue by parameter multiplier.

Command	Description
while	Takes the parameter whiletest.
while end	Ends while loop.

13. Displaying Tables in Wind River CLI

Of the createTable options, CLI offers only skipAhead, which takes only two parameters: pCliEnv (which describes the user session) and the skip ahead amount as an integer.

```
STATUS wmbTableSkipAhead
    (
    WMB_SESSION_T * pSession,
    sbyte4 skipAhead
    )
```

Many of the features of the createTable WindMark for the Web Server do not make sense for CLI. With the Web Server, you have to be able to programmatically filter and instance objects. For CLI, you are requesting in real-time.

The following example shows a command handler that uses the wmbTableCreate( ) function to display an SNMP table. For CLI table printing, enter code in the command handler like the following sample for the tcpConn table:

CLI SNMP Table Display Example

```
extern RLSTATUS
cliShowConnections(cli_env *pCliEnv, paramList *pParams, sbyte *pAuxBuf)
    {
        wmbTableCreate(pCliEnv, "tcpConnState", "tcpConnState.1",
        "tcpConnRemAddress,tcpConnRemPort,tcpConnState");
    wmbTablePrint(pCliEnv);
    wmbTableDestroy(pCliEnv);
    return OK;
}
```

The prototype for the wmbTableCreate( ) function is as follows:

```
STATUS wmbTableCreate
    (
    WMB_SESSION_T * pSession,
    sbyte * pIndexColumn,
    sbyte * pFirstInstance,
    sbyte * pSelectedColumns
    )
```

pSession

The user environment, normally the command handler uses **pCliEnv**.

pIndexColumn

The name of the index column. Also used as the name of the table.

WNDRVR

pFirst Instance

NULL, OID, or WindMark used to reference the first instance of the table created.

pSelected Columns

The comma-separated names in **pSelectedColumns** can be entered as just the name of the WindMark, and the spacing is determined for you based on the width of the WindMark name.

You can fix the column spacing by entering the name, label, and width in the format of name:label:width. name is the name of the WindMark, label is the heading to be put at the top of the column, and width is the width of the column, as in

```
tcpConnState:state:15
```

The width must be greater than the character width of the column heading. Data that extends beyond the column width is not truncated; it is displayed in its entirety, but pushes other items to the right, so that columns do not line up properly.

You cannot repeatedly use the wmbTablePrint( ) function. You must make each call to the wmbTablePrint( ) function from within a unique set of wmbTableCreate( ) and wmbTableDestroy( ) calls.

10. CREATING A WEB-BASED MANAGEMENT INTERFACE

1. Introduction

You can create a set of Web files and URLs your device's users can use to control, configure, and manage the device.

In designing the device's user interface, there are two things to keep in mind: you must create HTML files and you must decide how to organize them.

For creating the files, you can use any text/HTML editor you like—including the text editor built into the MCE.

You use the MCE to create the URL tree (site tree) and configure the target file system(s) for your embedded Web server. The URL tree, Web files, access control user groups and IP locations, and custom RPMs are displayed in the left pane of the MCE.

There is very little (if anything at all) you need to do to manually configure any required RPMs. As you build your URL tree and HTML file management interface, the MCE automatically configures the required RPMs for you in the generated code.

2. Creating a URL

In addition to creating the Web files that make up the management interface for the device, you need to design the navigational structure of the Web files from the Web client (browser) perspective.

About This Task

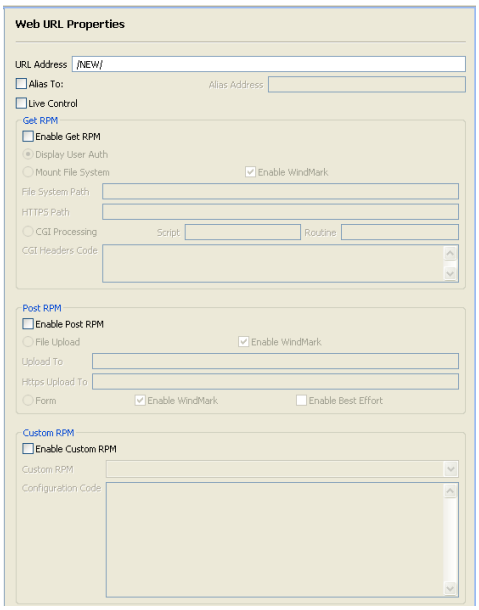
For each URL you create, you need to use the Web URL Properties dialog box to indicate exactly what processing should take place when an HTTP request is made of that URL.

To create a URL, do the following:

Procedure

1. In the left pane of the MCE, right-click **Web Server** and select **New Child > URL**.

   The Web URL Properties dialog box appears in the right pane of the MCE.



2. In the **URL Address** field, type a name for the new URL substring you wish to mount a file system at.

   The URL address must start with a **/** or *****, contain valid characters, and end with a **/**; for example, /fs/.

3. Set the properties of the URL using the Web URL Properties dialog box.

   For details on the options in the Web URL Properties dialog box, see Web URL Properties Reference on page 161.

4. Save **config.webcli**.

   Select **File > Save**, or press **CTRL+S**.

3. Setting Up URL Aliasing

URL aliasing treats a request made of the aliased URL exactly as if it were a request made of the (real) target URL.

About This Task

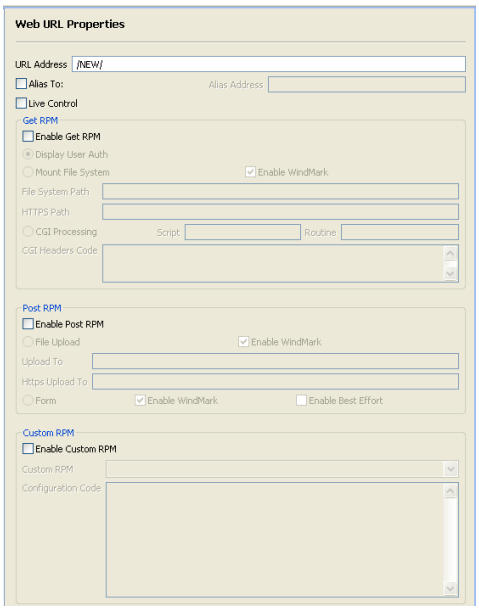
For example, you could have the URL **/fs/doc.html** alias to the URL **/fs/docs/documents.html**. In this case, the server treats a request for **/fs/doc.html** just as if it were a request for **/fs/docs/documents.html** and returns the appropriate response.

To set up URL aliasing, do the following:

Procedure

1. Create a new URL or select the desired URL from the left pane of the MCE.

   The Web URL Properties dialog box appears in the right pane of the MCE.



2. Select **Alias To**.
3. Type the target URL for this alias in the **Alias Address** field.

> Note: The target URL you specify must exist on the system at run-time. The tool does not check to see that there is a legitimate HTTP request at the target end of your new alias. Since you can add files at run-time, you may know that a file will exist at that URL, but it may not be there to check at design time.
>
> You can also add aliases dynamically at run-time using the API function httpAliasConfAdd( ). For more information, see the reference entry for httpAliasConfAdd( ).

4. Displaying User Authentication

Displaying user authentication is a useful debugging feature.

About This Task

This allows you to display the current user's authentication information by requesting a particular URL. For example, if you create the URL called **/showauth** and assign the show authentication RPM to that URL, the following HTTP request returns the current user's authentication information:

http://*myDeviceIP***/showauth**

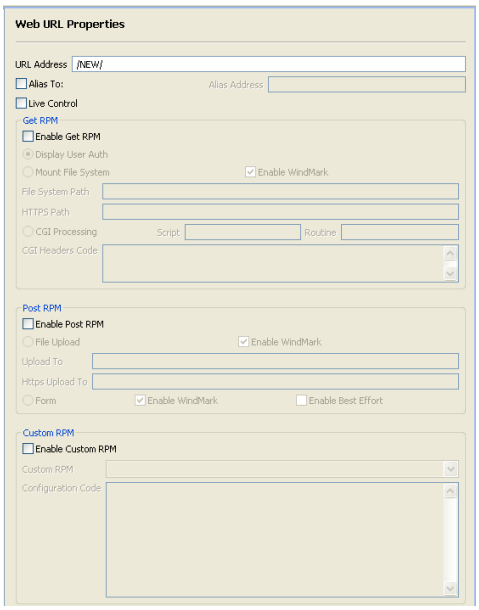
The returned authentication information is Base64 encoded and provides only minimal security. Also, the show user authentication RPM (**httpRpmAuth.c**) returns **RPM_DONE**, so it cannot be used with other URL-specific options except access control.

To enable the Display User Auth feature, do the following:

Procedure

1. Create a new URL or select the desired URL from the left pane of the MCE.

The Web URL Properties dialog box appears in the right pane of the MCE.



2. In the Get RPM section, select **Display User Auth**.

Next Steps

Note: Wind River Web Server provides the Display User Auth feature for debugging user authentication during development. You should remove this feature from your project prior to releasing it as a product.

5. Enabling CGI Processing

CGI is a standard that describes how external applications exchange information with an information server such as an HTTP server.

About This Task

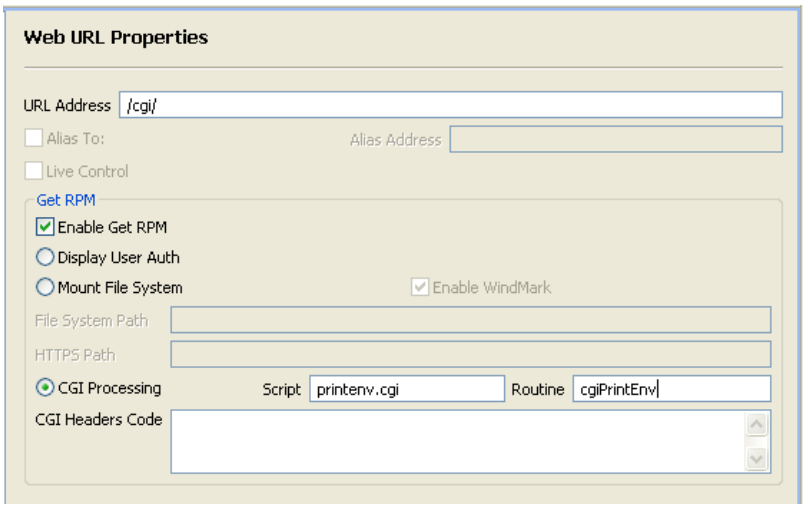
You can easily adapt applications that you have written to use the common gateway interface to run in the Wind River Web Server application space by means of the server environment that stores the CGI variables. Wind River Web Server provides the CGI RPM as a legacy porting aid. For information on how you can call a CGI application for requests made of a particular URL.

To enable CGI processing, do the following:

Procedure

1. Create a new URL or select the desired URL from the left pane of the MCE.

   The Web URL Properties dialog box appears in the right pane of the MCE.



2. In the Get RPM section of the Web URL Properties dialog box, select **CGI Processing**.
3. In the **Script** field, type the incoming request URL path for the CGI function.
4. In the **Routine** field, type the name of the CGI function that will process the incoming request.

   When you enable a CGI URL, any request of that URL must have the appropriate URL substring (such as **/cgi** in this example) prepended to the URL. Therefore, to access the CGI function in this example, the address in the browser (or more likely encoded in a form element) is: **http://***yourDeviceIP***/cgi/printenv.html**

6. Enabling File Uploads

Wind River Web Server allows RFC 2854 file-based uploads.

About This Task

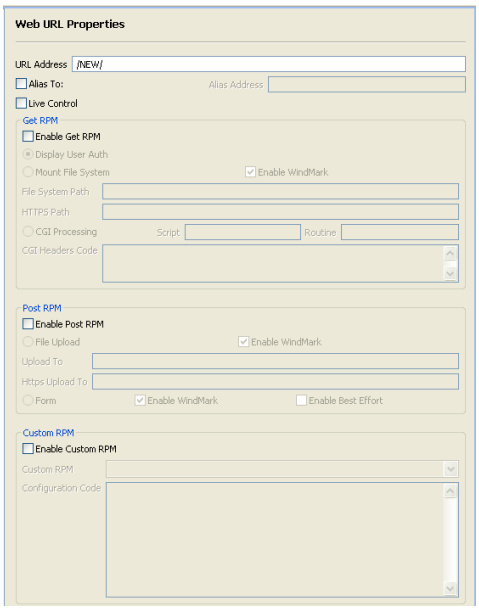
File uploads are only supported for ANSI or CUSTOM file systems. You cannot enable uploads for a ROMBUILD file system.

To specify an upload path, do the following:

Procedure

1. Create a new URL or select the desired URL from the left pane of the MCE.

   The Web URL Properties dialog box appears in the right pane of the MCE.



2. Select **File Upload**.
3. In the **Upload To** field, type the target path you wish to mount.

> **Note:** The selected file system for the upload path must be writable.

Results

Code is added to the generated file **wmw_httpconf.h** when it is rebuilt. For example, if you create a URL **/incoming** and specify its upload path as **/ram0**, the following code is added:

```
httpRpmConfAdd(HTTP_M_POST, "/incoming/", httpRpmUpload);
httpUploadConfAdd("/incoming/", "/ram0/");
```

The first call, httpRpmConfAdd( ), configures the upload RPM and associates the URL substring /incoming with it.

The second call, httpUploadConfAdd( ), specifies the request URL path (on the URL tree) and the upload path on the target file system.

You may want to do some post processing of uploaded files, or evaluate other form elements (text boxes, check boxes, and so on) that may be present in the upload page. If you select the Enable WindMark option for an upload URL, the backplane RPM parses the additional form elements in the POSTed message for WindMarks and (through the backplane) calls the appropriate handlers.

You can see which files have been uploaded by calling httpGetEnv( ) with the UPLOAD_FILE_NAME parameter, and querying the httpRpmUpload RPM module. For example, you can modify httpPostReply( ) to display the uploaded file's name and path:

```
char * uploadFileName;
uploadFileName = httpGetEnv(reqId, "UPLOAD_FILE_NAME");
printf("\n Upload File Name:%s\n", uploadFileName);
```

> **Note:** An environment variable such as *UPLOAD_FILE_NAME* is stored within each *ReqId* structure, so it can only be accessed when processing each request.

7. IPv6 support for SMTP

You can use e-mail alerts to notify users or administrators of certain conditions. The e-mail alert mechanism in Wind River Web Server uses the Simple Mail Transfer Protocol (SMTP).

The file **smtpLib.c** implements an interface to the Wind River Web Server mail transfer agent (MTA). The MTA sends email messages to an SMTP server on the device or the network, which in turn delivers them.

IPv6 support for SMTP is implemented in the VxWorks 7 file **wmnetVxWorks.c** The routines wmnetTcpConnect( ), tcpSocketCreate( ), bindSocketToAddr( ) and connectToServer( ) have been modified to support both IPv4 address and IPv6 address and DNS domain name.

8. Adding Web Files to Your Project

You can add Web files and specify the settings for the files in your project using the Web File Properties dialog box in the MCE.

About This Task

This is where you indicate how the server should access the file—ANSI_FS or MEM_FS. You can also flag files for custom handling. For more information, see Registering CUSTOM_FS Files on page 114.

The designations in the Web File Properties dialog box specify how to access a file. They do not specify the exact mount point, media type, or any other specifics. This information is typically provided by the file system RPM configuration data which the MCE generates from the settings in the URL configuration.

For example, the URL **/fs/index.html** mounts the ANSI file system whose target path is **/romfs/webfiles/doc**.

The code generated (in **wmw_httpconf.c**) for this URL looks like the following:

```
httpRpmConfAdd(HTTP_M_GET, "/fs/", httpRpmFs);
                 httpFsConfAdd("/fs/", "/romfs/webfiles/doc");
                 httpFileSetDefaultFs(ANSI_FS);
```

In this case, files in your project that are stored on the target file system under **/romfs/webfiles/doc** do not need any file settings provided at all.

> Note: Most Web files do not need any file-specific settings at all since the default file system settings work for them. The server can typically access them by the information generated for the file system RPM. Typically, only files with nonstandard handling requirements need to have settings specified.

For an incoming HTTP request for the URL **/fs/myfile.html**, the server first looks for a registry entry for myfile.html. If the file is not registered, the server accesses the mounted file system at **/romfs/webfiles/doc/** using the default file system access functions (ANSI_FS in this case) and retrieves the file **myfile.html**.

For more information on working with Web files, see Working with Web Content Files on page 122.

To add a file to your project, do the following:

Procedure

1. In the left pane of the MCE, right-click Web Server and select **New Child > Web Files**.

The File System Image Generation dialog box appears.

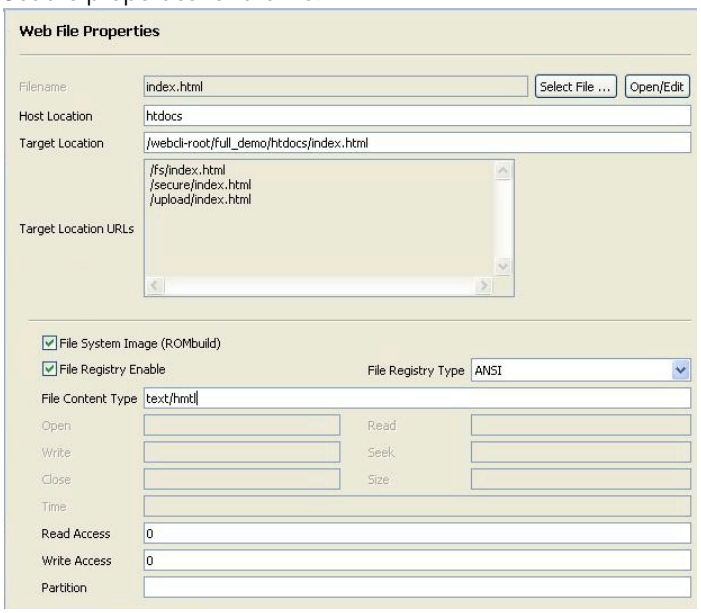
2. Set the build file system options.

   For more information, see File System Image Generation Reference on page 164.

3. In the left pane of the MCE, right-click **Web Files** and select **New Child > File**.

   The Web File Properties dialog box appears.

4. Click **Select File** and navigate to the desired file.
5. Set the properties for the file.



   For details on the options in the Web File Properties dialog box, see Web File Properties Reference on page 165.

9. Registering Files

The Wind River Web Server follows a prescribed file search order based on which file system type is specified as the default.

About This Task

See Web Server Resource Properties Reference on page 158.

The search order always begins with the file registry. If the requested file is not registered, the server attempts to find the file in the default file system. If the default file system does not contain the requested file, the server follows a specific search order (see File System Search Order on page 122).

To register a file, do the following:

Procedure

1. In the left pane of the MCE, select the desired file.

   The Web File Properties dialog box appears in the right pane of the MCE.

2. Select **File Registry Enable**.
3. Select the desired file type from the **File Registry Type** drop-down menu.

   Registering CUSTOM_FS Files on page 114 Because a file of type **CUSTOM_FS** must always be a registered file, a **CUSTOM_FS** file is processed with its registered file access functions.

9.1. Registering CUSTOM_FS Files

Because a file of type **CUSTOM_FS** must always be a registered file, a **CUSTOM_FS** file is processed with its registered file access functions.

About This Task

Accordingly, you cannot select **CUSTOM_FS** as the default file system.

When you register a **CUSTOM_FS** file, you must indicate the content type and choose which file system access functions to use. If you choose ANSI, compressed ANSI, ROMbuild, or compressed ROMbuild, the access functions are provided for you. If you choose custom, you must provide the following six file access functions (analogous to the ANSI standard C file access functions):

customOpen( )	**customRead( )**
customWrite( )	**customSeek( )**
customClose( )	**customSize( )**

To register a **CUSTOM_FS** file, do the following:

Procedure

1. In the left pane of the MCE, select the desired file.

   The Web File Properties dialog box appears in the right pane of the MCE.

2. Select **File Registry**.
3. From the **File Registry Type** drop-down menu, select **Custom**.
4. Provide the appropriate file access functions.
5. Save **config.webcli**.

Parent topic: Registering Files on page 113

10. Adding Access Control to Your Project

The Wind River Web Server access control model, if used, restricts HTTP request access by user group and IP group.

> Note: This is different from the backplane access control model. The backplane access control model restricts read or write access to WindMarks. The embedded HTTP server restricts access to HTTP requests (URLs), and not to WindMarks contained in those requests.

Conceptually, a Web Server user group is no different than a user group on a UNIX or Windows network—it is a means of restricting access to HTTP requests on a per URL basis. File access at each URL can also be restricted by IP group.

Creating User Groups on page 115 You define one or more user groups by creating a list of users and a group name.
Creating IP Locations on page 116 You define an IP location by listing a range or pattern of IP addresses and assigning that range or set of patterns an IP location name.
Adding Access Control to a URL on page 117 Once user and IP groups are defined, each URL can be assigned unique access control options.

10.1. Creating User Groups

You define one or more user groups by creating a list of users and a group name.

About This Task

You can define or modify user groups and IP groups at any time during the process of mapping URLs. It is not necessary to do this when you first create a project.

Additionally, using the provided API, you can add user or location groups programmatically at run time.

> Note: The scope of user groups is global. If you edit a particular user group, your changes affect every use of that user group.

To create a user group, do the following:

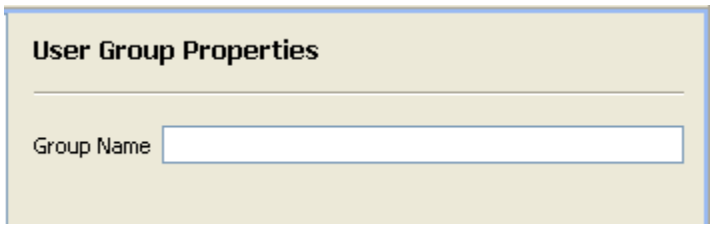
Procedure

1. Create your access control list.

   In the left pane of the MCE, right-click **Web Server** and select **New Child > Access Control List**.

2. Add a user group:
   1. In the left pane of the MCE, right-click **Access Control** and select **New Child > User Groups**.
   2. In the left pane of the MCE, right-click User Groups and select **New Child > Group**.

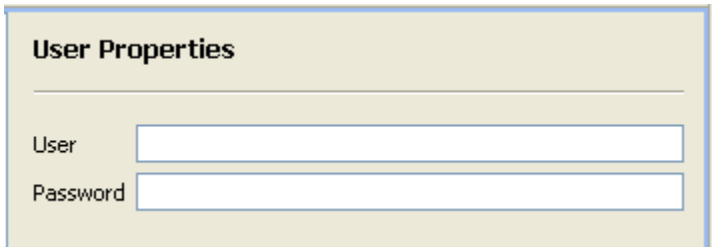
   The User Group Properties dialog box appears.



3. In the **Group Name** field, type a name for your user group.

3. Add users to the group:
   1. In the left pane of the MCE, right-click your new user group and select **New Child > User**.

      The User Properties dialog box appears.



   2. Enter the name of the user and the desired password.
   1. Repeat steps a and b as many times as necessary to add the desired number of users to your user group.

Parent topic: Adding Access Control to Your Project on page 115

Related information
Creating IP Locations on page 116
Adding Access Control to a URL on page 117

10.2. Creating IP Locations

You define an IP location by listing a range or pattern of IP addresses and assigning that range or set of patterns an IP location name.

About This Task

You can define or modify IP groups at any time during the process of mapping URLs. It is not necessary to do this when you first create a project.

Additionally, using the provided API, you can add location groups programmatically at run time.

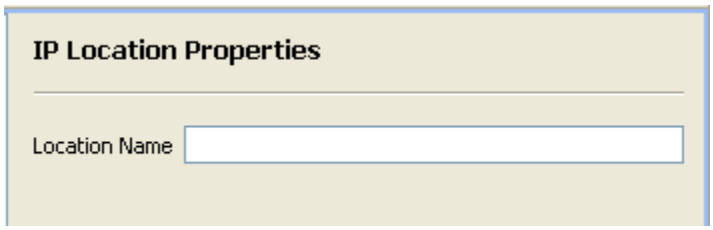
> Note: The scope of IP location names is global. If you edit a particular IP location, your changes affect every use of that IP location.

To create IP locations, do the following:

Procedure

1. Add IP locations:
   1. In the left pane of the MCE, right-click **Access Control** and select **New Child > IP Locations**.
   2. In the left pane of the MCE, right-click IP **Locations** and select **New Child > Location**.

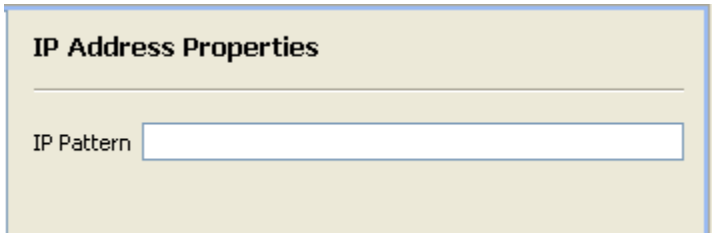
      The IP Location Properties dialog box appears.



3. Enter a name for your IP location in the **Location Name** field.

2. Add IP addresses:
   1. In the left pane of the MCE, right-click your new location and select **New Child > IPAddress**.

   The IP Address Properties dialog box appears.



   2. In the **IP Pattern** field, type the desired IP pattern.

   For IP pattern matching, the only meta character presently allowed is the asterisk * character. For example, if you enter 192.168.0.* as the IP pattern, it would include all clients with an IP address in the ranges of 192.168.0.0 through 192.168.255.255.

   3. Repeat steps a and b as many times as necessary to add the desired number of IP patterns to your location.

Parent topic: Adding Access Control to Your Project on page 115

Related information
Creating User Groups on page 115
Adding Access Control to a URL on page 117

10.3. Adding Access Control to a URL

Once user and IP groups are defined, each URL can be assigned unique access control options.

About This Task

Any URL can be restricted to a user group, an IP group, or both a user group and an IP group. If you do not assign user groups or IP locations to a URL, the default behavior is to allow all user groups and all IP addresses.

Procedure

1. In the left pane of the MCE, right-click the desired URL and select **New Child > ACL Group/Location Pairs**. The Access Control Group/Location Pair dialog box appears.
2. From the **Groups** drop-down list, select the desired user group.
3. From the **Locations** drop-down list, select the desired IP location.
4. Repeat steps 1 through 3 as many times as necessary to add the desired number of user groups and locations to your URL.

Parent topic: Adding Access Control to Your Project on page 115

Related information
Creating User Groups on page 115
Creating IP Locations on page 116

11. Adding Custom RPMs to Your Project

In addition to each of the processing options provided by the embedded HTTP server, you can create your own request processing modules (RPMs) and specify that all requests made of a particular URL be processed by your custom RPM.

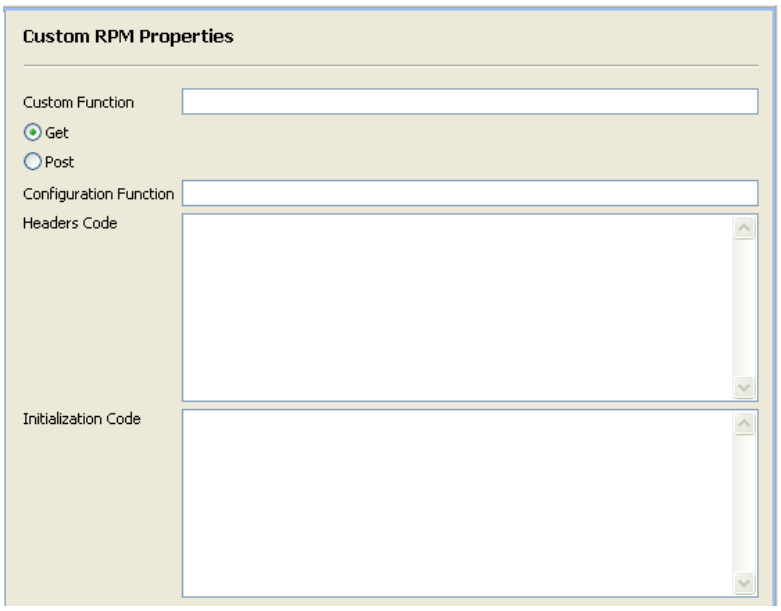
About This Task

To create a custom RPM, do the following:

Procedure

1. In the left pane of the MCE, right-click **Web Server** and select **New Child > Custom RPMs**.
2. In the left pane of the MCE, right-click **Custom RPMs** and select **New Child > Custom RPM**.

   The Custom RPM Properties dialog box appears.



3. Set the desired options for your custom RPM. For details on the options available, see Custom RPM Properties Reference on page 168.
4. After you create a custom RPM, you can specify that all requests made of a particular URL be processed by your custom RPM by doing the following:
   1. Create a new URL or select the desired URL from the left pane of the MCE.

      The Web URL Properties dialog box appears in the right pane of the MCE.

   2. Select **Enable Custom RPM**.
   3. In the **Custom RPM** field, provide the name of the custom RPM function.

If you have already entered information about a custom RPM you have written, you can select your RPM from the drop-down list.

12. Adding Custom MIMEs to Your Project

The MIME processing RPM is included by default in the minimal Wind River Web Server configuration.

About This Task

In the project facility, this RPM is the MIME processing component.

For manual configuration, include the **INCLUDE_HTTP_MIME_HDR_PROC** macro.

During server initialization, a table is created, which associates request header fields with the functions used to process them. The fields that make up the MIME header are stored in a generic list. The following calls are made by default in **httpglob.c** to populate the table with entries for the basic header fields:

```
httpMimeParseFnConfAdd ("CONTENT-TYPE", httpMimeContentType );
httpMimeParseFnConfAdd ("CONTENT-LENGTH", httpMimeContentLength );
httpMimeParseFnConfAdd ("ACCEPT_ENCODING”, httpMimeAcceptEncoding );
#ifdef INCLUDE_HTTP_PWD_CHECKING
httpMimeParseFnConfAdd ("AUTHORIZATION", httpMimeAuthorization );
#endif
#ifdef INCLUDE_HTTP_LANGUAGE_NEGOTIATION
httpMimeParseFnConfAdd ("ACCESS-LANGUAGE",httpMimeAcceptLanguage );
#endif
#ifdef INCLUDE_HTTP_PERSISTENT_CONNECTION
httpMimeParseFnConfAdd ("CONNECTION", httpMimeConnection );
httpMimeParseFnConfAdd ("KEEP-ALIVE", httpMimeKeepAlive );
httpMimeParseFnConfAdd ("PERSIST", httpMimePersist );
#endif
#ifdef INCLUDE_HTTP_COOKIE_SUPPORT
httpMimeParseFnConfAdd ("COOKIE", httpMimeReadCookie );
#endif
```

The Wind River Web Server includes the source code for the MIME processing RPM, so that you may create functions for processing other header fields.

The httpMimeParseFnConfAdd( ) function is used to associate the header fields and custom function.

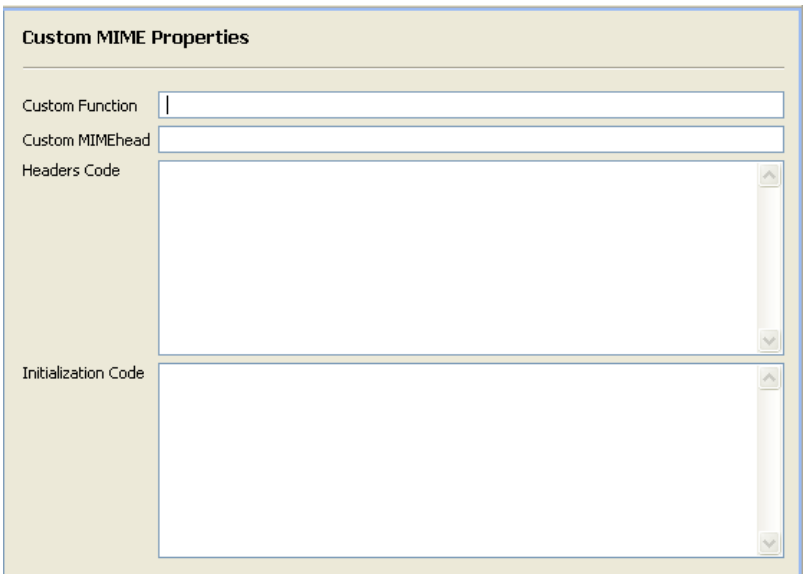
To create a custom MIME, do the following:

Procedure

1. In the left pane of the MCE, right-click **Web Server** and select **New Child > Custom MIMEs**.
2. In the left pane of the MCE, right-click **Custom MIMEs** and select **New Child > Custom MIME**.

   The Custom MIME Properties dialog box appears.

WNDRVR



3. Set the desired options for your custom MIME.

   For details on the options available, see Custom MIME Properties Reference on page 169.

13. Writing HTML Containing WindMarks

When you request an HTML page coded with WindMarks, Wind River Web Server replaces each WindMark found in the page with the current value of the device-resident variable the WindMark represents, then dynamically generates an HTTP response using the substituted values.

Following are the three parts to this process:

The Data

> The data can be a standards-based object, such as an SNMP object, or a custom object or variable unique to the device you are creating. The Wind River Web Server does not provide any of the device data for you.

The WindMark

> The WindMark is the name that represents the data and the corresponding method for getting or setting the data. For scalar variables and SNMP objects, the MCE generates the WindMark and handler code for you. For custom data objects, you must provide the handler code.

The HTML Markup Containing WindMarks

> In HTTP GET requests, the HTML markup specifies how the data is displayed. In HTTP POST requests, the markup indicates the WindMark name the accompanying form data value is paired with. You must provide the HTML code containing WindMarks.

Consider a rather generic example. The following line of HTML code displays a single paragraph of text:

```
<p>The name of this system is myDevice. </p>
```

Consider the same line of HTML with a WindMark in place of the hard-coded **myDevice**:

```
<p>The name of this system is $%deviceName#$. </p>
```

Now the actual initialization and storage for the value of *deviceName* could be handled in a variety of ways, but for the sake of this example, assume it is intended to represent a basic string (char[]) variable defined in a source file in your code as:

```
char deviceName[] = "My Favorite Device"
```

Create this WindMark as a string type and specify its absolute address as deviceName (the address of the char array). Now you have everything in place. You have implemented and initialized the string variable *deviceName*. You have created a WindMark named deviceName, and indicated (by specifying the address of the array deviceName) how to get the value of the variable. And lastly, you have some HTML code that references the WindMark deviceName.

When you request the page with the following code in it:

```
<p>The name of this system is $%deviceName#$. </p>
```

The embedded HTTP server parses the Web file and encounters the WindMark delimiters $% and #$. The server takes the name between the WindMark delimiters and queries the backplane for the value of this WindMark. The backplane passes the value of the WindMark back to the HTTP server, then the HTTP server assembles the HTTP response with the value of the WindMark substituted in place of the delimited name it encountered. The result displayed in the browser is a single paragraph of text like the following:

```
The name of this system is My Favorite Device.
```

This is a simplified example, but it is the basis of what Wind River Web Server does for you. It allows you to have HTML interface Web pages that are abstracted from the code that actually handles (GETs ad SETs) the device management information. You completely separate interface from implementation.

WindMark Delimiters and Naming Rules

The starting delimiter is **$%**. The ending delimiter is **#$**.

Note: You also have the option to set double bracket pairs ([[and]])as the delimiters, but do not do it in the tutorial as all the HTML files are currently written to use **$%** and **$#** as the delimiters.

The rules for the text which make up a WindMark name are a to z, A to Z, 0 to 9, and underscore (_).

```
$%field8_name#$
```

is an acceptable WindMark. WindMarks may even have arguments. Complex WindMarks use parentheses and commas, but no whitespace. For example:

```
$%customerinfo(cust_name,cust_email,cust_phone)#$.
```

11. WORKING WITH WEB CONTENT FILES

1. Introduction

The Web files in your project are the GUI your customer will interact with to configure or monitor the device.

Wind River Web Server gives you the flexibility to work with your Web files in a way that fits into your workflow.

You can manually code the files in your preferred HTML or text editor. You can also write the Web files in the editor/viewer built into the MCE.

The left pane of the MCE gives a host-side perspective of the target Web files in a project. It is important to understand that this has little relation to how the target file system looks from the target's perspective. A file is visible in your project for the purposes of editing the file and the WindMarks it references. The fact that a file is visible in your project does not mean it is automatically accessible from your target. You must either add the file to a ROMBUILD image, or manually ensure that it gets added to an ANSI file system mounted by the HTTP server. MCE allows you to do the following:

- Add (or remove) Web files to your project.
- Associate Web files in your project with a particular file system.
- Flag files to be built into the ROMBUILD image.
- Flag files to be registered—a file registry entry overrides any other file search mechanisms.
- Flag files for custom handling—in addition to the file registry, files that require access functions other than ANSI or NV memory file access functions can be flagged as **CUSTOM_FS**. You must provide custom handlers for each file flagged as **CUSTOM_FS**.

Building the zlib Compression Library

Wind River Web Server requires the use of the common zlib compression libraries. These source files are located in *installDir* **/ vxworks-7/pkgs_v2/os/utils/zlib-***x_x_x* **/**. To add zlib to your project, add the **INCLUDE_ZLIB** component to your VIP. For more information, see Additional VIP Components on page 52.

2. File System Search Order

When you have a Web server with multiple file systems, it is likely that a file exists in more than one file system.

For example, the original file system might be stored in nonvolatile memory. When the device is first used, a flash file system might be created from this ROMBUILD NVM file system. Perhaps patches or field updates overwrite the Flash file system, while the ROMBUILD NVM file system remains present for a "restore to factory default" type of reset.

In cases where you mount multiple file systems, the file systems are searched in a particular order. This order is different depending on which file system is selected as your default file system (for information on selecting a default file system, see Web Server Resource Properties Reference on page 158.

The file search order when the default file system selection is as follows:

ANSI, not compressed

1. File Registry
2. ANSI_FS
3. MEM_FS

4. 404 (file not found) error

With this setting, files in a compressed ANSI or compressed ROMBUILD NVM file system are not retrieved by the server unless they are registered.

NVM, not compressed

1. File Registry
2. MEM_FS
3. 404 error

With this setting, files in any file system other than uncompressed NVM are not retrieved by the server unless they are registered.

ANSI, compressed

1. File Registry
2. CMP_ANSI_FS
3. CMP_MEM_FS
4. 404 error

With this setting, files in an uncompressed ANSI or uncompressed NVM file system are not retrieved by the server unless they are registered.

NVM, compressed

1. File Registry
2. CMP_MEM_FS
3. 404 error

With this setting, files in any file system other than compressed NVM are not retrieved by the server unless they are registered.

In any case, a CUSTOM_FS file must be a registered file. Registered files are always retrieved first.

GZIP File Compression and Uncompression

You can configure the server to uncompress files automatically. If your default file system is uncompressed, you can uncompress files selectively by registering individual files as compressed files.

Note: Wind River HTTP Server does not support HTTP compression when sending data to the HTTP connection. That is, Wind River Web server cannot deliver compressed files to browsers.

3. Creating the URL for the File System

When you create a ROMBUILD file system, the files flagged to be included in the file system are stored in a C source file in coded form. The C file is built into the binary image.

About This Task

Even though the file will be built into the ROMBUILD file system, and there is no actual mounted file system, you must still provide a target file system path in the Web File Properties dialog box so that the file is correctly mapped to a URL. You do not need to create

an actual file system, just a placeholder file system name to establish the link between a URL and a file to be served. The name is arbitrary, but should be a potentially valid file system name (for example, “rom” or “dummy”).

To create the URL for the file system, do the following:

Procedure

1. In the left pane of the MCE, right-click **Web Server** and select **New Child > URL**.

   The Web URL Properties dialog box appears in the right pane of the MCE.

2. In the **URL Address** field, type the name of the file system.

   The URL address must start with a **/** or *****, contain valid characters, and end with a **/**; for example, **/rom/** or **/dummy/**.

3. Select **Enable Get RPM**.
4. Select **Mount File System**.
5. In the **File System Path** field, type the target file system’s mount point.

   For example, **/romfs/webfiles/doc**, or **/tffs0/htdocs**.

6. Save **config.webcli**.

4. Building Files into the ROMBUILD File System

A file can be both in the ROMBUILD NVM file system and registered for special handling.

About This Task

An example where you might want to do this is where the default file system is a compressed NV file system. There may be files in the NV file system that you do not want uncompressed by the server. The MIME type guessing RPM handles this decision for known MIME types, but in cases where the content type is ambiguous—such as a *.jar file—you could specify ROMbuild instead of CompressedROMbuild. Then, for that particular file, the server does not uncompress the file before sending the response.

For each file you want to build into the ROMBUILD file system, do the following:

Procedure

1. In the left pane of the MCE, select the desired file.

   The Web File Properties dialog box appears in the left pane of the MCE.

2. Enter the target path for the file in the **Target Location** field.
3. Select **File System Image (ROMbuild)**.
4. Select **File Registry Enable**.
5. Select either **ROMbuild** or **CompressedROMbuild** as the **File Registry Type**.
6. Save **config.webcli**.

WNDRVR

5. Special WindMarks and WindMark Macros Reference

Macros are special built-in functions that allow you to dynamically generate HTML based on various conditions.

These conditions can be simple hard-coded conditions or they can be based on the value of device resident variables (as represented by WindMarks).

Note:

Macros do not work inside the **fileInclude** WindMark. For example, you cannot display a table object that uses the **REPEAT** macro in a **fileInclude** WindMark. You can use any WindMark that does not use macros in a **fileInclude**.

The special WindMarks and WindMark macros are as follows:

REPEAT Macro

A loop mechanism that allows you to iterate through a segment of HTML code multiple times.

IF Macro

A conditional mechanism that allows you to generate HTML based on the evaluation of a test condition.

endRow Macro

Works in conjunction with the **createTable** macro to indicate the end of an SNMP table row.

endTable Macro

Works in conjunction with the **createTable** macro.

fileInclude WindMark

Allows you to piece together web pages from smaller templatized HTML file fragments.

getInstance WindMark

Allows you get the current instance of a WindMark table member.

For more information on the special WindMarks and WindMark macros, see WindMarks in HTML Form Elements on page 173.

6. Applets and Live Control

Live Control provides a method for updating the visual display of rapidly changing data values.

Since HTTP is a REQUEST-RESPONSE protocol, the requested HTTP resource must be reloaded each time a request is made. Additionally, the display can only update when a request is actually made. You could use a meta refresh HTML tag to reload the page at a specified interval, but this approach has limits and is not likely to achieve a realistic refresh rate better than a few seconds.

For many actions, the limitation of a user actually making a request is not an issue. For others, particularly conditions where you want to provide the ability to monitor rapidly changing data, you need the ability to display the data as it changes without the overhead of reloading the entire page. When you load a Java applet into a browser, the applet can display and update data values continuously without the need (and associated overhead) to refresh the entire HTML page each time a value changes. With applets, you can monitor changing data values in a much more deterministic fashion.

You can have multiple Live Control connections to different clients simultaneously and Live Control connections may be happening at the same time as normal client/server connections. A publish/subscribe protocol is used for the notification mechanism. The typical sequence of events, at an abstract level, is as follows:

1. The client downloads an HTML page containing a Java applet from the server.
2. The applet starts up in the client browser and establishes a connection to the embedded Live Control server component.
3. The applet code expresses an interest in some WindMarks or variables (subscribes to those WindMarks/variables) on the server, registers that interest with a client registry, and associates UI elements, such as text widgets, with those variables.

   The WindMarks used in the applet must be implemented in your project. For support of legacy applets used with Wind River Web Server projects, symbol table variables are also supported. If your applet uses symbol table variables, they must be declared in the symbol table and enabled for Live Control access by the application.
4. The embedded Live Control server components create a backplane transaction that polls these WindMarks at the frequencies specified in your configuration, and as changes occur, notifies the client applet, which in turn refreshes its display with the current value.

Loading your applets in your HTML code requires no special HTML markup.

> ⚠ Warning: At this time, only one applet is supported per Web page. Using more than one applet per Web page may cause instability in the Web server.

Polling Data Values and Server Push on page 126 When your Live Control applet subscribes to a WindMark or symbol, your two options for monitoring changes in data values are: polling and server push.
Live Control Packages Reference on page 126 Live Control also provides several Java packages to create client connections in your applets:

6.1. Polling Data Values and Server Push

When your Live Control applet subscribes to a WindMark or symbol, your two options for monitoring changes in data values are: polling and server push.

If you specify either the high or medium polling intervals (not specifying an interval is equivalent to medium) when you subscribe to a WindMark, the backplane polls the data value display at the chosen interval and notifies the client when the value changes.

If you specify none as the polling interval, the backplane does not poll the object's value. Instead, you must provide custom WindMark handler code to monitor changes and explicitly notify the client using httpLCNotifyChange( ). For more information, see the API Reference: Kernel Networking.

Parent topic: Applets and Live Control on page 125

Related information
Live Control Packages Reference on page 126

6.2. Live Control Packages Reference

Live Control also provides several Java packages to create client connections in your applets:

http.livecontrol.comm

This package implements the client side of the Live Control communications protocol. Use this package to create client connections to the Web Server.

http.livecontrol.common

This package provides common interfaces for client events.

http.livecontrol.converter

This package provides utilities for converting raw data (byte arrays) to specific datatypes and typed data to raw data.

http.livecontrol.directevents

This package implements the client side of the Live Control WindMark registry, including WindMark read, write, and subscribe services.

http.livecontrol.widgets

This package provides some sample Widgets for display and update of backplane data objects.

Each of these packages is located in your installation at *installDir* **/vxworks-7/pkgs_v2/app/webcli/http-***x.x.x.x* **/src/java/http/livecontrol/**.

Parent topic: Applets and Live Control on page 125

Related information

Polling Data Values and Server Push on page 126

12. ENABLING JAVASCRIPT AUTHENTICATION

1. Introduction

JavaScript authentication fills the security gaps in basic authentication. Using JavaScript authentication, there is no way for an unauthorized person on the line to recover the user's name and password.

When the Web Server encounters a restricted page (any page with an access level other than 0), the server returns a login HTML page instead of the page requested. This login page generates a unique identifier, called a nonce, each time the server accesses it.

When the user enters a name and password, the Web Server encrypts the name, the password, and the nonce, using MD5 one-way hashing. Even if someone knows that MD5 is being used, no data can be extracted from the hashed string.

This hashed string is sent to the server along with the unique nonce. The server then compares the received hashed string to the list of users, which is also hashed using the nonce. If there is a match, the user is approved for the IP address that submitted the request. The Web Server then sends the originally requested page to the user.

JavaScript Authentication sends a cookie once the Web Server approves the user, but the cookie contains only the nonce and the hashed name and password, along with a timestamp. The user's name and password cannot be extracted from the cookie, and the cookie is only approved for the original IP address; thus, another user cannot get access by duplicating the cookie. You can set an expiration time for these cookies in the **Idle Timeout** field of the Web Server Properties dialog box.

In addition to enabling JavaScript authentication with the WEBCLI_JAVASCRIPT_DIGEST_ENABLED VSB option, you must do the following:

- create a user database and access record
- provide an authentication function
- add a custom logon.html file
- set the access level for the Web files in your project
- add the WindMark nonce to your project

The following sections provide step-by-step instructions on how to complete each of these tasks.

2. Providing a User Database

To use JavaScript authentication, you must provide a user database. It can be as simple as a static list of names and passwords, or it can be a call to an authentication service like RADIUS in user-defined code. Wind River recommends that any user database be encrypted in some manner to protect the information.

```
/* User function: returns OK if user name and password are in user database,
else returns ERROR */
static STATUS VD_AUTH_GetPassword(sbyte *pUsername, sbyte *pPassword);
/* User function: returns pointer to access record for this user name, else NULL
if not found */
static WM_ACCESS_T* VD_AUTH_GetAccessLevel(sbyte *pUsername);
```

Following is an example of the simplest possible access record, giving minimal access rights for object reads and writes, which would be the values normally given to any allowed "guest" account. As stated above, this lowest-level user would not have the rights to access the protected content.

```
WM_ACCESS_T usrAccess;
usrAccess.read = 0;
usrAccess.write = 0;
```

3. Providing an Authentication Routine

You must create an authentication function, wmGetAuthenticationLevelPassword( ), to validate the user name and password.

An example authentication function in user-defined code follows:

```
extern WM_ACCESS_T*
wmGetAuthenticationLevelPassword(WMB_SESSION_T *envVar, sbyte *pUserName,
sbyte *pPassword)
{
WM_ACCESS_T *pAccessLevel;
STATUS status;
/* Test the user name and password, return NULL access pointer if no match */
pAccessLevel = NULL;
status = VD_AUTH_GetPassword( pUserName, pPassword );
    if ( OK > status )
        return pAccessLevel;
/* Return pointer to access record for this validated user */
pAccessLevel = VD_AUTH_GetAccessLevel( pUserName );
    return pAccessLevel;
}
```

This example authentication function depends on two user supplied functions: one to test the user name and password with the user's database, and a second that returns a pointer to the respective access structure for the user name.

Note: Before you can use JavaScript authentication, you must enable the WEBCLI_JAVASCRIPT_DIGEST_ENABLED VSB option and supply the custom logon HTML file.

4. Adding a Customized logon.html HTML Page

logon.html is the special login page used by JavaScript Authentication.

About This Task

If you are using basic authentication, users do not see this page when requesting access controlled pages. You must create your own customized **logon.html** HTML page and add it to your project.

Procedure

1. Create a customized **logon.html** page.

   A sample **logon.html** page is located in the following directory

   installDir **/vxworks-7/pkgs_v2/app/webcli/http-**x.x.x.x **/src/java/**

WNDRVR

2. Add **logon.html** to your registered file system.

5. Setting the Access Level for Web Files in Your Project

You must set the access levels for the Web files in your project you would like to protect.

About This Task

If you do not set the access levels, the login screen will not be invoked, the user's access level will not be changed, and the page's contents will be displayed.

For more information on access control, see Implementing Access Control on page 88.

Procedure

1. From the left pane of the MCE, select the desired file.
2. In the Web File Properties dialog box, select **File Registry Enable**
3. Use the **Read Access** and **Write Access** fields to set the access control options.

6. Adding the WindMark nonce to Your Project

The logon page generates a unique identifier, called a nonce, each time it is accessed.

About This Task

You must create a WindMark called *nonce* that represents the device status string.

To create the WindMark, do the following:

Procedure

1. Right-click **Backplane** in the left pane of the MCE and select **New Child > WindMark**.

   The WindMark Properties dialog box appears in the right pane of the editor.

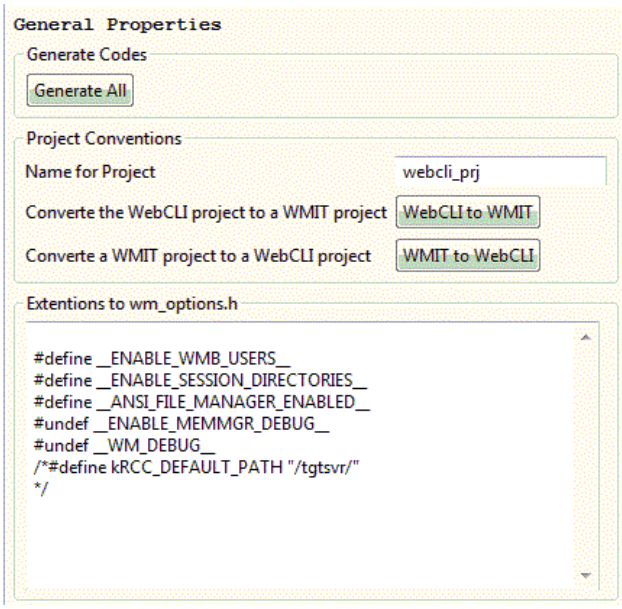
2. In the **Name** field, type **nonce**.
3. In the Type section, do the following:
   1. Select **string** from the **Type** drop-down box.
   2. Select **Sub Range**.
   3. Type **32** in the **Max Length** field.
4. In the Permissions section, do the following:
   1. Select **Enable readAccess**.
   2. Set the **Read Access Value** to **0**.
5. In the Handler section, do the following:
   1. From the **Type** drop-down men, select **External API**.
   2. In the **Handler** field, type the name of your handler, for example, wmbGetNonceHandler.
6. Save **config.webcli** by selecting **File > Save**, or by pressing **CTRL+S**.

13. THE MCE INTERFACE REFERENCE

1. Global Options Reference

Options from the General dialog box apply to the global project.

- The Generate All button creates the code for your project.
- The conversion options are for compatibility purposes only.
  The Extensions to wm_options.h panel allows you to add file and directory extensions options and basic debugging options.



For more information, see Configuring the Project-Level Settings on page 59.

2. The Backplane Properties Dialog Box

The Backplane Properties dialog box allows you to configure the backplane for use with your project.

The following sections provide details on the available options.

Backplane Properties

Name for this backplane	publicWMB
Deregistration Handler	NULL
Deregistration Cookie	NULL
Initial Max Resources	1000
Initial Max Components	14

For more information about the backplane, see Understanding the Backplane on page 12.

For information on creating the backplane, see Configuring the Backplane on page 61.

Basic Backplane Options

Instantiate a Wind River Management Backplane

Select this option if you want to instantiate a backplane within this project. In a flat memory space embedded OS, the typical model is to have one backplane, which all components register with—either statically at compile time, or dynamically at run-time. While there is nothing that prevents you from using multiple backplanes in a single memory space, doing so would come at the expense of efficiency and memory.

In a multi memory-space embedded OS, the typical configuration is to have a backplane in each memory space where a component exists. In this case, one backplane must be identified as the parent backplane, and other backplanes will communicate with it. In such a scenario, the backplanes handle all of the IPC for you.

Name for this backplane

Type an appropriate name for the backplane.

Deregistration Handler

Backplane deregistration handler.

Deregistration Cookie

This value is passed to the deregistration function. It is set to NULL by default, but can be a token value to help identify this particular component with a shared component deregistration function.

Initial Max Resources

Set the initial limit of WindMark resources that can register with the backplane instantiated in this project.

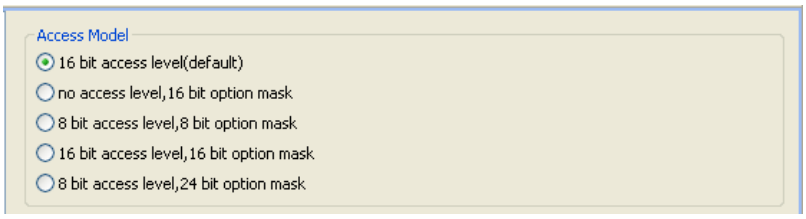
Initial Max Components

Set the initial limit of components that can register with the backplane instantiated in this project.

3. Access Model Dialog Box Reference

The Access Model dialog box allows you to set the access option labels for the access option bit-flags.

Values are automatically assigned by the code generation. However, these values can be overridden by the user either with numerical value or #define values.



Bitmap Access Control

Each WindMark has the option for restricting access based on an access level assigned to that WindMark. In order to have access to the resource, the user must have an access level greater or equal to that of the resource. The default access level for resources—which is specified in the WindMark Editor—is 0 (no restrictions).

The access option bit-mask model enables access to resources to be based on options installed in the device versus options allowed to the user. In order to have access, the user must have at least one access level bit common with the resource. The default access option flag for resources is the special case of the value zero (no restrictions).

These two access methods are internally combined into a single access value during code generation.

Setting Access Control

Access groups for the access option bit-flags are entered in the Access Group dialog box.

Values are automatically assigned by the code generation. However, these values can be overridden by the user either with numerical value or #define values.

Access Levels in Code Generation

In this example, if the access group list is as follows:

- Guest
- Admin
- User

The following is generated in **wmb_projectName.h**:

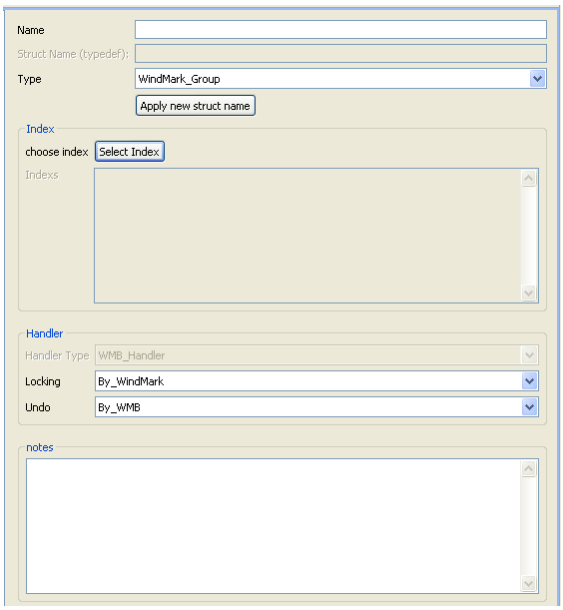
```
#define ENUM_ACCESS_GUEST (1 << __RLI_ACCESS_LEVEL_SHIFT__)
#define ENUM_ACCESS_AMDIN (ENUM_ACCESS_GUEST << 1)
#define ENUM_ACCESS_USER (ENUM_ACCESS_AMDIN << 1)
```

4. WindMark Table Dialog Reference

The WindMark Table dialog box allows you to create and configure WindMark tables.

A WindMark table is a higher level object that contains WindMarks. Such objects include the following:

- C data structures
- groups, which are collections of WindMarks that share a relationship, such as a dependency or a common handler



The following sections describe the available options.

General Options

Name

Type a name for your table.

Struct Name (typedef)

When you create a table of C data structures, specify the structure (or its typedef name) as the parent table.

Type

WindMark_Group

A group of WindMarks is a grouping of WindMarks that share a common handler and are not otherwise part of a table.

C_Structure

C data structures are treated as tables.

Apply new struct name

When you create C data structure tables, the WindMarks that represent the structure members must use the **structureAddress** storage cookie type. When you click **Apply** new struct name, the storage cookie type for all WindMark members in the group is set to **structureAddress** and the data cookie value is set to the structure offset. The structure offset is computed by the **WMB_OFFSET(typedef.field)** macro that allows support for structured tables.

Index Options

Click Select Index to add your WindMarks to the index for this table. The WindMarks included in the index appear in the Index field.

Handler Options

Handler Type

Select the handler type. By default, the MCE generates backplane handlers. It can also generate wrappers for legacy RapidControl 3.x handlers, connect the WindMark to the MIBway handler, create wrappers for newer MAPI handlers, or allow External APIs for variables or functions which are accessed through the backplane which are not WindMarks. If more than one variable or function is set as an External API, you must provide the code for selecting the proper variable or function.

Locking

Select this option to cause the MCE to insert the appropriate locking associations into the handler code. Locking may be done at several levels—by the WindMark, by the WindMark group, at the producer level, or at the user level. Whenever any locking is done, all the parents of the chosen level are locked automatically. This prevents deleting the producer when a WindMark that is part of that producer is being accessed.

By_WindMark

Select this option to lock only the WindMark.

By_WindMark_Group

Select this option to lock the WindMarks in the WindMark group while any WindMark in the group is being accessed.

By_Producer

Select this option to lock the entire WindMark repository (producer) while any WindMark or WindMark group within the repository is being accessed.

By_User_or_None

The customer–developer must provide the locking.

Undo

Select this option to cause the MCE to insert the appropriate undo associations into the handler code.

Notes

In this field you can enter notes. The text typed here is stored in the project file and not added to the generated code.

5. WindMark Properties Dialog Reference

The WindMark Properties dialog box allows you to create and configure WindMarks for use with your project.

The following sections provide details on the available options.

For information on creating WindMarks, see WindMarks in HTML Form Elements on page 173.

General WindMark Options

Name

Type the name value of the WindMark. This is a required value. The name can consist of the following characters: a-z, A-Z, 0-9, dash, underscore, and colon. Also, the first character of a WindMark name cannot be a number.

WNDRVR

Alternate

Type an alternate name value if desired. Using an alternate name causes the backplane to look up and use the alternate name in place of the regular name. This is useful for deprecated WindMarks—for example, RFC 1213 objects that have been superseded by RFC 2233 objects—where the physical element is now embodied in another WindMark.

Type Options

Type

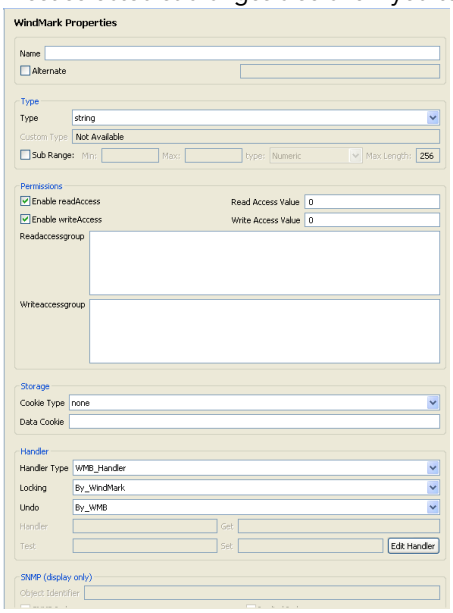
Select the WindMark's data type from the drop-down menu.

Sub Range

Select this option to further restrict the selected data type. The available options change for each WindMark type.

For example, if you select type string, then select Sub Range, you can select subranges such as Alpha, or Display 32-127 (which excludes control characters). You can also specify the maximum length of the string.

Most selected subranges also allow you to edit minimum and maximum values as appropriate to that type.



Permissions Options

Use these options to set the read and write access option level for a particular WindMark. You can also select read and write access groups that you have already defined.

For more information on creating access groups, see Adding Access Control to a WindMark on page 73.

Storage Options

Cookie Type

The data storage cookie type (if used) and the data cookie name. This setting inserts the proper variable name into the calls to the object action handler functions.

None

This option does not allow for any generation of a variable into the handler code segment. No value is allowed. A place holder is generated in the code. You must alter the generated code.

absoluteAddress

Type the address of the variable. The variable name may be preceded by an &, to designate that it is an address. Unless the choice is made to alter the handler, there is no need for modification by the customer–developer beyond initializing the variable.

SNMP_OID

The SNMP OID of this object.

structureAddress

The address of a C data structure. When you create C data structure tables, the WindMarks that represent the structure members must use the structure address storage cookie type. For more information, see WindMark Table Dialog Reference on page 133.

Other

Unlike Absolute Address, which implies that this option supplies a direct and valid pointer to the object in question, other makes no assumptions. This can be a local structure pointer, a MALLOC pointer, or a data cookie depending on the choice of the customer–developer. A place holder is generated in the code. You must alter the generated code.

Data Cookie

The data cookie is the actual reference to the object. For example, if the data is a string variable myString[32], the cookie type would be absolute address, and the data cookie would be myString—the address of the variable.

Handler Options

Type

Select the handler type. By default, the MCE generates backplane handlers. It can also generate wrappers for legacy RapidControl 3.x handlers, connect the WindMark to the MIBway handler, create wrappers for newer MAPI handlers, or allow External APIs for variables or functions which are accessed through the backplane which are not WindMarks. If more than one variable or function is set as an External API, you must provide the code for selecting the proper variable or function.

Locking

Select this option to cause the MCE to insert the appropriate locking associations into the handler code. Locking may be done at several levels—by the WindMark, by the WindMark group, or at the producer level. Whenever any locking is done, all the parents of the chosen level are locked automatically. This prevents deleting the producer when a WindMark that is part of that producer is being accessed.

By_WindMark

Select this option to lock only the WindMark.

By_WindMark_Group

Select this option to lock the WindMarks in the WindMark group while any WindMark in the group is being accessed.

By_Producer

Select this option to lock the entire WindMark repository (producer) while any WindMark or WindMark group within the repository is being accessed.

By_User_or_None

The customer–developer must provide the locking.

Undo

Select this option to cause the MCE to insert the appropriate undo associations into the handler code.

Handler

The name/address of the handler. This option is only available for handlers of type External_API.

Get

The name/address of the Get function. This option is only available for handlers of type **RCB1_Wrapper**.

Test

The name/address of the Test function. This option is only available for handlers of type **RCB1_Wrapper**.

Set

The name/address of the Set function. This option is only available for handlers of type **RCB1_Wrapper**.

Edit Handler

When you click **Edit Handler**, the generated file **wmb_projName.c** opens in a text editor. You can add code that is preserved between code regenerations.

Note: Before you can edit your WindMark handlers, you must build your project files. To build your project files, right-click your project and select **Build Project**.

SNMP Options

The SNMP section of the WindMark Properties dialog box is display only; the settings cannot be changed. For more information on SNMP WindMarks, see *Creating MIBway (SNMP) WindMarks*.

Object Identifier

This field displays the SNMP Object Identifier (OID) for a MIBway WindMark.

SNMP Index

Defines whether this WindMark is an SNMP index.

An SNMP table is an ordered collection of objects consisting of zero or more rows. Each row may contain one or more objects. Each object in a table is identified using the table index.

Implied Index

If this WindMark is an SNMP index, it is a defined index.

Using an implied index is optional. An implied index can be used for SNMPv2 tables for varying string length objects which have rules for ordering columns based on string length.

Snmp Read Access

Defines whether the WindMark has read access.

SnmpWrite Access

Defines whether the WindMark has write access.

Snmp Table

Defines whether the WindMark is an SNMP table.

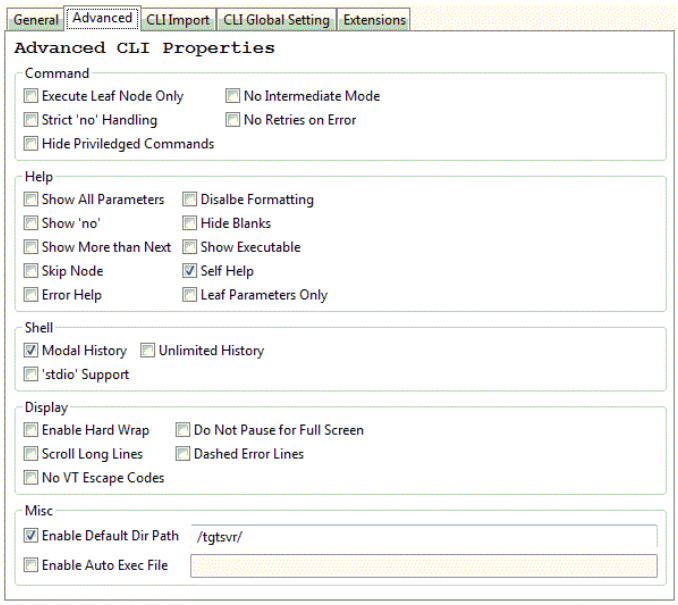
Belongs to table

If the WindMark belongs to an SNMP table, the name of the table is displayed in this field.

6. General CLI Properties Dialog Reference

The General CLI Properties dialog box allows you to configure the basic CLI properties for your project.

The following sections provide details on the available options.



For information on configuring CLI, see Configuring the CLI on page 61.

For information on advanced CLI properties, see Advanced CLI Properties Dialog Reference on page 142.

Connection Type Options

Telnet Only

Select this option to enable a connection from a telnet client to a telnet server.

SSH Only

Select this option to enable a connection to a telnet client through a secure shell.

Telent + SSH

Select this option to enable both telnet and secure shell connections as described above.

Telnet Options

Port Number

Specify the TCP port on which the telnet daemon listens for traffic. Normal telnet server operation listens on port 23. If you want to change this number, consult RFC 3232, the RFC that covers assigned numbers, to make sure your telnet server port does not collide with some other commonly pre–assigned number (for example, SMTP). The named constant **kRCC_FIXED_PORT** is #defined to whatever number you type in. Setting the Port Number to zero (0), disables the telnet server.

Thread Stack Size

Set the value for the named constant **kRCC_THREAD_STACK_SIZE** for target operating systems which require it, such as VxWorks.

Server Priority

Set the value for the server priority. This value delimits the priority for the telnet daemon thread. The named constant **kRCC_SERVER_PRIO** is #defined by this field.

Timeout

Specify the amount of time, in seconds, that the server waits to accept command data. The server waits for the specified amount of time and then publishes the following message:

```
This terminal has been idle for XX minutes. It will be logged out if it
remains idle for another XX minutes.
```

Note:

The value you provide in this field is in seconds, even though the timeout mechanism displays information in minutes. The CLI engine handles the conversion.

After the message is displayed, the server waits for another block of this specified time and drops the connection. The net result is that this value describes half of the amount of time between when the console goes idle, and when the connection is dropped. The named constant **kRCC_TIMEOUT** is #defined by this setting.

Max Connections

Specify the maximum number of simultaneous users that can access the server at a given time, including both serial and network session users. If you want your system to be managed by 100 simultaneous users, then you need to modify this setting as necessary. Keep in mind, though, that all connection/task information is statically allocated at server boot time; therefore, the number you specify here can have a serious impact on the total amount of volatile memory used by the CLI server. The named constant **kRCC_MAX_CLI_TASK** is #defined by the value in this field.

Enable serial console

Select this option to set up a specific task to manage a CLI session conducted over the target's serial port. If you enable this option, you must also do the following:

- Disable the kernel shell so that there is no conflict with the CLI serial console. To do this, remove the **INCLUDE_SHELL** from your VIP project.
- Ensure that the **kRCC_SERIAL_PORT** and **kRCC_SERIAL_SPEED** serial port configurations, from **rcc_telnetd.c**, match the terminal emulator/device port being used. For example:

```
#ifndef kRCC_SERIAL_PORT
#define kRCC_SERIAL_PORT "/ttyS0"
#endif
#ifndef kRCC_SERIAL_SPEED
```


WNDRVR

```
#define kRCC_SERIAL_SPEED 9600
#endif
```

Login Options

Login Length

Specify the buffer size used by the CLI server to receive the name of the person attempting to log into the system. The named constant **kRCC_MAX_LOGIN_LEN** is #defined to the value listed here.

Login Prompt

Type the text of the login prompt in this field. The login prompt is what users see when they connect to the device. The reply to this prompt is the users login name.

Password Length

Specify the maximum password length. Setting this value enumerates the buffer size used by the CLI server to receive the password of the given user. The named constant **kRCC_MAX_PASSWORD_LEN** is #defined by this value.

Password Prompt

Type the text of the password prompt your users see after they enter their login name.

Max Login Attempts

Specify the maximum number of times a person can attempt to log in to the embedded telnet server before the server drops the connection. The named constant kRCC_MAX_LOGIN_ATTEMPTS is #defined to the value in this field.

User Authentication Callback

Select this option to use a customized user authentication callback function.

User Callback

Type your customized authentication callback function name. For the prototype of the authentication function, see the reference entry for **RCC_TASK_ValidateLogin( )**.

Command Options

Prompt Length

Specify the buffer length for the prompt string. If you plan on having a lot of intermediate nodes, or if you have a long default prompt string, then you should increase this beyond 16. The named constant **kRCC_MAX_PROMPT_LEN** is #defined by the value in this field.

Command Length

Specify the buffer length for the entire command string, as entered by the user. That is, the core CLI code creates a buffer of the specified length, and the entire command string is copied into it as the user types. If you are expecting long command strings—if you have commands with a lot of parameters, or with parameters that have long names—then make this string bigger. The named constant **kRCC_MAX_CMD_LEN** is defined by the value here.

Output Buffer Size

Define the length of the output buffer, as used by the generated code. The named constant **kOUTPUT_BUFFER_SIZE** is #defined by the value here, which should be the same length as the WindMark substitution buffer size.

WNDRVR

The WindMark handler functions that are generated by the MCE (such as the handler functions for the show command) make use of a backplane abstraction function. This backplane abstraction function requires an output buffer into which the data can be copied.

Parameter Ordering

A command can be set up so that the parameters can be entered in any order as name and value (None), as a value only in the proper order (Unnamed), or as name and value in the proper order (Full).

Parameter Ordering must be set to Full to enable scripting support and stdio.h support. There is a sample command named system info you can use to view system information, including parameter ordering. This command is located in *installDir* **/ vxworks-7/pkgs_v2/app/webcli/cli-**x.x.x.x **/src/examples/debug.rcp**.

For more information on scripting and a listing of the commands in **scripting.rcp**, see Enabling Scripting Support on page 102.

Shell Options

Default Prompt

Type the text of the default prompt in this field. This is the default prompt that your user sees upon successfully logging into your device. The named constant **kRCC_DEFAULT_PROMPT** is set to the value specified in this field.

Prompt Tail

Type the text of the prompt tail. This value is always appended to prompt. The named constant **kRCC_DEFAULT_PROMPT_TAIL** is set to the value specified in this field.

History Buffer

Specify the number of prior commands to save in the history buffer as used by each shell, which a user can display by typing history. Furthermore, a user can reinvoke a previous command simply by typing ! and the command number as listed in the history buffer, such as !3 or !4.

Alias Buffer Size

Specify the size of the buffer (in bytes) that stores the aliases. An alias is a way to shorten the commands you have to type into the interface. For further details, see the command descriptions in the VxWorks 7 Creating a CLI Tool Tutorial. This buffer is shared by alias names and their replacement text.

Max Aliases

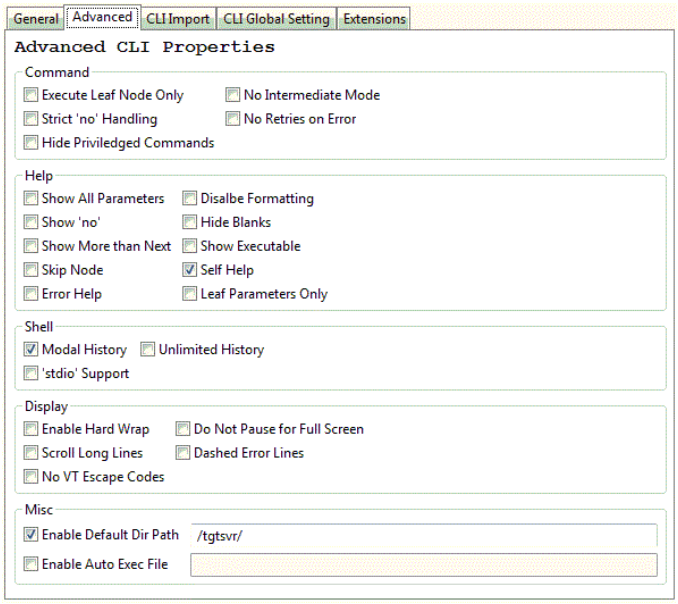
Specify the maximum number of aliases allowed.

7. Advanced CLI Properties Dialog Reference

The Advanced CLI Properties dialog box allows you to set advance CLI command properties.

Following are the available options.

WNDRVR



Command Options

Execute Leaf Node Only

Select this option to cause the command interpreter to execute only the handler for the last command node in the in command chain.

No Intermediate Mode

Select this option to prevent operation of intermediate mode commands. For more information on intermediate mode, see Understanding Intermediate Mode Commands on page 89

Strict 'no' Handling

Select this option to only allow no commands when the command's handler is explicitly created with support for no forms.

No Retries on Error

Ordinarily, when you enter a bad command, the CLI engine echoes the bad command on the screen with the cursor at the point where the command produced the error. Select this option to disable the echoing of a failed command.

Hide Privileged Commands

Select this option to hide commands from users with insufficient read access permissions.

Help Options

Show ALL Parameters

Select this option to show all available parameters along with their defined help strings.

Disable Formatting

Select this option to disable display formatting of help text (such as indents).

Show 'no'

Select this option to cause the help system to print help for no forms of commands you have configured to support no forms.

WNDRVR

Hide Blanks

Select this option to prevent the printing of blank lines for command nodes and parameters with empty help strings.

Show More than Next

Select this option to cause the help system to display more than only the next legal parameter.

Show Executable

Select this option to cause the help system to list the possible legal command nodes.

Skip Node

Select this option to cause the help system to print help for child nodes and parameters (and not for the current node) when a user enters ? at the command prompt.

Self Help

Select this option to cause the help system to print help for the current node as well as the child nodes and parameters when a user enters ? at the command prompt.

Error Help

Select this option to cause the help system to automatically display help for a command when the command fails.

Leaf Parameters Only

Select this option to cause the help system to only display help for leaf parameters.

Shell Options

Modal History

Select this option to create a separate history buffer for each intermediate command mode.

Unlimited History

Select this option to store command history up to the limit of the history buffer.

'stdio' support

Select this option to enable IO redirection and pipes.

Display Options

Enable HardWrap

Select this option to wrap the output display at a fixed, specified width, rather than letting the client console window wrap the text.

Do Not Pause for Full Screen

Ordinarily when output exceeds the length of the visible client shell window, the CLI engine pauses and waits for a user command to page down to the next screen (similar to the UNIX more command). Select this option to prevent the client shell from paging output.

Scroll Long Lines

Select this option to cause the client shell to scroll when entered text wraps to the next line.

WNDRVR

Dashed Error Lines

Select this option to display dashes instead of spaces before the ^, which points to the point of the failure.

No VT Escape Codes

Select this option to disable cursor control using VT escape codes.

Miscellaneous Options

Enable Default Dir Path

If your commands need to access files stored on a local target file system, select this option and enter the target file system path to use as the default.

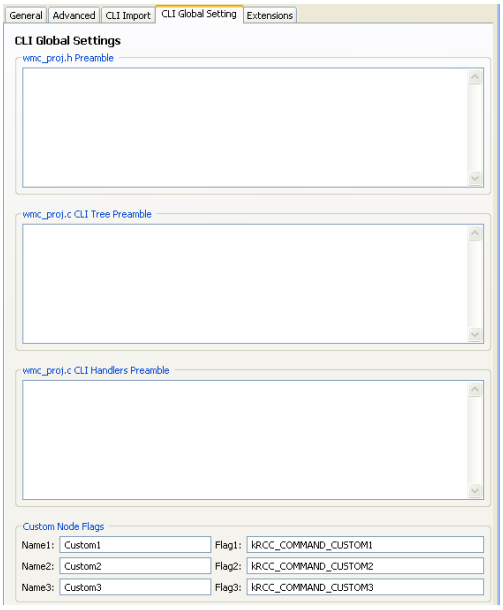
Enable Auto Exec File

If you want your CLI system to auto execute a script file, select this option and enter the file name here.

8. CLI Global Settings Dialog

The CLI Global Settings dialog box allows you to enter code that will be inserted at the top of the command files.

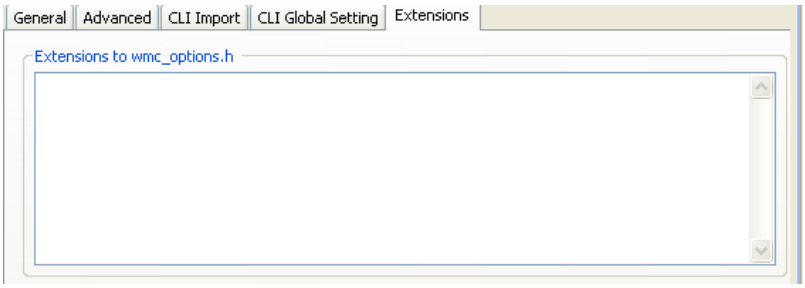
For example, if you want to add a #define to **wmc_projectName.**h, type the define in the **wmc_proj.h** Preamble text box, and the #define is inserted at the top of **wmc_projectName.h** every time it is generated by the MCE.



9. CLI Extensions Dialog

The CLI Extensions dialog box allows you to add your own code to the generated configuration files.

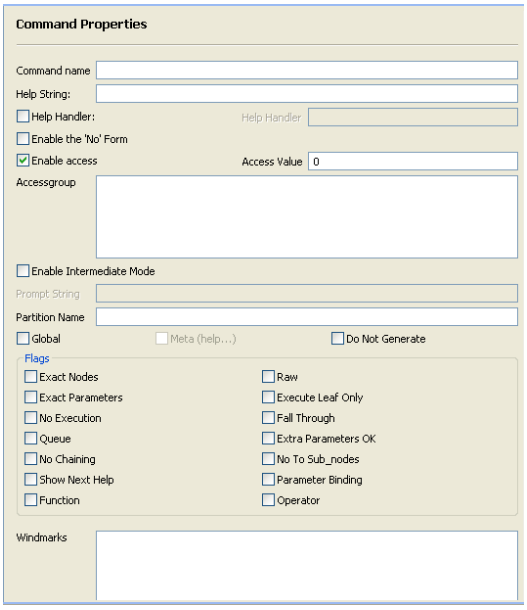
The custom code you enter in CLI extensions gets added to the generated file **wmc_options.h**.



10. Command Properties Reference

The Command Properties dialog box allows you to create command nodes and edit its basic properties.

The following sections provide details on the available options.



For information on creating command nodes, see Creating and Removing Command Nodes on page 92.

Command Options

Command Name

Type an appropriate name for your command in this field.

> Note: When naming your commands, keep in mind that command names cannot include spaces, and there are reserved keywords and characters that you should not use. For more information, see Reserved Keywords on page 84.

Help String

Type the text you want displayed when users request help for this command.

For information on using help as part of the CLI tool tutorial, see the VxWorks 7 Creating a CLI Tool Tutorial.

Help Handler

Instead of a help string, you can use a help handler to provide help. A help handler is useful if you want to display different help strings under different conditions. Select Help Handler and type the name of the help handler function (which you must provide). For more information, see Creating a Help Handler Routine on page 96.

Note: You cannot use both a help string and a help handler. You must use one or the other.

Enable the 'No' Form

Select this option if you would like to use the "no" form of your command. For more information on command "no" forms, see Understanding Command No Forms on page 88.

Enable Access

This option allows you to assign an access level to your command node. Using command-level access control, you can restrict access to individual commands.

For more information on access control, see Implementing Access Control on page 88.

Access Value

Type the desired access level in this field.

Accessgroup

This feature is currently not supported.

Enable Intermediate Mode

Select this option to enable intermediate mode support. Intermediate mode support allows the user to enter partial commands. For more information on intermediate mode commands, see Understanding Intermediate Mode Commands on page 89.

Prompt String

Type the desired prompt string value for the intermediate mode.

Partition Name

This feature is currently not supported.

Global

Select this option to set the command as a global command. The command will operate even when the user is in an intermediate mode.

Meta (help...)

Select this option if you want your command to be a meta command. Meta commands are commands that act on other commands. For more information on meta commands, see Using Meta Commands on page 91.

Do Not Generate

This feature is currently not supported.

Flags

Exact Nodes/Exact Parameters

Select these options to specify that the token entered on the command line must exactly match the node or parameter as it is defined.

WNDRVR

The CLI parser normally allows extra parameters and/or nodes if it finds an unambiguous match for the partial command. Setting this flag forces exact compliance with the registered syntax, and all optional nodes or parameters must be fully defined by the designer, or the array parameter must be used to group the extra entries for any variable command.

Raw

Do not look up the variable values to be used with the handler.

Execute Leaf Only

Only execute the final leaf, or final item, in the command chain.

No Execution

Do not execute any handlers associated with this command node. The handlers for any children nodes will still be executed.

> Note: The No Execution option should be used to assist in command tree development only. If you want to block execution at run time, use the access control feature for the desired nodes and parameters.

Fallthrough

When searching through the list of existing nodes, if the current node is not a match, then check the child nodes under it. This allows grouping of commands or the existence of an optional prefix command.

Queue

Use a queuing mechanism to ensure that only one task (telnet process) at a time is executing this handler.

Extra Parameters OK

Allow extra parameters to be entered on the command line.

No Chaining

Any child nodes under the top-level node are not allowed.

If you create a command node with intermediate mode selected, No Chaining forces you to always go into intermediate mode for the specific command before any subcommands are accepted.

No To Sub_nodes

If a 'no' was given to this command node, then all of the subnodes also inherit the 'no' directive.

Show Next Help

Show next help, even if HELP_FLAG_NEXT or kRCC_COMMAND_FALLTHROUGH are not set.

Parameter Binding

Treat the current node as a leaf node. In other words, bind it as a parameter.

Function

Like in programs, the command is a function. It is used as part of a larger command. The output of the current command is used as a parameter to another command. Used with commands with the Operator flag set.

Operator

Perform operations on information put onto a stack by commands with the **Function** flag set.

Windmarks

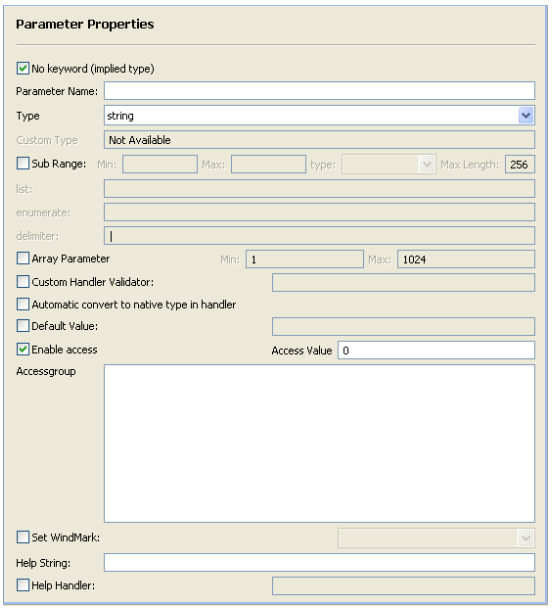
Select the WindMarks you want to associate with this command.

WNDRVR

11. Parameter Properties Dialog Reference

The Parameter Properties dialog box allows you to create and customize the parameters used by a command.

Following are the available options.



For information on creating parameters, see Creating and Editing Parameters on page 93.

No keyword (implied type)

Select this option to set your parameter as an implied type. If a parameter has a name, you must type the name of the parameter and the value. However, with an unnamed parameter, or an implied type, you type the value only.

The difference between a parameter with a name versus one without a name is best shown through an example. Suppose you have the following command:

```
reboot timeout 5
```

This line has a command of reboot, a parameter of timeout, and a value for that parameter of 5. Alternatively, consider the following command:

```
reboot 5
```

where 5 is an implicit timeout parameter—you do not need to specify timeout. The value 5, which follows reboot, is just assumed to be the timeout.

Parameter Name

Type the name you want to represent the parameter in the command tree and that will be typed in the console to invoke this parameter.

Parameter Type

Select the data type of the parameter from the drop-down list.

Sub Range

Use these fields to limit the valid values of the parameter to a subset of the default range for the data type. These fields change for each Parameter Type.

Array Parameter

This option makes the parameter an array type. Type the desired values in the **Min** and **Max** fields to indicate the minimum and maximum index entries for the array.

Custom Handler Validator

Following is a simple example of creating a custom parameter validator for data strings:

```
{
RLSTATUS status = OK;
    int month,day,year;
    /* Call the RC built-in validator for default parameter validation */
    /* In this simple case, it will just test the string length */
    if (OK > (status = CONVERT_Validate(pString,pTypeInfo)))
        return(status);
    sscanf(pString,"%d/%d/%d",&month,&day,&year);
    if ((month < 1) || (month > 12)) return ERROR_GENERAL_OUT_OF_RANGE;
    if ((day < 1) || (day > 31)) return ERROR_GENERAL_OUT_OF_RANGE;
    /* Test the different format ranges for the year */
    if ((year >= 1998) && (year <= 1999)) status = OK;
        else if ((year >= 2000) && (year <= 2019)) status = OK;
        else if ((year >= 98) && (year <= 99)) status = OK;
        else if ((year >= 00) && (year <= 19)) status = OK;
        else status = ERROR_GENERAL_OUT_OF_RANGE;
return(status);
}
```

The CLI backplane calls CONVERT_Validate( ), which then directly passes control to this custom validation function. The user can optionally call CONVERT_Validate( ) at the top of the custom validation function to do standard CLI validation (specifically, the parameter's data type and any subranges that the user specified in the MCE dialog box) before going on to specific user validation code.

Note: The **pTypeInfo** record passed in was adjusted by CONVERT_Validate( ), such that the subsequent call to CONVERT_Validate( ) inside this function does not cause an infinite loop.

Automatic convert to native type in handler

Select this option if you want Wind River CLI code generation to convert parameters from their string value to their native value. This allows enumerated values to be converted to numerical values in the handler.

Default Value

Select this option if you want the parameter to have a default value. Type the value in the **Default Value** field. The default value is the assumed value if it is not explicitly stated at the command prompt.

Enable Access

Select this option to enable parameter-level access control. Using parameter-level access control, you can create commands that have different options for different users. If a user does not have the appropriate access level to access a particular parameter, that parameter is not available to that user.

Access Value

Type the desired access level in this field.

WNDRVR

Accessgroup

This feature is currently not supported.

Set WindMark

Select this option if you wish to a associate a WindMark with this parameter. The drop-down list shows all the available WindMarks. The generated code includes code in the handler to set the WindMark with parameter data.

Help String

Type the text you want displayed when users request help for this parameter.

Help Handler

Instead of a help string, you can use a help handler to provide help. A help handler is useful if you want to display different help strings under different conditions. Select Help Handler and type the name of the help handler function (which you must provide). For more information, see Creating a Help Handler Routine on page 96.

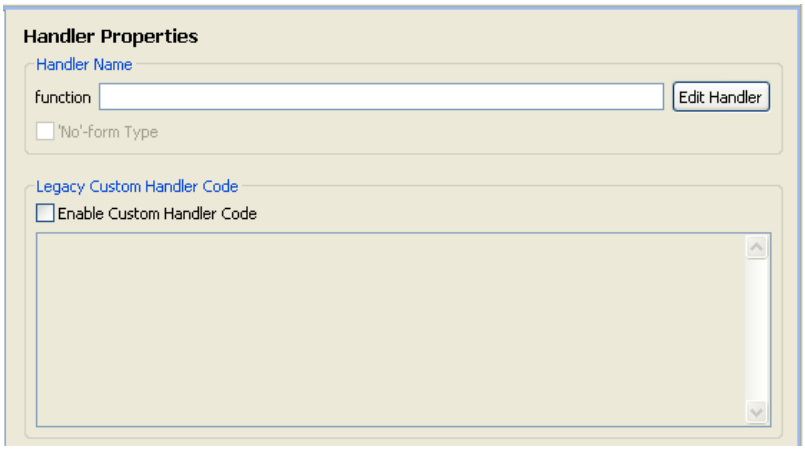
Note: You cannot use both a help string and a help handler. You must use one or the other.

12. Handler Properties Dialog Reference

The Handler Properties dialog box allows you to create command handlers and configure the features that can be associated with them.

Following are the available options.

For information on creating command handlers, see 12.5 Creating and Editing Command Handlers, p.115.



Function

Enter a name for your function handler.

Edit Handler

When you click Edit Handler, the generated file **wmc_projName.c** opens in a text editor. You can now add code that is preserved between code regenerations. Add your code between the beginning and end of the following code:

```
/* %%BeginUserCode myNewHandler_HandlerBody */
    /* TO DO: Add your handler code here */
/* %%EndUserCode myNewHandler_HandlerBody */
```


WNDRVR

> **Note:** Before you can edit your handler, you must build your project files. To build your project files, right-click your project and select **Build Project**.

'No'-form Type

Select this option if you want to enable the no form of the handler. If a node has two or more handlers, you must specify which (if any) is the no form. For more information, see Understanding Command No Forms on page 88.

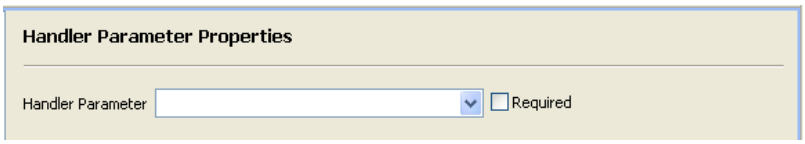
Enable Custom Handler Code

The **Enable Custom Handler Code** option allows you to keep custom handlers in the **rcc_handler.c** file without losing changes when you regenerate this command file. By default, Enable Custom Handler Code is not selected. If you select this option, the custom code box is not prefilled with the standard code for the handler as it is currently configured (with its parameters and WindMarks). Input your own handler code.

13. Handler Parameters Properties Dialog Reference

The Handler Parameters Properties dialog box allows you to select what parameters are associated with your handler, and if they are optional or required.

Following are the available options.



For information on creating handler parameters, see Creating Handler Parameters on page 95.

Handle Parameter

Select the desired parameter from the list of available parameters in the **Handler Parameter** drop-down list.

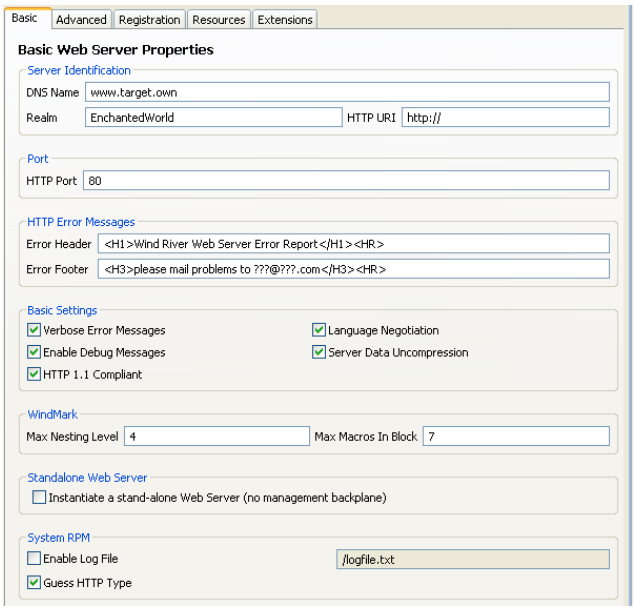
Required

Select this option to add the parameter as a required handler parameter.

14. Basic Web Server Properties Reference

The Basic Web Server Properties dialog box allows you to set the basic Web Server properties.

The following sections provide details on the available options.



Server Identification Options

DNS Name

This is an optional setting. Values typed in this field should follow the format www.hostname.com. Do not include the root URI (for example, http://) or a trailing slash "/".

Realm

This is an optional setting. The realm value is typically used in the challenge:response scheme associated with basic (base64) HTTP server authentication. Basic authentication is considered to be a nonsecure, minimal level of access control. (For a standards-based discussion of basic and digest authentication approaches, see RFC 2617, at http://www.ietf.org.)

HTTP URI

Set the root URI segment of the HTTP uniform resource identifier (URI) for the server. This is typically set to represent the protocol used. Common values for this field are http://, for standard HTTP servers, and https://, for secure socket layer (SSL) encrypted servers.

Port Options

HTTP Port

Type the port number the HTTP daemon monitors for incoming HTTP requests. Port 80 is the well known port for HTTP messages. In most cases, 80 should be the server port number value. You can change the port value to any other port number not in use by another process (and typically not another well known port). The consequence of changing the port setting is that URLs typed into a browser will probably require you to specify the port number in the URL; for example **http://www.exampledevice.com:8080/**.

⚠ Warning:	Only clear the selection for HTTP Port when you intend to build a server that only accepts secure (SSL) connections using the URI scheme https://.

HTTP Error Messages Options

Enter the HTML header and footer for HTTP error messages (for example, 404 - file not found). The content of non-HTTP error messages is determined by the user implemented functions httpPostReply( ), httpWindMarkGetFailed( ), and httpWindMarkSetFailed( ).

Basic Settings Options

Verbose Error Messages

Select this option to send detailed run-time debug messages to the console.

Enable Debug Messages

Select this option to send run-time debug messages to the console.

HTTP 1.1 Compliant

Select this option to enable HTTP 1.1 header processing. It is enabled by default, and is required for HTTP 1.1 compliance. You should leave this option enabled in all cases except those where you need the smallest server size you can obtain and can afford to omit the functionality.

Language Negotiation

Select this option to cause the embedded HTTP server to parse inbound HTTP request headers for the Accept-Language token. This feature is useful when you intend to provide localized sets of HTML files. When a browser requests a file in a localized language that you provide, it sends the requested page in the appropriate language. To use this feature, you must provide localized equivalents for all of your HTML files. For more information, see RFC 2616.

Server Data Uncompression

Select this option to cause the embedded HTTP server to uncompress requests whose Content-Encoding values are gzip, compress, or deflate. For more information, see RFC 2616.

WindMark Options

Max Nesting Level

Specify the maximum nesting depth for the **IF** and **REPEAT** macros. Set this value greater than or equal to the greatest nesting depth you use in your Web content files.

Max Macros in Block

Specify the maximum number of **IF** and **REPEAT** macros that can appear in a block. Set this value greater than or equal the highest number of macros used in a single macro block in your Web content files.

For more information on the IF and REPEAT macros, see Special WindMarks and WindMark Macros on page 179.

Standalone Web Server Options

Instantiate a stand-alone Web Server (no management backplane)

Select this option if your project is a standalone Web Server project; that is, a project without backplane, CLI, or SNMP components.

System RPM Options

Enable Log File

Select this option to enable request logging. Enter a fully qualified target path and filename where you want to create the log file.

When you regenerate the config files, they include the necessary calls to add the request logging RPM and configure it to write a log file to the path and filename you entered. You must ensure that the path you specified for the log file exists in a mounted read/write file system when the server is initialized.

Guess HTTP Type

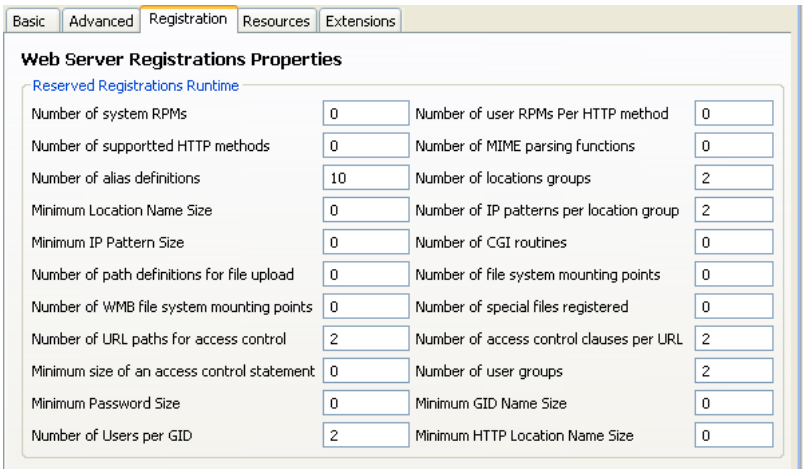
Select this option to allow the server to guess a file's MIME type—based on the file extension—for the most efficient processing.

> Note: MIME-type guessing is required for WindMark processing.

15. Web Server Registration Properties Reference

When you generate your embedded HTTP server configuration files, many compile macro values are determined automatically by the MCE.

The Registration dialog box allows you to increase the values of these macros to a fixed number beyond that determined by the MCE.



Following are the available options.

Number of system RPMs

This value sets *HTTP_SYSRPM_CONF_MAX* the specified number greater than required by your project configuration. The MCE sets the value of this macro to the minimum number required for your project configuration. If you plan to use system RPMs programmatically that you have not configured using the MCE, you must increase this value by 1 for each RPM you plan to configure later. For example, if you have not configured any aliases using the MCE (in which case the aliasing system RPM is not configured), but plan to configure aliases at run-time, you must increment the value here by 1. For more information about system RPMs, see Embedded HTTP Server on page 26.

Number of user RPMs Per HTTP method

This value sets *HTTP_RPM_CONF_MAX* the specified number greater than required by your project configuration. User RPMs are those that are mapped to a specific URL substring, and only called for HTTP requests to URLs relative to that substring. You must increase this value by 1 for each additional user RPM you plan to configure to a particular URL substring programmatically (that you have not configured using the MCE).

Number of supported HTTP methods

This value sets *HTTP_RPM_DISP_MAX* the specified number greater than required by your project configuration. GET and POST are the only HTTP methods currently supported; therefore, there is no need to increment this number.

Number of MIME parsing functions

This value sets *HTTP_MIMEPARSE_CONF_MAX* the specified number greater than required by your project configuration.

Number of alias definitions

This value sets *HTTP_ALIAS_CONF_MAX* the specified number greater than required by your project configuration. You must increase this value by 1 for each additional URL alias you plan to dynamically configure using httpAliasConfAdd( ).

Number of locations groups

This value sets *HTTP_LOCATION_CONF_MAX* the specified number greater than required by your project configuration. You must increase this value by 1 for each additional IP location group (for IP address access control) you plan to dynamically configure using httpIpConfAdd( ).

Minimum User Name Size

This value sets *HTTP_USERNAME_SIZE*. If you configure other user names using the MCE with names larger than the value entered here, the MCE sets this constant to the length of the longest name you statically configure.

Number of IP patterns per location group

This value sets *HTTP_IP_PER_LOCATION_MAX* the specified number greater than required by your project configuration. This value should be the difference between the number of members in the largest IP location group you configure using the MCE (if any) and the largest group size you plan to configure dynamically. For example, if you configure a single location group with a single member using the MCE, the tool determines that the value of *HTTP_IP_PER_LOCATION_MAX* should be 1. If you plan to dynamically configure location groups with up to 16 members each, you must set this value greater or equal to 15.

Note: The MCE automatically sets the macros *HTTP_LOCATION_NAME_SIZE* and *HTTP_IP_PATTERN_SIZE* to the largest IP group name and IP address pattern size you statically configure. If you plan to dynamically configure IP filtering, you must adjust these values to accommodate the largest group name and IP pattern you plan to use.

Minimum IP Pattern Size

This value sets *HTTP_IP_PATTERN_SIZE*. If you configure other IP address filtering patterns using the MCE with filter strings larger than the value entered here, the MCE sets this constant to the length of the longest name you statically configure.

Number of CGI functions

This value sets *HTTP_CGI_FN_CONF_MAX* the specified number greater than required by your project configuration. You must increase this value by 1 for each additional CGI function you plan to dynamically configure using httpCgiFnConfAdd( ).

Number of path definitions for file upload

This value sets *HTTP_UPLOAD_CONF_MAX* the specified number greater than required by your project configuration. You must increase this value by 1 for each additional upload path you plan to dynamically configure using httpUploadConfAdd( ).

Number of file system mounting points

This value sets *HTTP_FS_CONF_MAX* the specified number greater than required by your project configuration. You must increase this value by 1 for each additional file system you plan to dynamically configure using httpFsConfAdd( ).

Number of WMB file system mounting points

This value sets *HTTP_WMB_PATHS_MAX* the specified number greater than required by your project configuration. You must increase this value by 1 for each additional file system you plan to dynamically configure with WindMark parsing using httpRpmConfAdd( ).

Number of special files registered

This value sets *HTTP_FILE_REGISTRY_MAX* the specified number greater than required by your project configuration. You must increase this value by 1 for each file you plan to dynamically register. Registered files are those that require special handling. For example, if your default file system is compressed ANSI, and you have certain files that should not be uncompressed when sent to the browser, you must register those files. For more information about file systems and registered files, see Working with Web Content Files on page 122.

Number of URL paths for access control

This value sets *HTTP_CTRL_CONF_MAX* the specified number greater than required by your project configuration. You must increase this value by 1 for each access controlled URL you plan to configure dynamically.

Number of access control clauses per URL

This value sets *HTTP_MEMBERS_PER_CTRL* the specified number greater than required by your project configuration. You must increase this value by 1 for each additional access control statement you plan to dynamically apply to any given URL using httpCtrlConfAdd( ). For example, if you statically configure 3 access-controlled URLs with one user group each, the tool determines that this number must be at least 1. If you plan to dynamically configure four additional user groups for each of these URLs, you must increase this value to 5 or greater.

Minimum size of an access control statement

This value sets *HTTP_CTRL_URL_SIZE* the specified number greater than required by your project configuration. This value should be the difference between the largest access control clause statically configured and the largest access control statement you plan to dynamically configure.

Number of user groups

This value sets *HTTP_GID_CONF_MAX* the specified number greater than required by your project configuration. You must increase this value by 1 for each additional user group (for user group based access control) you plan to dynamically configure using httpPwdConfAdd( ).

Minimum Password Size

This value sets *HTTP_PASSWORD_SIZE*. If you configure other user passwords using the MCE with passwords larger than the value entered here, the MCE sets this constant to the length of the longest name you statically configure.

Minimum GID Name Size

This value sets *HTTP_GID_NAME_SIZE*. If you configure other user groups using the MCE with names larger than the value entered here, the MCE sets this constant to the length of the longest name you statically configure.

Number of Users per GID

This value sets *HTTP_USER_PER_GID* the specified number greater than required by your project configuration. You must increase this value by 1 for the additional number of users per user group you plan to dynamically configure.

Minimum HTTP Location Name Size

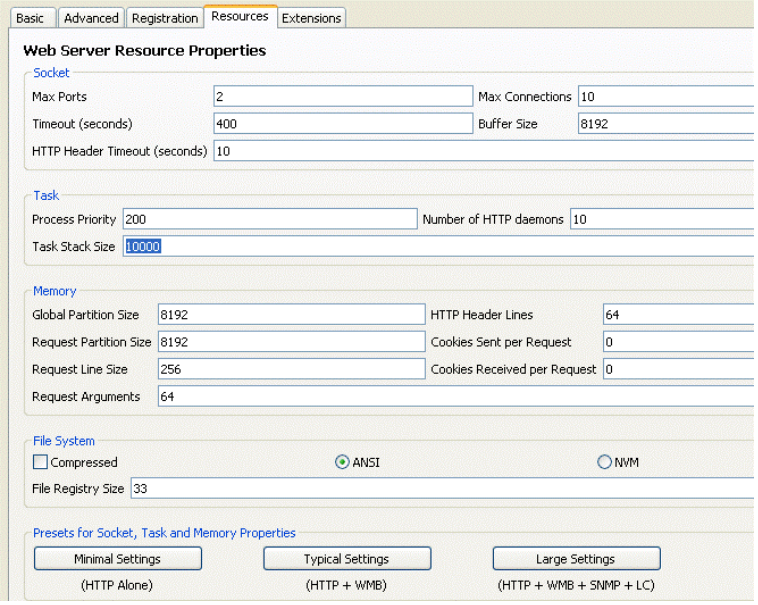
This value sets *HTTP_LOCATION_NAME_SIZE*. If you configure other location groups using the MCE with names larger than the value entered here, the MCE sets this constant to the length of the longest name you statically configure.

WNDRVR

16. Web Server Resource Properties Reference

The Web Server Resource Properties dialog box allows you to set socket, task, and memory options.

The following sections provide details on the available options.



For information on configuring the Web Server, see Configuring the Web Server on page 62.

Socket Options

Max Ports

Specify the maximum number of ports the server uses.

Max Connections

Specifies the maximum number of concurrent client connections. The value is equal to Number of HTTP daemons and cannot be modified.

Timeout (seconds)

Specify the server timeout. The server returns an error if an HTTP connection is idle longer than this time value (seconds).

Buffer Size

Specify the server's socket internal memory buffer size. A larger value allows queueing more network packets, but also uses more memory.

HTTP Header Timeout

Specify the timeout, in seconds, for the HTTP header receipt. The server will close the HTTP connection if the server cannot receive the entire HTTP message, start-line and headers, within this time value.

If the *HTTP_HEADER_TIMEOUT* is set to a value that is small and reasonable, it will prevent clients from issuing unreasonably slow requests to the Web Server.

WNDRVR

Task Options

The following settings specify a subset of the task creation parameters for the embedded HTTP server tasks.

The call to taskSpawn( ), which starts the HTTP server, is wrapped in the OS abstracted function wmosThreadCreate( ). In this call, the VxWorks task options flag is hard coded as zero (0), which means the HTTP server is spawned without the following task options:

- VX_FP_TASK – floating point coprocessor support
- VX_UNBREAKABLE – no breakpoint debugging
- VX_PRIVATE_ENV – private environment support
- VX_NO_STACK_FILL – do not fill the stack for use by checkstack( )

Process Priority

Specify the priority of the embedded HTTP server task(s). The priority for VxWorks application level tasks should be between 100 and 250. The MCE allows you to enter a value outside this range. However, the task creation call wmosThreadCreate( ) uses a default priority of 101 if your specified value is outside this range.

Number of HTTP daemons

Indicate the number of HTTP daemon tasks that are spawned at initialization. Running multiple server tasks allows faster processing of HTTP requests. It also can speed up processing by sending a response out on a different process while still listening on the first process.

As with any option, there is a trade-off. Each task has its own stack and task control block, therefore running a high number of tasks comes at the cost of memory.

Note: When the live control feature is disabled and this value is less than 3, the actual number of HTTP daemon tasks will be 3. When the live control feature is enabled and this value is less than 6, the actual number of HTTP daemon tasks will be 6. If Max Connections is set at less than the actual number of HTTP daemon tasks, the actual maximum number of concurrent client connections will be equal to the actual number of HTTP daemon tasks.

Task Stack Size

Set the stack size for each HTTP daemon process spawned at initialization.

Memory Options

Global Partition Size

Set the internal memory buffer size in bytes for the embedded server application. The best value for your device may depend on your target system.

For target systems that allow only aligned access, assign a value for the buffer size that conforms to the alignment boundary size. For example, on a 32-bit-aligned system, the buffer size should be a multiple of 4 (8 bits/byte x 4 bytes = 32).

Request Partition Size

Set the memory pool size in bytes for servicing each HTTP request structure. The default value is 4096. A larger value improves transfer speed, but it does so at the expense of more memory usage. The best value for your device may depend on your target system.

For target systems that allow only aligned access, assign a value for the pool size that conforms to the alignment boundary size. For example, on a 64-bit-aligned system, the pool size should be a multiple of 8 (8 bits/byte x 8 bytes = 64).

WNDRVR

Request Line Size

Set the maximum character length of an HTTP request line. An HTTP request line consists of the HTTP method (GET, HEAD, POST, and so forth), a space, the request URI, a space, the HTTP version, and a CRLF:

```
request line = http method<SP>request URI<SP>HTTP version<CRLF>
```

The default request line setting is 128. This should suffice for any request made to a backplane managed element. If you implement any custom CGI applications that use long URL query strings, you need to adjust this value accordingly.

Note: This is also the value of the longest acceptable MIME header. Very long file names on Content-Disposition headers (used in file uploads) may require a large number for this setting.

Request Arguments

For HTTP request arguments—which is everything after the ? in the request line—this value sets the maximum number of request arguments you can pass into the request structure. This is used for CGI and GET request forms.

HTTP Header Lines

Specify the maximum number of HTTP message headers the server can process. The HTTP 1.1 specification does not put a limit on the number of headers that an HTTP request may contain, but the Web server needs to have a maximum defined.

Cookies Sent per Request

The exact number of cookies sent per request is determined by your use of cookies. Set this value to match the highest number of cookies per request you expect to send with any given request.

Cookies Received per Request

The exact number of cookies received per request is determined by your use of cookies. Set this value to match the highest number of cookies per request you expect to receive with any given request.

File System Options

Compressed

If your default file system is compressed, either **ROMBUILD** compressed or a compressed **ANSI** file system, select Compressed so that the server uncompresses the files as they are requested.

If you plan to serve a combination of compressed and uncompressed files, you must choose one as the default operation, and the other files should all be registered.

ANSI

Select this option to choose **ANSI** as the default file system.

NVM

Select this option to choose **NVM** as the default file system.

File Registry Size

This value must be at least as large as the number of registered files served by your target system. If you plan to add files at run-time that may be registered, this value must be set high enough to accommodate them.

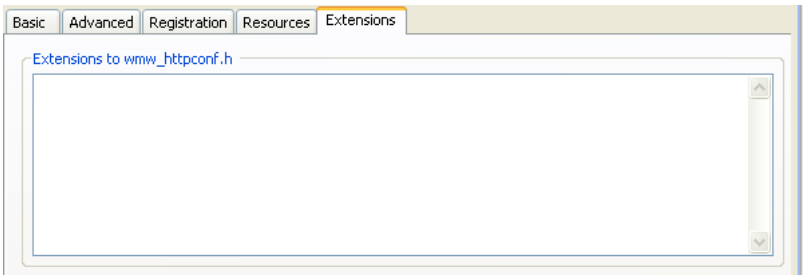
Presets for Socket, Task and Memory Properties

Use the buttons in this section to set initial suggested values for each item in the Socket, Memory, and Task groups.

17. Web Server Extensions

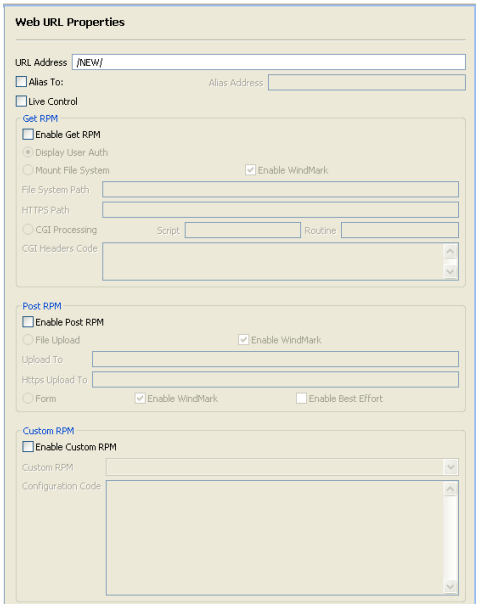
The Web Server Extensions dialog box allows you to add your own code to the generated configuration files.

The custom code you enter in Web server extensions gets added to the generated file **wmw_httpconf.h**.



18. Web URL Properties Reference

The Web URL Properties dialog box allows you to create URLs and indicate exactly what processing should take place when an HTTP request is made of that URL.



The following sections provide details on the available options.

For information on creating and configuring URLs, see Creating a Web-Based Management Interface on page 106.

Basic Options

URL Address

Type a name for the new URL substring you wish to mount a file system at. The URL address must start with a / or *, contain valid characters, and end with a /; for example, **/fs/**.

Alias To

Select this option if you want to create an alias for your URL. For more information on URL aliasing, see Setting Up URL Aliasing on page 107.

Alias Address

Type the target URL for this alias.

Note: The target URL you specify must exist on the system at run-time.

Live Control

Select this option to enable **Live Control**.

Get RPM Options

Enable Get RPM

Select this option if you want this URL to process HTTP GET requests.

Display User Auth

Selecting this option allows you to display the current user's authentication information by requesting a particular URL. For more information, see Displaying User Authentication on page 108.

Mount File System

Select this option to specify file system and HTTPS paths.

Enable WindMark

This option provides a simple means to turn WindMark processing files on or off. You should typically enable WindMarks. However, if you have a file system that only contains static files and requires no data substitution for dynamic content, you can reduce the processing overhead for those static files by not selecting this option.

File System Path

For ANSI or CUSTOM file systems, type the target file system's mount point. For example, **/romfs/webfiles/doc**, or **/tffs0/htdocs**.

For ROMBUILD file systems, leave this field empty.

HTTPS Path

Type the target file system's mount point for an SSL secured file system. For example, **/romfs/webfiles/ssl**, or **/tffs0/ssl_htdocs**.

Warning: If you are not using SSL, do not enter a path here. If you enter any value here, the MCE enables SSL. The generated code will have definitions that require the SSL libraries.

Note: File System mount points can also be created dynamically using the API function httpFsConfAdd( ). For more information, see the reference entry for httpFsConfAdd( ).

CGI Processing

The CGI RPM is provided as a legacy porting aid. Select this option to call a CGI application for requests made of a particular URL. For more information, see Enabling CGI Processing on page 109.

Script

Type the incoming request URL path for the CGI function.

Routine

Type the name of the CGI function that will process the incoming request.

CGI Headers Code

Add any #include directives your CGI function requires.

Post RPM Options

Enable Post RPM

Select this option if you want this URL to process HTTP POST requests. A post request sends data along with the HTTP request headers. In the case of Wind River Web Server, the data that accompanies a post request is typically a new (updated) WindMark value. There are two types of post requests the Web server can handle: file uploads and form data posts.

File Upload

Select this option to enable file uploads for your URL. For more information on file uploads, seeEnabling File Uploads on page 110.

Enable Windmark

You may want to do some post processing of uploaded files, or evaluate other form elements (text boxes, check boxes, and so on) that may be present in the upload page. If you select the Enable WindMark option for an upload URL, the backplane RPM parses the additional form elements in the POSTed message for WindMarks and (through the backplane) calls the appropriate handlers.

Upload To

For ANSI or CUSTOM file systems, type the target path you wish to mount. You cannot enable uploads for a ROMBUILD file system.

HTTPS Upload To

Type the target path you wish to mount for an SSL secured file system.

> Note: The selected file system for the upload path must be writable.

Form

Select this option to enable form data posts for your URL.

Enable WindMark

When you select Enable WindMark, the HTTP server parses the posted form data for WindMarks, and all the WindMark name:value pairs are passed to the backplane to be processed as a single set transaction.

Enable Best Effort

When you select **Enable Best Effort**, the posted WindMarks (that are passed to the backplane as a transaction) are each set as possible regardless of the success of the other WindMarks in the transaction. When **Enable Best Effort** is cleared, the transaction only succeeds if all WindMarks in the transaction succeed.

> Note: All SNMP WindMarks are still treated as a single transaction, even with Enable Best Effort selected. This is because MIBway does not have complete control over the actions of the SNMP agent.

WNDRVR

Custom RPM Options

Enable Custom RPM

Select this option to specify a URL for custom RPM processing.

Custom RPM

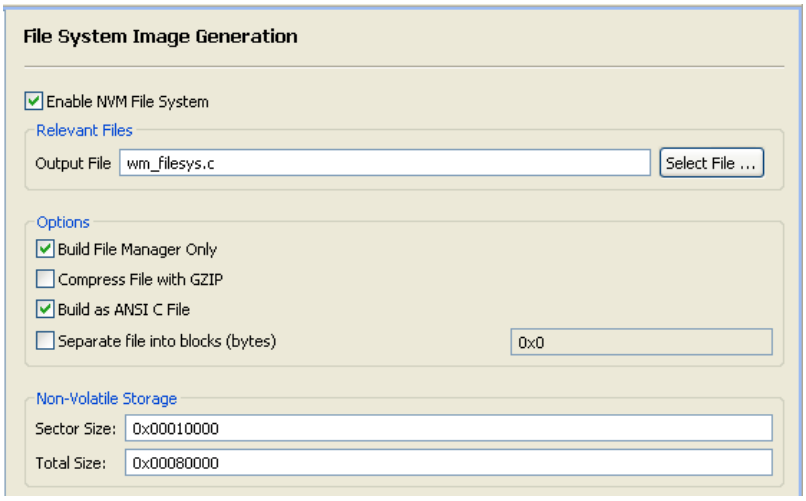
Provide the name of the custom RPM function. If you have already entered information about a custom RPM you have written, you can select your RPM from the drop-down list.

Configuration Code

Enter any #include directives required by the function, and any initialization code your RPM requires.

19. File System Image Generation Reference

The File System Image Generation dialog box allows you to set the build file system options.



General Options

Enable NVM File System

Select this option to enable the NVM file system.

Relevant Files Options

Output File

Select the output ROMBUILD NVM image file.

Options

Build File Manager Only

Select this option to build the ROMBUILD NVM file system image only.

Compress File with GZIP

When you create a ROMBUILD file system, the files flagged to be included in the file system are stored in **wm_filesys.c** in coded form. This file is built into the binary image. If you select this option, the contents of **wm_filesys.c** are GZIP files. The Web Server will uncompress the files as they are requested.

Build as ANSI C File

Select this option to build the ROMBUILD NVM image as an ANSI C file that can be compiled into your project.

Separate file into blocks (bytes)

Select this option to break the ROMBUILD NVM file system image into multiple files of the specified size.

Non-Volatile Storage Options

The default action is for the ROMBUILD NVM file system image to be built as an ANSI C file that can be compiled into your project. The options in this section support systems where you wish to place a fixed array in a particular memory location on your system; for example, fixed ROM chips. The contents of your ROMBUILD file system image are placed into a binary array.

Sector Size

Specifies the sector size for the host NVRAM.

Total Size

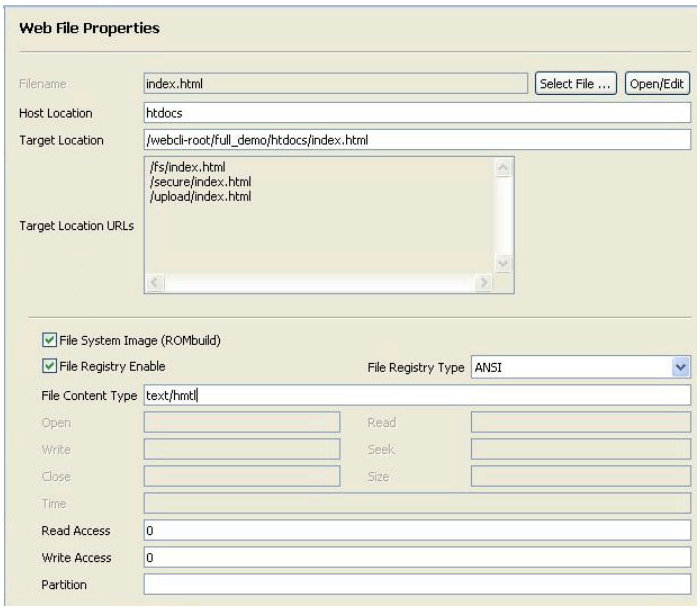
The total size of the memory file system.

20. Web File Properties Reference

The Web File Properties dialog box allows you to specify the settings for the files in your project.

This is where you indicate how the server should access the file— ANSI_FS or MEM_FS. Following are the available options.

For information on files settings and adding files to your project, see Adding Web Files to Your Project on page 112.



Filename

The name of the web file.

Select File

Click **Select File** to browse to the location of the file you want to include in your project.

Open/Edit

Clicking **Open/Edit** opens the file for editing in a built in text editor.

Host Location

Displays the complete directory path to the file.

Target Location

Type the target path for the file you are adding.

Target Location URLs

If the value specified in the **File System Path** field of the Web URL Properties dialog box for a URL matches the Target Location, the URL name is listed in this field.

File System Image (ROMbuild)

Select this option if you want to build the file into the ROMBUILD file system.

File Registry Enable

Select this option to register the file with the file registry.

File Registry Type

Select the desired type from the drop-down menu. The options are as follows:

- ANSI
- ROMbuild
- Compressed ANSI

- Compressed ROMbuild
- Custom

If you choose ANSI, Compressed ANSI, ROMbuild, or Compressed ROMbuild, the access functions are provided for you.

If you choose Custom, you must indicate the content type and choose which file system access functions to use. You must also provide the following file access functions (analogous to the ANSI standard C file access functions):

- customOpen( )
- customRead( )
- customWrite( )
- customSeek( )
- customClose( )
- customSize( )
- customTime( )

File Content Type

Set the type of file and its encoding. For example, if the file is a picture, you might set the type of file to image and the encoding to jpeg, so the appropriate entry would be image/jpeg. The default file type is determined by the file's extension (e.g., the file foo.txt will have a default file type of text/plain, while foo.html will have a file type of text/html, and so on).

The following file type and encoding pairs are recognized:

Table 1. Application file types

application/java	application/octet	application/octet-stream
application/pdf	application/postscript	application/rtf
application/sgml	application/x-fax-\manager	application/x-postscript
application/x-pdf	application/x-www-\urlformunencoded	application/zip

Table 2. Audio file formats

audio/aiff	audio/basic	audio/x-wav

Table 3. Image file formats

image/gif	image/jpeg	image/rgb
image/tiff	image/x-cmu-raster	image/x-pict
image/x-rgb	image/x-xbitmap	

Table 4. Text file formats

text/html	text/javascript	text/plain
text/richset	text/vbscript	text/x-sgml

Read Access

Set the read access level for the file.

WNDRVR

Write Access

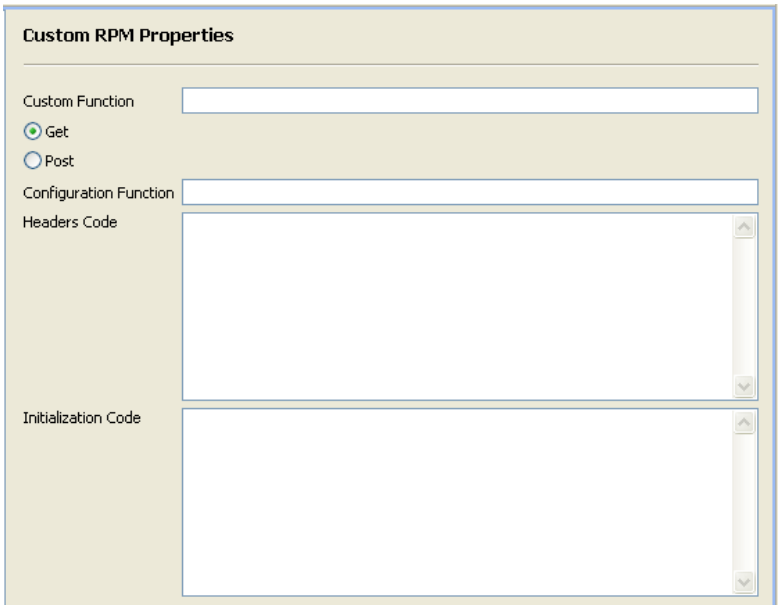
Set the write access level for the file.

Partition

When you create a ROMBUILD file system, the files flagged to be included in the file system are stored in a C source file in coded form. The C file is built into the binary image. The default file is named **wm_filesys.c**. The text you enter in this field will be appended to the file name. For example, if you enter foo in the Partition field, the name of the C source file becomes **wm_filesys_foo.c**.

21. Custom RPM Properties Reference

The Custom RPM Properties dialog box allows you to create your own RPMs.



Custom Function

Type a name for your custom RPM.

Get

Select this option if your custom RPM handles GET requests.

Post

Select this option if your custom RPM handles POST requests.

Configuration Function

Allows you to specify a function to call when the RPM is used.

Headers Code

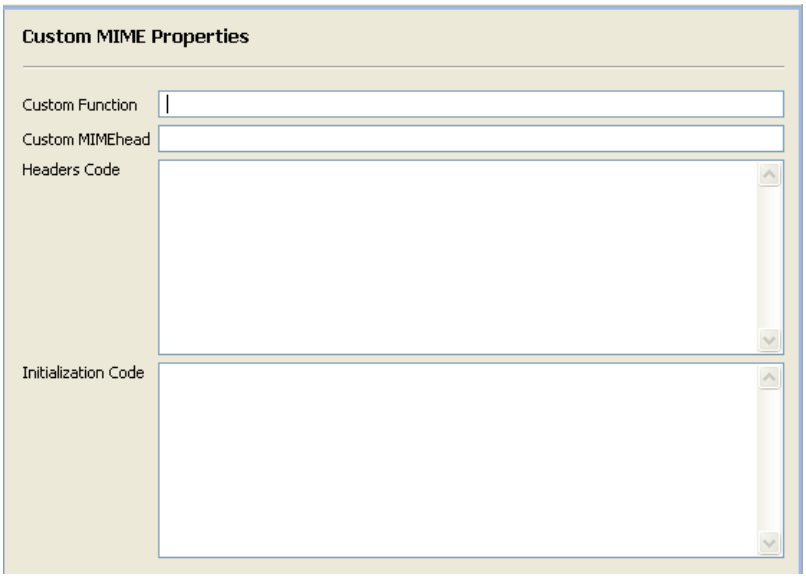
Add any #include directives your RPM requires.

Initialization Code

Add any initialization code your RPM requires.

22. Custom MIME Properties Reference

The Custom MIME Properties dialog box allows you to create custom MIMEs.



The options are as follows:

Custom Function

Type a name for your custom MIME.

Custom MIMEhead

Specify your custom MIME header field.

Headers Code

Add any #include directives your MIME requires.

Initialization Code

Add any initialization code your MIME requires.

14. SAMPLE GLOBAL COMMANDS

1. General Purpose Commands Reference

The project file **commands.rcp** contains sample implementations of several useful general purpose global commands—such as logout, exit, and write.

commands.rcp is located in the directory *installDir* **/vxworks-7/pkgs_v2/app/webcli/cli-**x.x.x.x **/src/examples/**

Table 1. Commands in commands.rcp

Command	Description
alias	Establish an alias for a string—for syntax and more information, see the VxWorks 7 Creating a CLI Tool Tutorial.
broadcast	Broadcast a message to all terminals—to send a message to one user, use write (see below).
clear	Clear screen.
echo	Echo entered text.
date	Print date.
enable	Enter *privileged* mode.
exec	Execute a script.
exit	Exit intermediate mode and access one level up in the CLI hierarchy tree.
help	Show help for command.
history	Show history buffer.
logout	Log out from the system.
show	Show network information.
sleep	Sleep for x seconds.
stty	Set terminal columns and/or rows, get status of same.
user access	Change user access level.
user add	Add a user to the system.

Command	Description
user delete	Delete a user from the system.
user password	Change a user's password.
user show	Show users in system (regardless of login status).
who	Display users who are logged on.
whoami	Display user logon.
write	Write to a user. If message contains spaces, put the message in quotes: `write user7 "Hello user7"`

2. Debug Command Reference

The project file **debug.rcp** contains sample implementations of global commands useful for debugging purposes.

debug.rcp is located in *installDir* **/vxworks-7/pkgs_v2/app/webcli/cli-**x.x.x.x **/src/examples/**

These commands are intended for development purposes only.

Table 1. Debug Commands

Command	Description
memory	Display amount of memory used in prompt.
wmget	Get value of a WindMark.
wmset	Set a WindMark.
wmnext	Get the value of the next instance of a tabular object.
sleep	Sleep for x seconds.
system info	Show current user name, access level, and parameter ordering.

3. File System Command Reference

The project file **files.rcp** contains sample implementations of the following global commands useful for debugging purposes.

files.rcp is located in the directory *installDir* **/vxworks-7/pkgs_v2/app/webcli/cli-**x.x.x.x **/src/examples/**

These commands are intended for development purposes only.

Table 1. File System Commands

Command	Description
cat	Display contents of a file.
ls	List files in the current directory.
cd	Change to a new directory.
pwd	Print the working directory.

Note: These commands require the ANSI file system to be enabled. The code is included regardless of other blades purchased, but the tool currently does not allow enabling it unless Wind River Web Server is enabled as well. On the Extensions page of the Project Settings dialog box: #define **__ANSI_FILE_MANAGER_ENABLED__** and **__ENABLE_SESSION_DIRECTORIES__**.

4. SNMP Command Reference

The project file **snmp.rcp** contains a sample SNMP related global command.

snmp.rcp is located in the directory *installDir* **/vxworks-7/pkgs_v2/app/webcli/cli-**x.x.x.x **/src/examples/**

This command is intended for development purposes only.

Table 1. SNMP Commands

Command	Description
commit	Commit a command in intermediate mode. If the user is in a subnode and a handler writes to an SNMP WindMark, that write is usually queued up. The commit command executes that (and any other pending SNMP) write. Exiting the subnode discards that write. Note: This is only when in a subnode; at the root node, commits are automatic.

15. WINDMARKS IN HTML FORM ELEMENTS

1. Introduction

WindMarks can be inserted into HTML form elements.

This chapter discusses the form elements and describes how the insert WindMark and remove WindMark operations handle each type of element. Note that the exact HTML code generated differs for some types of HTML form elements.

2. Text, Hidden, and Password Form Elements

Manipulate the text, hidden and password form elements using the insert and remove WindMarks.

Insertion

The insert WindMark operation handles text, hidden, and password form elements in an identical fashion. It takes the string in the **NAME** field and creates a **VALUE** field equal to the **NAME** field with a beginning ($%) and ending (#$) delimiter. For example, running either of these two HTML tags:

```
<INPUT TYPE="text" SIZE="20" NAME="IncomingText" VALUE="Default Incoming Text">
<INPUT TYPE="text" SIZE="20" NAME="IncomingText">
```

through the Insert WindMarks operation yields the following:

```
<INPUT TYPE="text" SIZE="20" NAME="IncomingText" VALUE="$%IncomingText#$">
```

Removal

The remove WindMark operation also handles text, hidden, and password form elements in an identical fashion. It simply removes the WindMark. For example, passing the form element above through the remove WindMark operation yields the following:

```
<INPUT TYPE="text" SIZE="20" NAME="IncomingText" VALUE="">
```

The remove WindMark operation does not replace any VALUEs that might have previously been in the HTML. It simply removes the WindMark.

3. Radio Button Form Elements

Manipulate radio button form elements using the insert and remove WindMarks.

Insertion

For radio buttons, the

```
insert WindMark
```

operation uses the NAME field as the main WindMark and uses the **VALUE** field for each radio button as an argument to each of the main WindMarks. The insert WindMark operation removes any CHECKED strings from the HTML.

For example, running this set of radio buttons:

```
<INPUT TYPE="radio" NAME="size" VALUE="small"> small
<INPUT TYPE="radio" NAME="size" VALUE="respectable"> respectably sized
<INPUT TYPE="radio" NAME="size" VALUE="medium"> medium
<INPUT TYPE="radio" NAME="size" VALUE="large" CHECKED>big
```

through the

```
insert WindMark
```

operation yields:

```
<INPUT TYPE="radio" NAME="size" VALUE="small" $%size(small)#%> small
<INPUT TYPE="radio" NAME="size" VALUE="respectably sized"
    $%size(respectable)#%> respectably sized
<INPUT TYPE="radio" NAME="size" VALUE="medium" $%size(medium)#$> medium
<INPUT TYPE="radio" NAME="size" VALUE="large" $%size(large)#$>big
```

Removal

The removeWindMark operation goes through and deletes all WindMarks in the radio elements. Passing the sample above through the removeWindMark operation yields

```
<INPUT TYPE="radio" NAME="size" VALUE="small" > small
<INPUT TYPE="radio" NAME="size" VALUE="respectable"> respectably sized
<INPUT TYPE="radio" NAME="size" VALUE="medium" > medium
<INPUT TYPE="radio" NAME="size" VALUE="large" >big
```

The removeWindMark operation does not replace any CHECKED strings that might have previously been in the HTML. It simply removes the WindMark.

Checked Handler

In addition to inserting WindMarks as described above, you can also use the checkedHandler( ) function to store CHECKED values and put them back in the HTTP response.

The checkedHandler( ) takes two parameters in the object instance field: a WindMark name and a value. If the value matches at least one of the WindMark values, the returned value is CHECKED. Otherwise, it is an empty string. For usage information, see the Wind River CLI, Web, MIBway API Reference.

This handler only supports GET requests.

4. Check Box Form Elements

Manipulate check box form elements using the insert and remove WindMarks.

Insertion

For check boxes, the insert WindMark operation uses the check box's NAME attribute as the WindMark name. The insert WindMark operation overwrites any checked strings that might be in the HTML tag.

For example, running the following check boxes:

```
<INPUT TYPE="checkbox" NAME="check0" VALUE="television">TV
<INPUT TYPE="checkbox" CHECKED NAME="check1" VALUE="books">Books
<INPUT TYPE="checkbox" NAME="check2" VALUE="radio" CHECKED>Radio
```

through the insert WindMark operation yields:

```
<INPUT TYPE="checkbox" NAME="check0" VALUE="television" $%check0#$>TV
<INPUT TYPE="checkbox" NAME="check1" VALUE="books" $%check1#$>Books
<INPUT TYPE="checkbox" NAME="check2" VALUE="radio" $%check2#$>Radio
```

However, check boxes are somewhat more complicated. The way that CGI processing works is that check boxes that are not CHECKED by the end user do not show up in the stream.

Namely, a check box that is checked will have a NAME=VALUE pair show up in the CGI stream (or a NAME=ON if there is not a value —that is, the default VALUE=ON). However, check boxes that are not checked do not show up in the stream at all. Therefore, in order to have the post engine properly update the WindMark using the backplane on the device, it is necessary to provide the engine with knowledge of all of the check boxes on a given page—even the ones that are not checked. This is accomplished by having an INPUT=HIDDEN field with a NAME of CheckBoxManager and a VALUE that is a comma-delimited string of all of the check boxes on a page.

The insert WindMark operation automatically generates this hidden field and puts it right before the closing </FORM> HTML tag. For example, in the case of the check boxes given above, the insert WindMark operation writes:

```
<INPUT TYPE="hidden" NAME="CheckBoxManager" VALUE="check0,check1,check2">
```

right before the </FORM>.

Removal

The removeWindMark operation automatically removes the WindMark in a check box element. Passing the example above through the WindMark removeWindMark operation, yields the following:

```
<INPUT TYPE="checkbox" NAME="check0" VALUE="television">TV
```

The remove WindMark operation also entirely removes the hidden **CheckBoxManager** element and its attributes. It does not replace any CHECKED strings that might have previously been in the HTML tag.

Checked Handler

In addition to inserting WindMarks as described above, you can also use the checkedHandler( ) function to store CHECKED values and put them back in the HTTP response.

The checkedHandler( ) takes two parameters: a WindMark name and a value. If the value matches at least one of the WindMark values, the returned value is CHECKED. Otherwise it is an empty string. For usage information, see API Reference: Kernel Networking.

WNDRVR

This handler only supports GET requests.

5. Select Form Elements

Manipulate select form elements using the insert and remove WindMarks.

Insertion

The insert WindMark operation uses the NAME attribute in the SELECT tag as the WindMark name, and it uses the VALUE attribute of each OPTION tag as an argument to the WindMark. If there is no VALUE attribute set, it uses the content of the OPTION tag. Any SELECT strings are removed completely from the HTML.

For example, running this Select element:

```
<SELECT NAME="BKRC" size="2">
<OPTION SELECTED>Football</OPTION>
<OPTION VALUE="Sport Of Kings">Soccer
<OPTION>Baseball
<OPTION VALUE="Volley" SELECTED>Volleyball
<OPTION>Water Polo
</SELECT>
```

through the insert WindMark operation yields:

```
<SELECT NAME="BKRC" size="2">
<OPTION $%BKRC(Football)#$>Football</OPTION>
<OPTION VALUE="Sport Of Kings" $%BKRC(Sport Of Kings)#$>Soccer
<OPTION $%BKRC(Baseball)#$>Baseball
<OPTION VALUE="Volley" $%BKRC(Volley)#$>Volleyball
<OPTION $%BKRC(Water Polo)#$>Water Polo
</SELECT>
```

However, as with check boxes, this is not the complete story. This has to do with the way CGI handles multiselect boxes (that is, select boxes that allow you to make more than one selection simultaneously).

When a multiselect box is defined in HTML, it consists of a name and a set of optional values. Each value that is chosen is then sent back to the Web server as a NAME=VALUE pair. In other words, the Web server receives multiple values attached to the same name. Also, if an option is not selected, then the value is not sent. This means that when an option that was previously selected is now cleared, the Web Server does not know anything about its state unless it knows which select elements are actually multiselect ones.

To resolve this, it is necessary to provide the Web Server with knowledge of all the multiselects on a given page—even the ones that are not selected. This is accomplished by having an INPUT=HIDDEN field with a NAME of **SelectBoxManager** and a VALUE that is a comma-delimited string of all the multiselects on a page.

The insert WindMark operation automatically generates this hidden field and puts it right before the closing </FORM> HTML tag, like the HTML shown below:

```
<INPUT TYPE="hidden" NAME="SelectBoxManager" VALUE="name1,name2,...,nameN">
```

where name1...nameN are the names of each multiselect box which is contained within the specified HTML page.

However, unlike the **CheckBoxManager**, which is taken care of in the Web server source code, it is necessary for you to write code to deal with the multiselects.

Removal

The removeWindMark operation removes the WindMarks from a select element, including any arguments to that WindMark. For example, if you passed the select box above through the removeWindMark operation, you get the following:

```
<SELECT NAME="BKRC" size="2">
<OPTION >Football</OPTION>
<OPTION VALUE="Sport Of Kings" >Soccer
<OPTION >Baseball
<OPTION VALUE="Volley" >Volleyball
<OPTION >Water Polo
</SELECT>
```

The remove WindMark operation also entirely removes the SelectBoxManager, if it is there, including the HTML syntax and the listing of all of the multiselects. It does not replace any SELECTED strings that might have been previously in the HTML select tag.

Selected Handler

In addition to inserting WindMarks as described above, you can also use the selectedHandler( ) function to store SELECTED values and put them back in the HTTP response.

The selectedHandler( ) takes two parameters: a WindMark name and a value. If the value matches at least one of the WindMark values, the returned value is SELECTED. Otherwise, it is an empty string. This handler works correctly with multiple selects, that is, selects where more than one option can be selected. For usage information, see the API Reference: Kernel Networking.

This handler only supports GET requests.

6. Text Area Form Elements

Manipulate text area form elements using the insert and remove WindMarks.

Insertion

The insert WindMark operation replaces the default text attribute of a WindMark using the NAME value of the TEXTAREA as the WindMark name. The WindMark is inserted in the same place whether there is any default text or not. Any default text in the text area is removed completely from the HTML

For example, running the following text area element:

```
<TEXTAREA NAME="response" ROWS="5" COLS="60">This is the default text which
appears in the text area. It can be replaced by the user at any time.</TEXTAREA>
```

through the insert WindMark operation yields:

```
<TEXTAREA NAME="response" ROWS="5" COLS="60">$%response#$</TEXTAREA>
```

Removal

The remove WindMark operation removes the WindMark from the text area and leaves an empty string as the default text. Running the remove WindMark operation on the generated text area code gives you the following output:

```
<TEXTAREA NAME="response" ROWS="5" COLS="60"></TEXTAREA>
```

It does not replace any default text that might have previously been in the HTML.

16. SPECIAL WINDMARKS AND WINDMARK MACROS

1. Introduction

Macros are special built-in functions that allow you to dynamically generate HTML based on various conditions.

These conditions can be simple hard-coded conditions or they can be based on the value of device resident variables (as represented by WindMarks).

This chapter provides a description and usage information for each of the special WindMarks and WindMark macros.

2. The REPEAT Macro

The **REPEAT** macro is a loop mechanism that allows you to iterate through a segment of HTML code multiple times.

The **REPEAT** macro is useful for generating HTML tables where the number of rows or cells is dynamic. You can nest **REPEAT** macros within other **REPEAT** macros to display multidimensional lists or arrays. The **REPEAT** macro can be nested within other macros up to the specified nesting depth (see Basic Web Server Properties Reference on page 152.

The **REPEAT** macro starts with

```
$%REPEAT(BEGIN,x.y)#$
```

where x is the first instance displayed, and y is the number of times to iterate through the repeat loop. The **REPEAT** macro ends with

```
$%REPEAT(end)#$
```

In the example below, the loop starts with instance zero and iterates nine times. The **REPEAT** macros must be within an HTML comment block as in the following example:

```
<!-- $%REPEAT(BEGIN,0,9)#$ -->
    <tr>
        <td>$%ipAdEntIfIndex#$</td>
        <td>$%ipAdEntAddr#$</td>
        <td>$%ipAdEntNetMask#$</td>
        <td>$%ipAdEntBcastAddr#$</td>
        <td>$%ipAdEntReasmMaxSize#$</td>
    </tr>
<!-- $%REPEAT(end)#$ -->
```

Note: The 0 and 9 arguments used in the **REPEAT** example are arbitrary. You can use WindMarks that evaluate to int values here so long as the second argument always evaluates to zero or greater and the third argument evaluates to 1 or greater (a **REPEAT** must iterate at least once).

3. The IF Macro

The **IF** macro is a conditional mechanism that allows you to generate HTML based on the evaluation of a test condition.

The **IF** macro can be nested within other macros up to the specified nesting depth (see Basic Web Server Properties Reference on page 152).

The **IF** macro starts with the

```
$%IF(BEGIN, condition)#$
```

where condition is a comparison test between the values of two WindMarks (or a WindMark and a constant). The following comparison operators can be used:

```
<, <=, =, ==, >, >=, <>
```

If condition tests true, the HTML between the opening and closing **IF** macro WindMarks is processed. The example below shows the **IF** macro being used to color code values as they compared to a reference value.

String constants must be enclosed by double quotes; all integers are in decimal. For strings, the single-equal = operator ignores case, whereas the double-equal operator == does an exact compare. Extra spaces around the arguments are ignored. Lastly, as a catch-all for unspecified conditions, you can use

```
$%IF(ELSE)#$
```

as shown in the example.

```
$%IF(BEGIN, tableIndex < watchValue )#$
    <font color="green">,$%tableIndex#$</FONT>
$%IF(ELSE, tableIndex = watchValue )#$
    <font color="blue">,$%tableIndex#$</FONT>
$%IF(ELSE)#$
    <font color="red">,$%tableIndex#$</FONT>
$%IF(END)#$.
```

Note: The WindMarks used in the comparison tests do not use the $% and #$ WindMark delimiters. The WindMarks used to display WindMark values in HTML must use them.

4. The endRow Macro

The **endRow** macro works in conjunction with the **createTable** macro to indicate the end of an SNMP table row.

This is part of the filtering mechanism that allows you to selectively display parts of an SNMP table. The **endRow** macro takes one argument; the name of the table used to create the table object with **createTable**. The **endRow** macro usage is

```
$%endRow(wm_table)#$
```

5. The endTable Macro

The **endTable** macro works in conjunction with the **createTable** macro.

When you are finished generating the HTML for a table, **endTable** removes the table descriptor and frees the associated memory.

WNDRVR

The **endTable** macro takes one argument; the name of the table used to create the table object with **createTable**. The **endTable** macro usage is

```
$%endTable(wm_table)#$
```

6. The fileInclude WindMark

The **fileInclude** WindMark allows you to piece together Web pages from smaller templatized HTML file fragments.

The **fileInclude** WindMark takes one argument: the relative URL to the file to include. Following is an example of the **fileInclude** WindMark:

```
$%fileInclude(templates/overview.html)#$
```

Note:

Do not use leading path delimiters (such as / or ../) on the relative URL argument for

```
$%fileInclude( )#$
```

This argument is appended to the file system mount point to determine the file resource to send. If you use leading path delimiters, the file reference will contain these as extra characters. For example, the following usage does not work with ROMBUILD file systems:

```
$%fileInclude(./dirname/filename.html)#$
```

The above example references a file **/mountpoint/./dirname/filename.html**. This is not a problem for ANSI file systems, but this fails for ROMBUILD file systems.

Note:

Macros do not work inside the **fileInclude** WindMark. For example, you cannot display a table object that uses the **REPEAT** macro in a fileInclude WindMark. You can use any WindMark that does not use macros in a fileInclude.

7. The getInstance WindMark

The **getInstance** WindMark allows you get the current instance of a WindMark table member.

This is particularly useful when you need to update the value of a WindMark inside a dynamic table (created by using the **REPEAT** macro). The getInstance WindMark can be appended to another WindMark to display the instance number as the instance changes during each iteration. The **getInstance** WindMark takes one argument; the WindMark table name you wish to get the current instance of. Use the **getInstance** WindMark as follows:

```
$%getInstance(wm_table)#$
```

This WindMark returns the int value of the current instance of the WindMark **wm_name**. Typically you append getInstance as an instance identifier to a WindMark you use in a GET or SET operation. The example below shows the usage of **getInstance** in a dynamic table display. The HTML that draws the table is shown in its entirety here for context. The actual use of **getInstance** is in bold type.

```
<!-- $%createTable(ifIndex,*,,,,include,ifIndex,ifDescr,ifType,ifMtu,ifSpeed,ifPhysAddress,ifAdmin
Status,ifOperStatus,ifLastChange,ifInOctets,ifInUcastPkts,ifInNUcastPkts,ifInDiscards,ifInErrors,i
fInUnknownProtos,ifOutOctets,ifOutUcastPkts,ifOutNUcastPkts,ifOutDiscards,ifOutErrors,ifOutQLen,if
```

```
Specific)#$ -->
<p> </p>
<p class="bold"> Read-Write SNMP Table Object Display: ipAddrTable</p>
<p> In this example of createTable, you can
        update the object's value in the table. If you set the value of the
        object <span class="bold">ifAdminStatus</span> to 2, the interface
        will be taken down. <span class="bold">Note</span>: If your target
        has only one external interface and you set this to 2, you will
        lose the connection to your target. </p>
<p>
<table border="true" cellpadding="3" cellspacing="0">
        <tr class="body_t" bgcolor="#00008B" style="color:white;">
           <th>ifIndex</th>
           <th>ifDescr</th>
           <th>ifType</th>
           <th>ifMtu</th>
           <th>ifSpeed</th>
           <th>ifPhysAddress</th>
           <th>ifAdminStatus</th>
           <th>ifOperStatus</th>
           <th>ifLastChange</th>
   </tr>
<!-- $%REPEAT(BEGIN,0,9)#$ -->
        <tr class="body_t" bgcolor="#E6E6FA">
           <td>$%ifIndex#$ </td>
           <td>$%ifDescr#$ </td>
           <td>$%ifType#$ </td>
           <td>$%ifMtu#$ </td>
           <td>$%ifSpeed#$ </td>
           <td>$%ifPhysAddress#$ </td>
           <!-- The next table row uses the getInstance WindMark to generate a
                        select form option for each table instance (and also it will
                        also repeat for each row (interface instance) in the
                        generated HTML.  -->
           <td><form action="./macros.html" method="post">
                          <select name="ifAdminStatus.$%getInstance(ifIndex)#$">
                          <!-- NOTE: the SELECTED special WindMark takes 2 args.
                                 The first arg is a WindMark, the second is a value
                                 to compare against the value of agr1. If the
                                 comparison matches, the SELECTED WindMark simply
                                 places the string "SELECTED" inside the select form
                                 element in the HTML response.
                                 This causes the option that matches the current
                                 value to display as the default.

                                 The SELECTED special WindMark does not require
                                 you to indicate the instance of an object. It will
                                 retreive all instances and return SELECTED if
                                 any instances match.
                           -->
                                 <option value="1" $%SELECTED(ifAdminStatus,1)#$ >
                                         1: Up</option>
                                 <option value="2" $%SELECTED(ifAdminStatus,2)#$ >
                                         2: Down</option>
                                 <option value="3" $%SELECTED(ifAdminStatus,3)#$ >
                                         3: Testing</option>
                          </select>
```

```
          </td>
          <!-- The following table cells use the IF macro to display
                       additional useful text similar to the select values.
                       A device user might not know what the ifOperStatus
                       integer values are defined as, therefore you can provide
                       additional text based on the objects value.
                       Similar to SELECTED, you do not need to use
                       getInstance here.
               -->
          <td>$%ifOperStatus#$
                  $%IF(begin,ifOperStatus=1)#$
                               :<span class="italic">Up</span>
                  <!-- You can use any number of IF(else, condition) statements
                               as long as the total number is lower than your HTTP
                               configuration setting for mzx number of macros in a
                               block.
                       -->
                  $%IF(else, ifOperStatus=2)#$
                               :<span class="italic">Down</span>
                  $%IF(else, ifOperStatus=3)#$
                               :<span class="italic">Testing</span>
                  <!-- you can use IF(else) without a condition as a default
                               case (for when no conditions are true).
                       -->
                  $%IF(else)#$
                               :<span class="italic">current is ($%ifOperStatus#$)</span>
                  $%IF(end)#$
          </td>
          <td>$%ifLastChange#$ </td>
        </tr>
<!-- $%endRow(ifIndex)#$ -->
<!-- $%REPEAT(end)#$ -->
<!-- $%endTable(ifIndex)#$ -->
</table>
```

17. WMIT EXTENSIONS

1. WMIT Extensions Overview

The WMIT contains features to facilitate some indirectly related development tasks.

These include adding your own custom tool commands and running the MIB Compiler that is provided with Wind River SNMP from within the WMIT.

2. Custom Tools

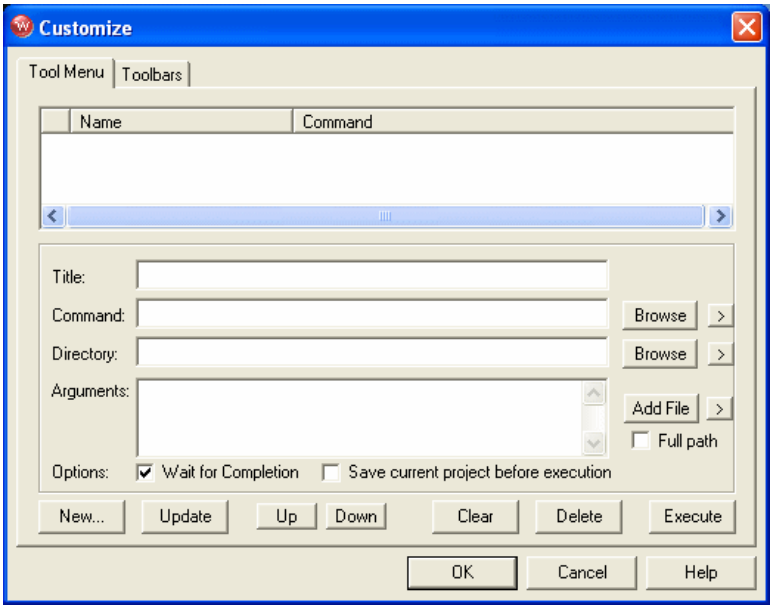
External commands can be added to the WMIT tool menu.

Their settings are preserved in the Windows registry so that they are available to any WMIT session. Up to 20 commands can be defined. The first nine can be directly executed anywhere within the WMIT using the **ALT+1** to **ALT+9** keys. All commands are listed in the **Tools** menu.

To customize the WMIT tool menu, select **Tools > Customize**.

A command is automatically quoted in case of spaces in the path name. The current working directory is set to the **Directory** selection before executing the command, and is returned to the previous directory when it is closed.

Add File inserts selected file names into the argument list. Files can be added with or without the full path name. File names are enclosed in quotes if there are spaces in the name.



The execution of the command is identical to the following examples entered at **Start > Run** or at a console command line.

```
cmd /C "command" arguments     (normal)
cmd /K "command" arguments     (keep open)
```

3. The Wind River SNMP MIB Compiler

The WMIT provides a graphical front end for the command-line MIB compiler.

This graphical front end is a set of Windows dialog boxes used to collect your choice of output formats and options. It uses these parameters and values to construct a MIB compiler command line, submit the command line to the DOS command interpreter for execution, and display the results in a window to review. It can generate all output formats available with the command-line version of the MIB Compiler, except the following: -compat11, -partial, and -skel.

The WMIT also provides separate tools for automatically generating MIBway WindMarks from MIB files. For more information about this feature, see Creating MIBway (SNMP) WindMarks on page 78.

Using the MIB Compiler on page 185 The MIB Compiler uses projects to preserve specific settings; you can use the default project created when the MIB Compiler starts or create a new project.
MIB Compiler Options on page 187 The MIB Compiler allows you to select one or more output modes, including any options that you choose.
Viewing MIB Objects on page 189 The MIB Viewer provides a visual representation of the MIB objects processed by the MIB Compiler.
Error Detection on page 191 The MIB Compiler performs a syntax check on each nonblank line contained in the input files that it processes. When a fatal error is detected, the MIB Compiler stops processing the file and displays an error message.

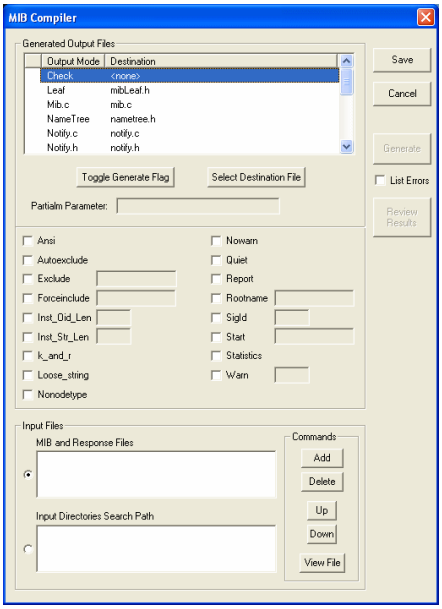
3.1. Using the MIB Compiler

The MIB Compiler uses projects to preserve specific settings; you can use the default project created when the MIB Compiler starts or create a new project.

Select **File > New Project** to create a new project.

When you are finished, you may save the project (**File > Save Project**. You can create any number of projects containing different sets of MIB Compiler parameters for different purposes (for example, one for development, another for builds.)

To start the MIB Compiler, launch the WMIT and select **Tools > SNMP > Run MIB Compiler**.



Output Mode

To select an output mode, click to highlight the mode and click **Toggle Generate Flag**.

For a list of output modes, see MIB Compiler Options on page 187.

MIB and Response Files

The MIB Compiler displays MIB files, control files, or automated response files (**.arf**) in the **MIB and Response Files** text box.

To enter a file, click **Add** and select a file name.

You can enter as many as 30 file name.

File names are processed in the order in which they appear in the **MIB and Response Files** text box. Click **Add**, **Delete**, **Up**, or **Down** to manage the file names.

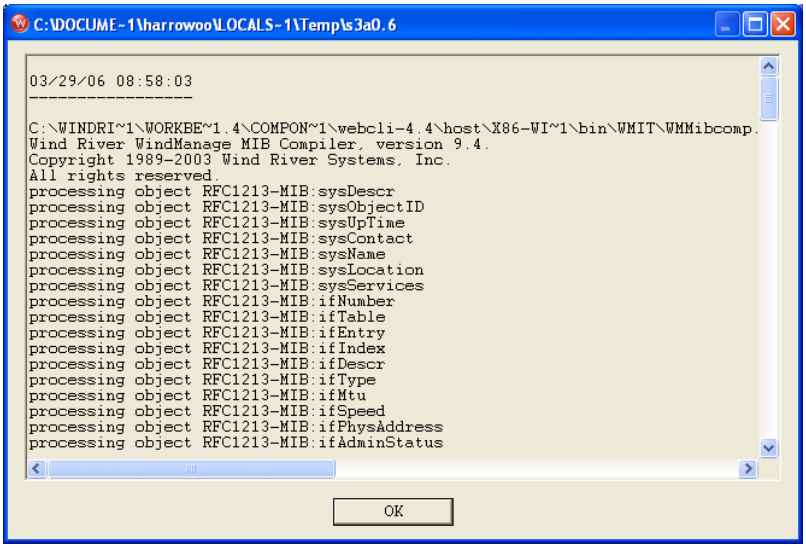
Input Directories Search Paths

If you want to specify search directories for unqualified file names entered in the **MIB and Response Files** text box, enter the directory names in the **Input Directories Search Path** text box.

Directories are searched in the order in which they appear in the **MIB and Response Files** text box. Click **Add**, **Delete**, **Up**, or **Down** to manage the directory names.

Output File Generation

To generate output files and see the results, click **Generate**. One Windows command interpreter appears for each output format chosen, and view displays the results of the operations.



To close the MIB Compiler dialog box without performing any further actions, click **Cancel**.

To review the results of the last operation performed during this session, click **Review Results**. If you exit and restart the MIB Compiler, the results of the last operation will not be available.

To display error and warning messages generated while processing the command line, click **List Errors**.

Parent topic: The Wind River SNMP MIB Compiler on page 185

3.2. MIB Compiler Options

The MIB Compiler allows you to select one or more output modes, including any options that you choose.

To open the MIB compiler, see Using the MIB Compiler on page 185.

A check mark indicates that an output mode is currently selected. The following output modes are available.

Check

Cause MIB Compiler to perform consistency checks on the input files. This option does not generate any output.

Leaf

Generate a C header file containing symbolic constant definitions specifying the last component of every object's OID, enumerated values associated with the object's **SYNTAX**, and any known value bounds.

Mib.c

Generate a C module that defines a MIB tree for an SNMP agent.

NameTree

Generate a C module that defines an array of C structures for SNMP managers.

Notify.c

Provide code to send SNMPv3 notifies, using the Wind River SNMP SNMP_Send_Notify_Name( ) API.

Notify.h

Provide function prototypes for the functions generated by **notify.c**.

Numbers

Generate a listing of every node in the MIB tree, along with each node's OID, and the **SYNTAX** of leaf node MIB objects.

Partialm

Generate a new skeletal MIB file, suitable for compilation by the MIB Compiler. Selecting this format causes the Format Parameter edit box to be enabled for entering module-name, a required parameter.

ReadTree

Generate an ASCII text file, which a manager can load at run-time to look up OIDs by name.

Skel.h

Generate a C header file that declares all functions defined in the C code generated by the stub option and the same input files.

Stub

Generate a C module containing method routines. This option is used with skel.h.

Trap.c

Generate a C module containing SNMP trap message functions.

Trap.h

Generate a C header file defining SNMP trap function prototypes. This option is used with trap.c.

The MIB compiler dialog box also contains the following options. Click the appropriate check box to select an option; a check mark indicates an active option.

Quiet

Suppress progress messages.

Nowarn

Suppress warning messages.

lowMem

Instruct the MIB Compiler to ignore **DESCRIPTION** and **REFERENCE** strings when processing the any **OBJECT-TYPE** or **NOTIFICATION-TYPE** macros.

To change the default destination or output filename, click **Select Destination File**. If the file already exists, it is overwritten with new output.

Output Format Options

The available options are:

Ansi

Use ANSI C prototyping for any generated C code. This option and k_and_r are mutually exclusive.

Autoexclude

Automatically skip unused nodes

Exclude *module-name.object-name1* , ... *module-name.object-nameN*

Remove the specified subtrees from compiled MIBs.

Forceinclude *filename1* , *filename2* , ... *filenameN*

Include the named file in the output file with an #include statement.

Inst_Oid_Len *length*

Set the number of object identifier components reserved for variables that are of type **OCTET STRING**. This option is used with trap.c or man.

Inst_Str_Len *length*

Set the number of octets reserved for variables which are of type **OCTET STRING** with no size constraint. This option is used with trap.c or man.

k_and_r

Use traditional (non-ANSI) prototyping for any C code that is generated. This option and ansi are mutually exclusive.

Loose_string

Allow backward compatibility with MIB files, which contain strings with embedded quote marks and 8-bit characters. These special characters were deprecated in RFC 2578.

WNDRVR

Nonodetype

Produce older versions of node-type support macros.

Nowarn

Suppress warning messages.

Quiet

Suppress progress messages.

Report

Generate simple statistics about the MIB compilation, including scalar leaf, columnar leaf, and table objects.

Rootname *rootname*

Specify the name of the tree root.

SigId *positive-integer*

Specify the number of significant characters in an identifier.

Start *module-name.object-name*

Specify from which node to begin output.

Statistics

Generate simple statistics about the MIB compilation.

-warn *warning_level*

Control which level of warning messages the MIB Compiler displays while processing the input files. The required parameter *warning_level* defines the threshold at which the MIB Compiler displays warning messages. The MIB Compiler displays all messages with an importance level at or below the specified value. The possible values for *warning_level* are as follows:

Value	Description
0	Display no warning messages. Using a warning level of 0 is equivalent to issuing the mibcomp command with the -nowarn option.
1	Display high-level (level 1) warning messages.
2	Display medium- and high-level (level 1 and 2) warning messages. This is the default warning level.
3	Display medium-level (level 1, 2 and 3) warning messages.
4	Display low-level (level 1, 2, 3 and 4) warning messages.
5	Display all warning messages.

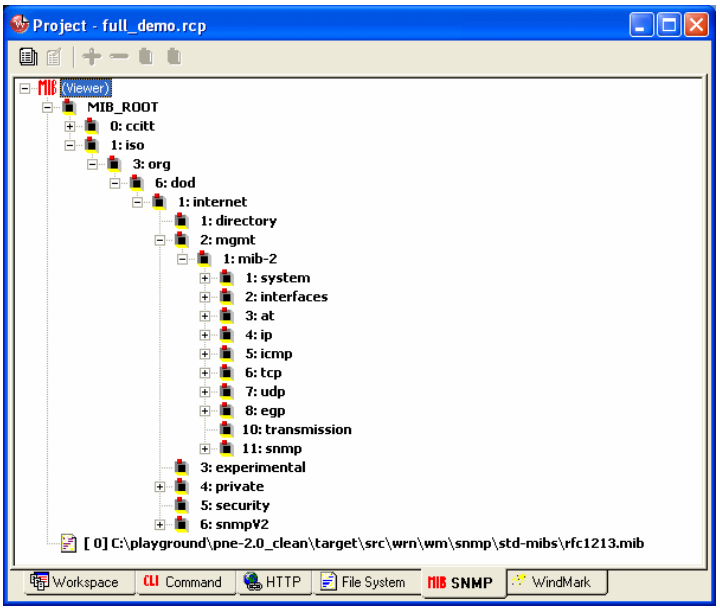
Parent topic: The Wind River SNMP MIB Compiler on page 185

3.3. Viewing MIB Objects

The MIB Viewer provides a visual representation of the MIB objects processed by the MIB Compiler.

About This Task

Using output from **mib.c**, it presents the MIB objects in an expandable tree form.



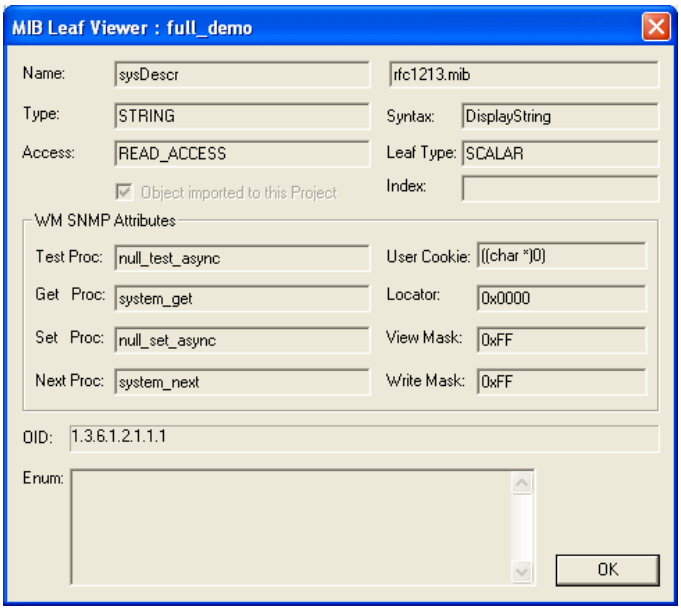
Procedure

1. Click **Generate** to generate a **mib.c** output file.
2. Click **Save**.

> Note: If the file exists and can be parsed, the GUI displays an acknowledgement listing the number of leaf nodes and nonleaf nodes. Click OK to dismiss the dialog box.

If there are valid objects defined in the output file, the MIB Viewer loads and displays those objects in tree-like form.

3. Click the minus (-) and plus (+) signs to expand and collapse objects.
4. Double-click an object to display end-node object characteristics, also known as leaf view.



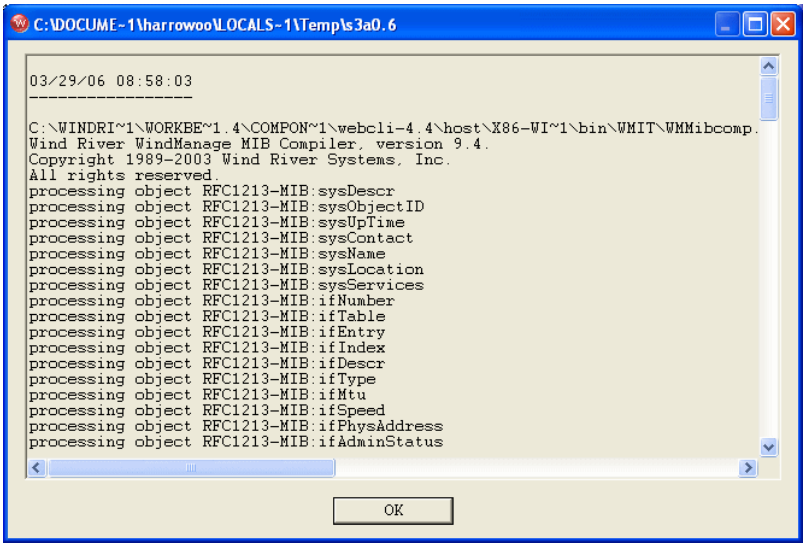
Parent topic: The Wind River SNMP MIB Compiler on page 185

3.4. Error Detection

The MIB Compiler performs a syntax check on each nonblank line contained in the input files that it processes. When a fatal error is detected, the MIB Compiler stops processing the file and displays an error message.

When the MIB Compiler detects a nonfatal error, it continues processing the file and displays a warning message. You can control the level of warnings displayed by specifying a warning value to the MIB compiler. For more information about warning levels, see MIB Compiler Options on page 187. Both types of messages contain the type of error, the file name, and the offending line number.

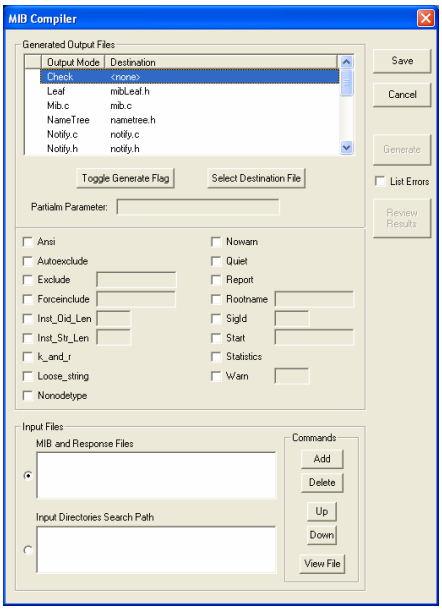
The MIB Compiler lists any errors or warnings detected in the MIB in a temporary file, which it displays in the results view shown below.



Because this is a temporary file, the MIB Compiler deletes it when you exit the GUI. To close this view, click **OK**. If there are any error messages, the Errors and Warnings view appears.

Error Messages

To list error and warning messages, select **List Errors**, which is located below Generate as shown below, and then click Generate.



The MIB Compiler processes the command line and the Results view appears. If there are any error messages when you close the Results view, the Errors and Warnings view appears. This view contains the file name in DOS format, the line number, and a brief error message. To close this view, click **Close**.

To edit the file associated with an error message, select the file name and click **Edit**. You can also double-click the file name. You can edit only one version of a particular file at a time. If more than one file contains an error, select a file name from the list and click **Edit**. You can open a separate MIB file text editor view for each file. If a particular file contains more than one error, only the message selected is highlighted.

Error Correction

The MIB file text editor view contains the following information: the expanded file name is displayed in the title bar, the line number and error message is displayed in the text box, and the contents of the file is displayed in an edit box with an arrow pointing to the offending line. The red arrow disappears when any key is pressed and focus is set to the text.

The MIB Compiler positions the cursor at the line containing the error. To move the cursor, you can use standard cursor movement keys, including the arrows and the **HOME** and **END** keys, in addition to **CTRL+HOME** (to immediately move to the top of the file) and **CTRL+END** (to move to the bottom of the file). Character insertion and deletion occurs at the cursor location. To access standard cut-and-paste operations within in the edit view, click your right mouse button.

To close the view and discard your changes, click **Cancel**. To close the view and save your changes, click **Save**. This renames the previous version of the file as *filename* **.bkp** and saves the new version as the original filename.

Parent topic: The Wind River SNMP MIB Compiler on page 185

18. CONSOLE TOOLS

1. Introduction

Console tools (also known as shell commands) are provided so that you can generate the same files you would with the MCE from a console command line.

The console tools are as follows:

wmbbd

Generates the files **wmb_projName.c** and **wmb_projName.h**.

wmcbd

Generates the files **wmc_projName.c**, **wmc_projName.h** and **wmc_options.h**.

wmwbd

Generates the files **wmw_httpconf.c** and **wmw_httpconf.h**.

wmmbd

Generates the file **wmm_options.h**.

rombuild

Generates a ROMBUILD file system.

The executable files for the console tools are located in the following directories:

DOS executable:

installDir **\vxworks-7\pkgs_v2\app\webcli\tools**_*x.x.x.x* **\bin\x86-win32**

Linux executable:

installDir **/vxworks-7/pkgs_v2/app/webcli\tools**_*x.x.x.x* **/bin/x86-linux2/**

Customizing Code Generation

Customized templates can be generated; however, doing so requires in-depth knowledge of the product. Template customization is outside the scope of this document. For more information, contact Wind River Support.

2. The wmbbd Console Tool

Name

wmbbd – generate the handler code files

Synopsis

wmbbd *projName options*

Where *projName* is your project with ***.rcp** extension.

Description

The wmbbd console tool generates the handler code files **wmb**_*projName*.**c** and **wmb**_*projName*.**h**, and optionally **wm_options.h**. The console tools use a template-driven code generator. You can customize the code generation if you need to. For more information about customizing the code generation templates, see Console Tools on page 193.

wmbbd only works with 4.0 or later **.rcp** project files. Also, if you choose to modify the **.rcp** project files manually, you should check the changes you make with an XML validator before loading them into wmbbd to ensure that your changes are valid. wmbbd does do some checking of the files, but not any sophisticated line-by-line, character-by-character checking.

-h

Show help listings.

-wmb

This option specifies that the Wind River CLI,Web, MIBway global and backplane configuration files **wm_options.h** and **wmb_options.h** should be [re]generated.

-o *outputDirH*

This option tells wmbbd where to place the generated file **wmb**_*projName*.**h**. If specified, this option overrides the project's setting for output files.

-c *outputDirC*

This option tells wmbbd where to place the generated file **wmb**_*projName*.**c**. If specified, this option overrides the project's setting for output files.

-t *template*

This option tells wmbbd to add the file template as a code generation template file. There can be up to four files. The code segments are searched first from the last file, down to the built-in code segments.

-x *extractFile*

This option tells wmbbd to extract the built-in code generation template to the indicated file.

Examples

The command wmbbd **test.rcp** produces the following files and places them in the directories specified by the project's settings:

- **wmb_test.c**
- **wmb_test.h**

The command wmbbd **test.rcp** -o **C:\temp\h** -c **C:\temp** produces the following files:

- **C:\temp\wmb_test.c**
- **C:\temp\h\wmb_test.h**

The command wmbbd **test.rcp** -o **C:\temp\h** -c **C:\temp** -wmb produces the following files:

- **C:\temp\wmb_test.c**
- **C:\temp\h\wmb_test.h**
- **C:\temp\h\wm_options.h**
- **C:\temp\h\wmb_options.h**

3. The wmcbd Console Tool

Name

wmcbd – generate the Wind River CLI command files

Synopsis

wmcbd *projName options*

Where *projName* is your project with ***.rcp** extension.

Description

The command wmcbd is used to generate the Wind River CLI command files **wmc**_*projName* **.c/h**, and optionally the Wind River CLI configuration file **wmc_options.h**.

The file **wmc**_*projectName* **.c** implements all the custom handlers in your Wind River CLI project and the **wmc**_*projectName* **.h** is the header file that contain all the prototypes for the custom handlers defined in **wmc**_*projectName* **.c**.

Note that all your command files are generated to the default directory where your wmcbd executable file is located.

Note: This tool only generates the CLI embedded files. You must also generate the backplane files using wmwbd.

Note that all your command files are generated to the default directory where your wmcbd executable file is located.

-h

This option shows the help listing.

-wmc

This option specifies that the Wind River CLI configuration file **wmc_options.h** should be [re]generated.

-o *outputDirH*

This option tells wmcbd where to place the generated file **wmc**_*projName* **.h**. If specified, this option overrides the project's setting for output files.

-c *outputDirC*

This option tells wmcbd where to place the generated file **wmc**_*projName* **.c**. By default, wmcbd uses the path specified in the project file.

-p *outputDirP*

This option tells wmcbd where to place the finished build of the portfolio file **wmc**_*projName* **.xm**l. Portfolios are not supported for Wind River CLI in this version. The command wmcbd generates an XML portfolio file based on your project. However, this file is not used by Wind River CLI.

-t *template*

This option tells wmcbd to add the file template as a code generation template file. There can be up to four (4) files. The code segments are searched first from the last file, down to the built-in code segments.

-x *extractFile*

This option tells wmcbd to extract the built-in code generation template to the indicated file.

4. The wmwbd Console Tool

Name

wmwbd – generate the Wind River Web Server configuration and portfolio files

Synopsis

wmwbd *projName options*

Where *projName* is your project with ***.rcp** extension.

Description

The command wmwbd is used to generate the Wind River Web Server configuration and portfolio files **wmw_httpconf.c/h** and **wmw_projName.xml**.

> Note: This tool only generates the Web server embedded files. You must also generate the backplane files using wmwbd.

-h

This option shows the help listing.

-o *outputDirH*

This option tells wmwbd where to place the generated file **wmw_httpconf.h**. If specified, this option overrides the project's setting for output files..

-c *outputDirC*

This option tells wmwbd where to place the generated file **wmw_httpconf.h**. If specified, this option overrides the project's setting for output files..

-p *outputDirP*

This option tells wmwbd where to place the finished build of the portfolio file **wmw_projName.xml**. If specified, this option overrides the project's setting for output files.

-t *template*

This option tells wmwbd to add the file template as a code generation template file. There can be up to four (4) files. The code segments are searched first from the last file, down to the built-in code segments.

-x *extractFile*

This option tells wmwbd to extract the built-in code generation template to the indicated file.

5. The rombuild Console Tool

Name

rombuild – build a nonvolatile (memory) file system

Synopsis

rombuild *projName* **.wcp** *options*

Where *projName* **.wcp** is the name of the Wind River Web Server project **.wcp** file. Note that this argument must be the *projName* **.wcp** file, not the *projName* **.rcp** file.

Description

Use rombuild to build a nonvolatile (memory) file system for use with Wind River Web Server.

-o *outputFile*

This option generates a file system image named *outputFile* in the current current directory.

-b *blockSize*

Specifies the sector size for the host NVRAM; for example, -b 512. You can use $ to denote hexadecimal notation for numerical arguments.

-a *address*

Specifies the starting address for host NVRAM. You can use $ to denote hexadecimal notation for numerical arguments; for example, -a $00000000.

-l *length*

otal size of the memory file system, You can use $ to denote hexadecimal notation for numerical arguments; for example, -l $00100000 or -l 1048576.

-i *inputFile*

This option has been deprecated.

-e [b | l]

Endian-ness of the host system. If specified, this value overrides the value specified in the MCE.

-s

Silent mode—suppress build messages.

-f

Include the file manager only.

-c

Build file as ANSI C source code file.

-g

Compress files using GZIP.

-h

Displays this list.

-p *searchPath*

Searches in searchPath for files to include in file system image.

-q *segmentSize*

Separates the final image into blocks of *segmentSize* or less.

Note: Enclose file name or path arguments that contain spaces in quotation marks.

19. VXWORKS PROJECT HELPER SCRIPTS

1. Introduction

The VxWorks project helper scripts, prjCreate.tcl and httpCreate.tcl, are TCL scripts that provide a convenient way to create .wpj projects and makefiles.

prjCreate.tcl creates a new downloadable application module project that includes all configured Wind River CLI,Web, MIBway features. **httpCreate.tcl** creates a new downloadable application module project that creates only the standalone HTTP server.

You can use these tools to facilitate **.wpj** project file creation or command-line builds. These tools use the settings in an **.rcp** project file to create/build a downloadable **.wpj** project file with the appropriate files and include directives.

Note: The scripts and methods in this chapter have been deprecated and will not be supported in future releases.

Before you can use these scripts, your environment must be properly set and your project files must follow naming conventions that are stricter than those imposed by your host operating system.

Environment Variables

The helper scripts require a TCL interpreter (such as the one included in the your distribution) in your search path. The best way to ensure the proper interpreter is available to these scripts is to use the wrenv utility to create a command shell with the correct environment set up.

Project File Naming Requirements

The helper scripts require that the name of the .rcp file (excluding the extension) match the directory name that the file resides in. For example, if you want to use the **prjCreate.tcl** script on a **.rcp** project file named **myProj.rcp**, the project file must reside in a directory named **myProj**. The example **~/projects/myProj/myProj.rcp** is acceptable, whereas the example **~/projects/myProj.rcp** produces an error.

2. The prjCreate.tcl Script

The helper script **prjCreate.tcl** creates a **.wpj** project file and a makefile based on the settings in an **.rcp** project file.

The generated **.wpj** project file name matches the file name (except for the extension) of the **.rcp** project file the project is based on.

The usage for **prjCreate.tcl** is as follows:

```
wtxtcl prjCreate.tcl CPU tool
```

You must specify the CPU and tool arguments as values only, not assignments. For example, the following command is correct:

```
wtxtcl prjCreate.tcl PPC604 gnu
```

Do not enter these values as assignments such as CPU=PPC604 TOOL=gnu. This is incorrect.

Consider the following example

```
$ (/usr/local/projects/myProj)>ls
myProj.rcp myProj.wcp custom.files prjCreate.tcl
$ (/usr/local/projects/myProj)>wtxtcl prjCreate.tcl PPC604 gnu
```

The above command, wtxtcl prjCreate.tcl PPC604 gnu, produces a **.wpj** project file named **myProj.wpj**, a subdirectory **PPC604gnu**, and a makefile in the current directory **/usr/local/projects/myProj**.

Note: This example assumes you have copied the script **prjCreate.tcl** to your project directory. If you do not copy the script to your project directory, you must specify the script file with its path.

Specifying Additional Files to Add to the Project

The script **prjCreate.tcl** adds generated project files and all C source files in your project directory to the generated **.wpj** project. If you need to add other source files or dependency #include directories to the generated **.wpj** project file, you can do so by creating an ASCII text file named **custom.files** in your project directory and adding the files and dependency paths to this file. Each file or dependency path needs to be on its own line; for example:

```
../../src/myfiles/myFile.c
../common/anotherFile.c
../../include/myIncludeDir
```

3. The httpCreate.tcl Script

The **httpCreate.tcl** script is functionally almost identical to **prjCreate.tcl**.

The exception is that it creates the **.wpj** project file and makefile with only the source files and include directives necessary to build the embedded HTTP server as a standalone server; that is without the backplane or other Wind River CLI,Web, MIBway components. Usage and requirements are the same as for **prjCreate.tcl**. The standalone embedded HTTP server is a special (nontypical) use case.

See The prjCreate.tcl Script on page 198, for more information.

20. GLOSSARY

1. Glossary

absolute

A built-in parameter type that has no value associated with it—either it is present or it is not. In the example below, up is a parameter of type absolute that tells what to do to the parameter keyword address.

```
set ip address x.x.x.x up
```

where:

```
set = node
ip = node
address = param #1 keyword
x.x.x.x = param #1 value
up = param #2 (absolute value)
```

backplane

A set of libraries that provide an interface between applications that access resources (consumers) and applications that contain resources (producers). Also referred to as Wind River management backplane, or WMB.

branch node

A branch node (or intermediate node) is a command node that exists hierarchically (on the command tree) between other nodes. A branch node is subordinate to another node (perhaps only the root node) and has nodes subordinate to it.

command

The entire string of text a device an administrator enters at the CLI prompt to execute some function. A command can consist of multiple command nodes and parameters.

command handler

Command handlers link the commands entered on the command line to WindMark names and WindMark handlers.

command node

A single token of a command. For example, show is a command node in the following command:

```
show system uptime
```

command tree

A command tree is the organized hierarchy of commands that your CLI users can access. A command tree consists of three types of command nodes: root nodes, branch nodes (or intermediate nodes), and leaf nodes.

component

An application code segment that communicates with the backplane. Examples include the embedded HTTP server, the embedded telnet daemon, a project that implements a command tree (or a subset of a larger command tree), and a Web portfolio.

A Wind River CLI,Web, MIBway component is distinct from a VxWorks component, which is a program that can be linked and manipulated by VxWorks. A VxWorks component or application can contain several Wind River CLI,Web, MIBway components.

console tools

The console tool set provides the code generation features for command-line use on Windows hosts. The console tools consist of rombuild, wmbbd, wmcbd, wmmbd, and wmwbd.

global command node

Global commands are accessible whether or not you are in an intermediate mode.

handler

The function that executes when a request is made of a particular resource. Some handlers, such as command handlers and WindMark handlers, are implemented by you. Other handlers, such as the MIBway handler, HTML form SELECTED and CHECKED handlers, and built-in global CLI command handlers, are provided as part of Wind River CLI, Web, MIBway.

intermediate mode

Intermediate mode temporarily promotes a node so that the system behaves as if the intermediate node is the root node. For example, if the command node set has an intermediate mode, you can enter that intermediate mode and execute any command as if to the root node You can specify an intermediate mode for any branch node. When you enter the intermediate mode, you can execute commands subordinate to the intermediate node without having to retype the intermediate node each time. Consider the commands:

```
set some system variable01 valueParameter
set some system variable02 valueParameter
set some system variable03 valueParameter
set some system variable04 valueParameter
```

leaf node

If you specify set some system as an intermediate mode, you could then execute each command by only typing **variable01** *valueParameter* , and so forth.

MIB Object

A command node that has no other nodes subordinate to it. When entering commands, only parameters, not other commands, can follow a leaf node.

MIB

A type of resource that manages SNMP data objects. It differs from WindMark resource objects in that it conforms to SNMP instancing and is tied to an SNMP object identifier (OID). MIB objects cannot yet directly represent WindMark objects.

normal command node

Unlike *global command* nodes, a normal command node is only accessible by entering the entire command string, or a partial string when in intermediate mode.

object

A structured instance of a resource whose attributes can be GET or SET.

OID

Object ID.

portfolio

An XML formatted file that contains file system configuration information for the embedded HTTP server. Portfolios are a means of implementing run-time file system configuration updates.

resource

A generic term used to represent any device management element (data objects, files, CLI trees, and so forth). The specific meaning of resource can often be determined by the context in which it is used.

root node

The base of a particular command tree.

RPM

Request processing module.

WindMark

A type of resource that manages data objects. It differs from MIB resource objects in that it is not tied to an OID and can receive a string argument as opposed to a OID instance. WindMarks can represent MIB objects.

Wind River Management Integration Tool

A GUI application that is used to develop resources and generate target code for configuring and building embedded management applications. Also referred to as the WMIT. Not supported for VxWorks 7.